

INTRODUCTION TO MACHINE LEARNING

MIHIR RAO

Contents

1	Introduction	6
1.1	Classification: predicting a category	6
1.2	Different classifiers fit different patterns	7
1.3	Digit images as points in a feature space	7
1.4	Training error, test error, and overfitting	8
1.5	Validation: choosing a model without using the final test	9
1.6	Supervised and unsupervised learning	10
2	Linear Classifiers and Perceptrons	11
2.1	From observations to a decision function	11
2.2	Level sets and linear boundaries	12
2.3	Vectors, dot products, and hyperplanes	13
2.4	Normals and signed distances	14
2.5	The centroid method	15
2.6	Preparing the perceptron objective	16
2.7	Risk and the optimization problem	17
3	Gradient Descent and Hard-Margin Support Vector Machines	19
3.1	Feature space and weight space	19
3.2	Gradient descent on the risk	20
3.3	One-point updates: the perceptron	22

3.4	Learning an offset and convergence	22
3.5	Maximum-margin classification	23
3.6	The hard-margin quadratic program	24
3.7	Constraints in parameter space	25
4	Soft-Margin Support Vector Machines and Features	27
4.1	Allowing margin violations	27
4.2	The soft-margin objective and the choice of C	28
4.3	Nonlinear features and parabolic lifting	30
4.4	Quadratic features	31
4.5	Higher-degree polynomial features	32
4.6	Choosing features without overfitting	33
4.7	Image features from edge orientations	34
5	Machine Learning Abstractions and Optimization	36
5.1	Four levels of a learning system	36
5.2	Unconstrained optimization and convexity	37
5.3	Numerical methods and line search	39
5.4	Smooth equality constraints	40
5.5	Linear programs and feasible regions	41
5.6	Quadratic programs and the SVM objective	42
6	Decision Theory and Generative Models	45
6.1	From class frequencies to posterior probabilities	45
6.2	Loss and the risk of a decision rule	46
6.3	The Bayes decision rule	47
6.4	Optimal risk in the screening example	48
6.5	Continuous features and probability density	48

6.6	Continuous Bayes classification and risk	49
6.7	Decision boundaries in several dimensions	51
6.8	Three ways to build a classifier	52
7	Gaussian Discriminant Analysis and Maximum Likelihood	54
7.1	Gaussian class models and discriminant scores	54
7.2	Quadratic discriminant analysis	55
7.3	Recovering posterior probabilities	56
7.4	Equal variances give linear discriminants	57
7.5	Equal priors recover the centroid method	59
7.6	Maximum likelihood and class priors	60
7.7	Maximum likelihood for an isotropic Gaussian	61
7.8	Fitting QDA and LDA to labeled data	62
8	Eigendecomposition, Quadratic Forms, and Anisotropic Gaussians	64
8.1	Eigenvectors preserve a line	64
8.2	Symmetric matrices and quadratic forms	65
8.3	Eigenvalue signs determine curvature	67
8.4	Constructing a matrix from its axes	68
8.5	Matrix powers and a symmetric square root	69
8.6	Anisotropic Gaussian densities	70
8.7	Precision as a transformed distance	72
8.8	Covariance and Gaussian independence	72
9	Anisotropic Gaussians, QDA, and LDA	75
9.1	Covariance, widths, and precision	75
9.2	Fitting full covariance matrices	76
9.3	Quadratic discriminants and posterior probabilities	78

9.4	Multiclass QDA regions	79
9.5	Shared covariance gives linear discriminants	80
9.6	Projection and model flexibility	82
9.7	Priors, losses, and posterior contours	84
9.8	Design matrices and empirical covariance	85
9.9	Decorrelation, sphering, and whitening	85
10	Linear and Logistic Regression	88
10.1	Regression has three interchangeable parts	88
10.2	Least squares and the design matrix	89
10.3	The normal equations	90
10.4	Pseudoinverse, hat matrix, and projection	91
10.5	Logistic regression fits probabilities directly	93
10.6	The sigmoid derivative and the cost gradient	94
10.7	Gradient descent and stochastic updates	95
10.8	Separable data: decreasing cost without a finite minimum	96
10.9	Revision checklist	98
11	Newton’s Method, ROC Curves, and Polynomial Regression	99
11.1	Polynomial regression is linear in its coefficients	99
11.2	Overfitting, oscillation, and extrapolation	100
11.3	Weighted least squares	101
11.4	Newton’s method uses a local quadratic model	102
11.5	What Newton’s method can and cannot guarantee	104
11.6	Newton’s method for logistic regression	104
11.7	Why the Newton solve is called iteratively reweighted least squares	105
11.8	LDA versus logistic regression	106
11.9	ROC curves measure a threshold tradeoff	107

11.10 Revision checklist 109

LECTURE 1: INTRODUCTION

MACHINE LEARNING uses patterns in data to make predictions. A model describes the patterns we are willing to consider; statistics helps us reason about them; an optimization algorithm fits the model to the data. The amount and quality of the data limit what can be learned. More data can also make methods effective that worked poorly with smaller collections.

The central question is whether a learned pattern works on new examples. Remembering the examples already seen is not enough.

1.1 Classification: predicting a category

Definition 1.1 [CLASSIFICATION]. CLASSIFICATION predicts which category, or CLASS, an input belongs to. A LABEL records the known class of an example. A CLASSIFIER is a rule that assigns a predicted class to an input.

Example: *Credit card default.* For each cardholder, record the balance and annual income, together with whether the cardholder defaulted. These labeled records are TRAINING DATA. Use them to learn a rule that predicts default for a new applicant. The two classes are “default” and “no default.” In this example, balances distinguish the groups more clearly than incomes do, although the groups still overlap.

The measured inputs are FEATURES. Collecting them in a FEATURE VECTOR makes each example a point: two features give a point in a plane. A DECISION BOUNDARY separates regions where the classifier predicts different classes. It separates *predictions*; it need not separate every observed label correctly.

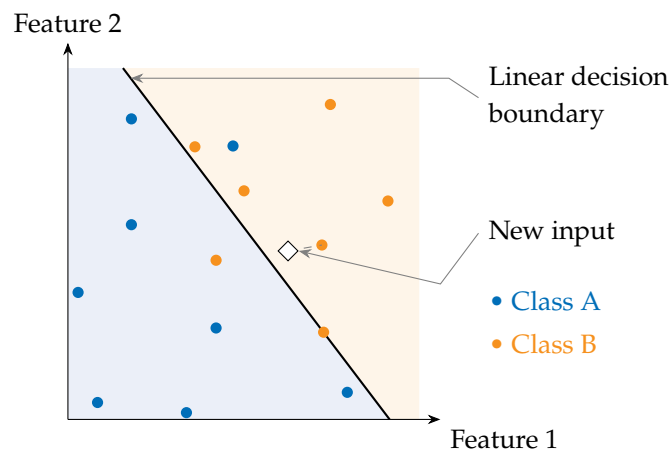


Figure 1: A schematic classification problem. The colored points have known labels; the diamond is a new input. One rule uses the nearest labeled point, while another uses the side of a straight boundary.

1.2 Different classifiers fit different patterns

The same training data can support different prediction rules.

Definition: The 1-NEAREST NEIGHBOR CLASSIFIER finds the training point closest to a new input and returns its label. The k -NEAREST NEIGHBOR CLASSIFIER instead finds the k closest training points and lets their labels vote. For two classes and odd k , one class has a majority.

At $k = 1$, the rule can respond to each individual training point. At $k = 15$, a point's label is balanced against fourteen other labels. This can give less intricate prediction regions, but may misclassify some training points. With distinct training inputs, 1-nearest neighbor classifies each training point correctly when that point itself is included among the possible neighbors: its distance to itself is zero. Conflicting labels at identical inputs would prevent this conclusion.

A LINEAR CLASSIFIER uses a straight decision boundary in two dimensions. This is a stronger restriction on the shape of the pattern. It can miss genuine nonlinear structure, but it also avoids the small islands that a nearest neighbor rule can create around individual points.

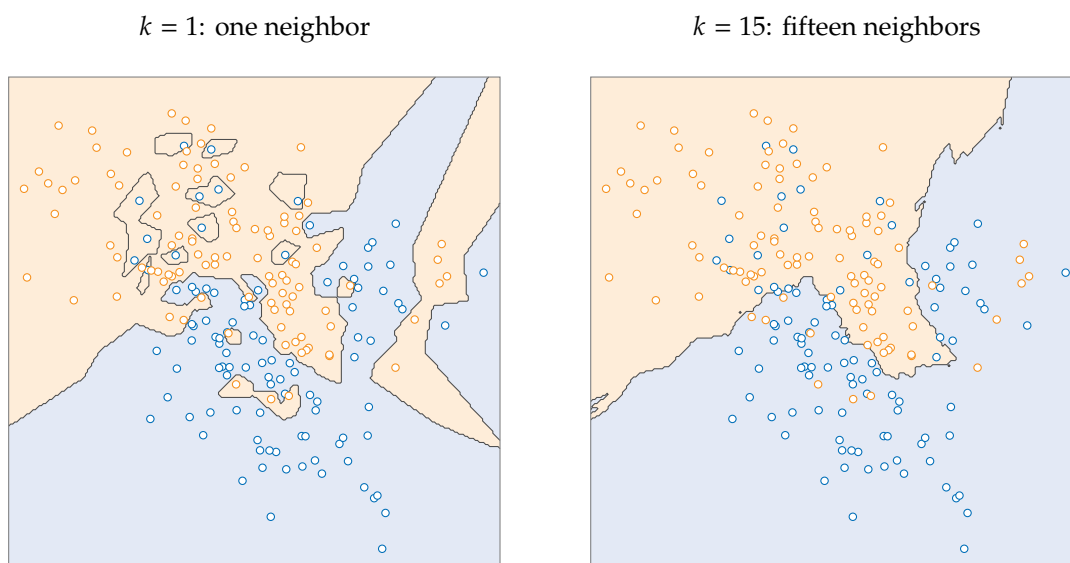


Figure 2: The same 200 labeled points classified with $k = 1$ and $k = 15$. Background colors show predictions; circles show the training labels. At $k = 1$, small regions follow individual points. At $k = 15$, votes produce broader regions, and some circles lie in a region of the other color.

Correctly classifying all training points is therefore not enough to choose a rule. A very intricate boundary may capture accidental details rather than a pattern that will persist in new data.

1.3 Digit images as points in a feature space

Example: *Handwritten 7s and 1s.* Read the sixteen pixels of each 4×4 image in a fixed order to obtain a point in $\mathbb{R}^{16}$. Labeled images of the two digits train a classifier.

4 × 4 pixel array

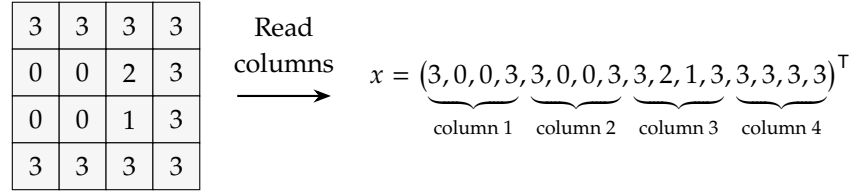


Figure 3: A 4 × 4 array becomes a sixteen-coordinate vector by reading down each column, from left to right. Training and new images must use the same coordinate order.

A **HYPERPLANE** is the higher-dimensional version of a straight line boundary. Thus a linear classifier for these images has a hyperplane as its decision boundary in sixteen-dimensional feature space.

The classification pipeline has two stages:

1. *Training*: compute feature vectors for the labeled images, then learn a classifier from the vectors and their labels.
2. *Prediction*: compute the feature vector of a new image in the same way, then evaluate the learned classifier on that vector.

1.4 Training error, test error, and overfitting

Definition 1.2 [TRAINING AND TEST ERROR]. The **TRAINING ERROR** is the fraction of training examples that the classifier labels incorrectly. The **TEST ERROR** is the fraction labeled incorrectly in a separate collection of new examples that were not used to train the classifier.

If a labeled set contains n examples, the error rate is

$$\text{error rate} = \frac{\text{number of incorrect predictions}}{n}.$$

Training error measures performance on familiar inputs. Test error checks **GENERALIZATION**: performance on new examples. The zero training error of 1-nearest neighbor says little by itself about generalization.

Definition 1.3 [OUTLIERS AND OVERFITTING]. An **OUTLIER** is a point whose label is atypical for its location, such as a borrower who appears solvent but defaults anyway. **OVERFITTING** occurs when sensitivity to outliers or other accidental patterns makes performance on new data worse.

A decision boundary should capture useful patterns. Conversely, **UNDERFITTING** occurs when the rule is too restrictive to capture the useful pattern. Very large k can ignore local distinctions; very small k can react too strongly to individual examples.

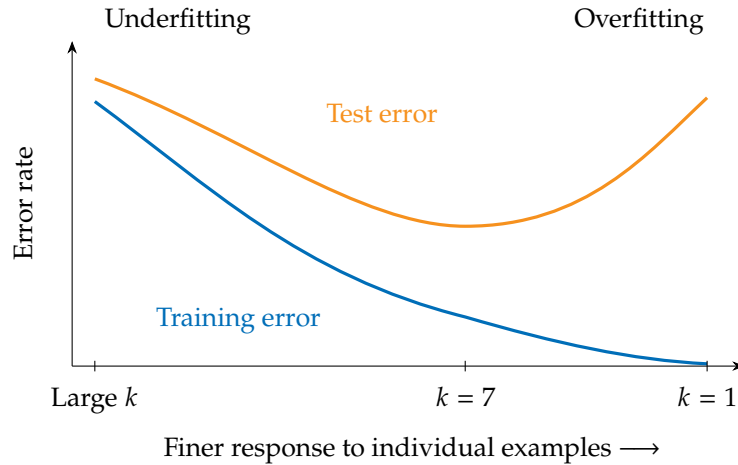


Figure 4: Schematic error trends as k decreases and the rule responds to finer details. Training error can keep falling while error on new data rises. The illustrated comparison has its best test performance near $k = 7$; this is an example, not a universal choice of k . No measured error rates are encoded by these curves.

1.5 Validation: choosing a model without using the final test

Definition: A **HYPERPARAMETER** is a setting that controls how a model is trained or how flexible its prediction rule is. The neighbor count k is one example. **VALIDATION** uses held-back labeled data to choose hyperparameters or compare models.

For a choice such as k , proceed as follows:

1. Hold back a validation set from the labeled data.
2. Train candidate classifiers with different settings, using the training set.
3. Compare their validation errors and choose the setting that performs best on the validation set.

The three sets have different jobs:

Set	Purpose
Training	Learn the model's fitted values, such as its weights.
Validation	Choose settings such as k , or choose among models.
Test	Evaluate the final choice once, after the choices are settled.

Keep the test set out of tuning. If its results influence model selection, it no longer provides the intended final check. A plot of test error against k can explain overfitting; choosing k in a new experiment requires validation data.

Prediction competitions illustrate the same issue. Repeated choices can be guided by public feedback, while a private set supplies the final score. Much stronger public results than private results can indicate overfitting to the feedback used during development.

1.6 Supervised and unsupervised learning

SUPERVISED LEARNING uses examples with target outputs. In classification, the target is a category. In REGRESSION, it is a numerical value. UNSUPERVISED LEARNING seeks structure in the inputs without supplied target labels.

Task	Example question
Classification	Is this email spam?
Regression	How likely is this patient to have cancer?
CLUSTERING	Which DNA sequences are similar to each other?
DIMENSIONALITY REDUCTION	What are common features and differences among faces?

Clustering groups similar inputs. Dimensionality reduction represents important variation with fewer coordinates. Both concern structure in the inputs rather than predicting a supplied label.

Revision. Classification predicts a category from a feature vector. A decision boundary describes where the prediction changes. Nearest neighbor rules and linear rules impose different shapes on that boundary. Generalization is the goal: training accuracy alone cannot establish it.

Common mistakes: choosing a classifier only because its training error is zero; assuming smaller k always helps; changing the pixel order between training and prediction; or using the final test set to tune a model.

Check yourself: Why can 1-nearest neighbor have zero training error and still overfit? Why does a 4×4 image have sixteen coordinates? Which set chooses k , and which set evaluates the final choice? What distinguishes supervised from unsupervised learning?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 1: Introduction](#), PDF pp. 2–6. Explanatory steps are completed and figures redrawn. The neighbor comparison uses the 200 plotted points on PDF p. 4 (originally ESL, Figures 2.1–2.3); error curves are schematic. Administrative material and unrelated text within textbook crops are omitted.

LECTURE 2: LINEAR CLASSIFIERS AND PERCEPTRONS

2.1 From observations to a decision function

Suppose we have n observations, each described by d features, also called `PREDICTORS`. Some observations belong to a class C ; the others do not. Represent an observation by a feature vector $x \in \mathbb{R}^d$. Its coordinates are the input variables used for prediction.

For bank loans, the two features might be income and age. The class C contains borrowers who defaulted. A classifier uses these observations to predict whether a future borrower will default from their income and age. Different rules can produce very different boundaries.

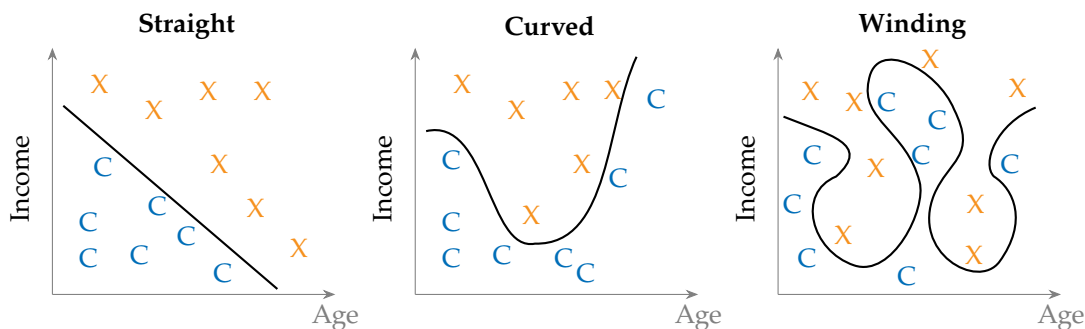


Figure 5: Schematic loan data. A straight boundary suffices on the left. The middle arrangement needs a curved boundary. The winding boundary on the right fits the observed labels closely and risks overfitting.

A boundary partitions the input space into regions assigned different classes. A complicated boundary can fit the training observations well while making poor predictions on future observations.

Definition: A `DECISION FUNCTION`, or predictor function, maps an input x to a real score $f(x)$. Under the convention used here, the classifier predicts C when $f(x) > 0$, and predicts the other class when $f(x) \leq 0$.

For a continuous score that changes sign across its zeros, the decision boundary is the zero set

$$\{x \in \mathbb{R}^d : f(x) = 0\}.$$

This is typically a surface of dimension $d - 1$: a curve in two dimensions or a surface in three dimensions. Some classifiers specify their prediction regions without using a scalar decision function.

2.2 Level sets and linear boundaries

An **ISOSURFACE**, or level set, collects the inputs with the same score. For an **ISOVALUE** c , it is $\{x : f(x) = c\}$. The decision boundary uses $c = 0$; other level sets help us see how the score changes away from it.

Example: Consider the radial score

$$f(x, y) = \sqrt{x^2 + y^2} - 3.$$

The square root is the distance from the origin. Subtracting 3 makes the score zero on the circle of radius 3, negative inside that circle, and positive outside it. Its graph is a cone. Its level set for $c \geq -3$ has radius $c + 3$; at $c = -3$ it reduces to the origin.

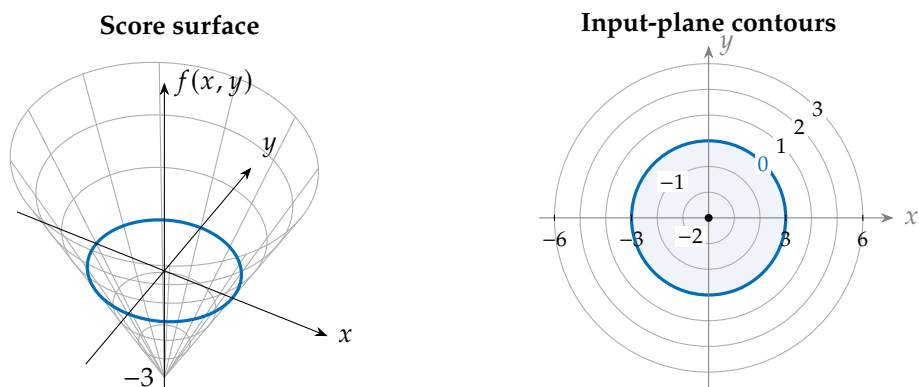


Figure 6: The same score viewed as a surface and as contours in the input plane. The blue circle is $f = 0$. The contours $f = -2, -1, 1, 2, 3$ have radii 1, 2, 4, 5, 6, respectively.

In three dimensions, the analogous score $f(x) = \|x\| - 3$ has concentric spherical level sets. The sphere of radius 3 is its decision boundary. The score is still a scalar, even though its input has three coordinates.

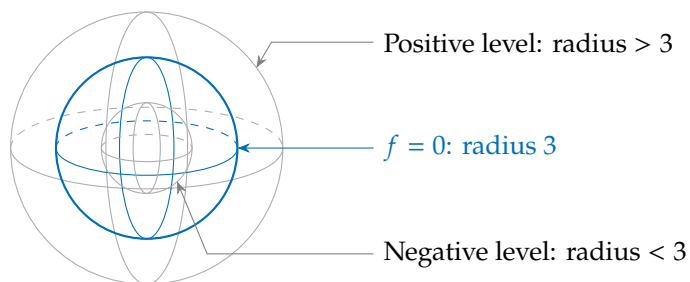


Figure 7: Spherical level sets in three-dimensional input space. Only the middle sphere, where $f = 0$, is the decision boundary.

A **LINEAR CLASSIFIER** has a flat decision boundary: a line in two dimensions, a plane in three dimensions, or a hyperplane in higher dimensions. We will describe it using $f(x) = w \cdot x + \alpha$.

2.3 Vectors, dot products, and hyperplanes

Write a vector as a column. For example, a point with five features is

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = [x_1 \ x_2 \ x_3 \ x_4 \ x_5]^T.$$

The transpose T exchanges rows and columns. Lowercase Roman letters usually denote vectors; uppercase letters often denote matrices, sets, or random variables. Greek letters such as α often denote scalars. Here n counts observations, d counts features, and i, j, k are indices. These are conventions, so context still matters.

The INNER PRODUCT, or dot product, multiplies corresponding coordinates and adds the results:

$$x \cdot y = \sum_{j=1}^d x_j y_j = x^T y.$$

The EUCLIDEAN NORM measures a vector's length:

$$\|x\| = \sqrt{x \cdot x} = \sqrt{\sum_{j=1}^d x_j^2}.$$

To NORMALIZE a nonzero vector, divide it by its length. The result $x/\|x\|$ is a UNIT VECTOR, with length 1. For nonzero x, y and the angle θ between them,

$$\cos \theta = \frac{x \cdot y}{\|x\| \|y\|} = \frac{x}{\|x\|} \cdot \frac{y}{\|y\|}.$$

Thus the sign of the dot product tells us whether the angle is acute, right, or obtuse.

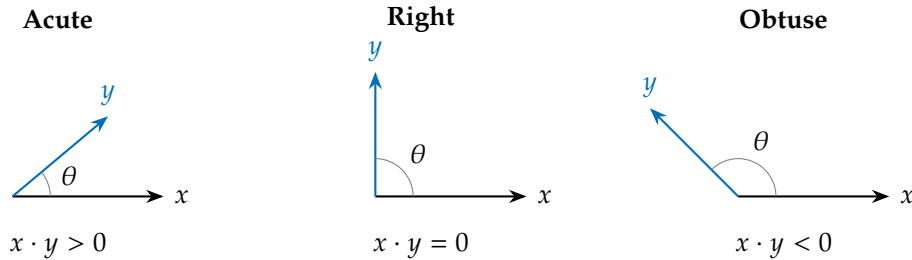


Figure 8: Dot products distinguish acute, perpendicular, and obtuse directions. The vectors must be nonzero for their angle to be defined.

Definition 2.1 [HYPERPLANE]. For $w \neq 0$, the zero set of $f(x) = w \cdot x + \alpha$ is

$$H = \{x \in \mathbb{R}^d : w \cdot x = -\alpha\}.$$

It is a flat, infinite set of dimension $d - 1$ that divides $\mathbb{R}^d$ into two half-spaces.

The score is commonly called linear. More precisely, it is **AFFINE** when the offset α is nonzero; its boundary is still a hyperplane.

2.4 Normals and signed distances

The weights w determine which direction is perpendicular to the boundary. The offset α determines its position.

Theorem 2.2 [NORMAL TO A HYPERPLANE]. If $x, y \in H = \{u : w \cdot u = -\alpha\}$, then $w \cdot (y - x) = 0$.

Proof. Subtract the two equations defining membership in H . Since $w \cdot x = w \cdot y = -\alpha$,

$$w \cdot (y - x) = w \cdot y - w \cdot x = -\alpha - (-\alpha) = 0.$$

Every displacement between points on the hyperplane is therefore perpendicular to w . □

The vector w is a **NORMAL VECTOR** of H . Parallel hyperplanes can share the same normal while having different offsets.

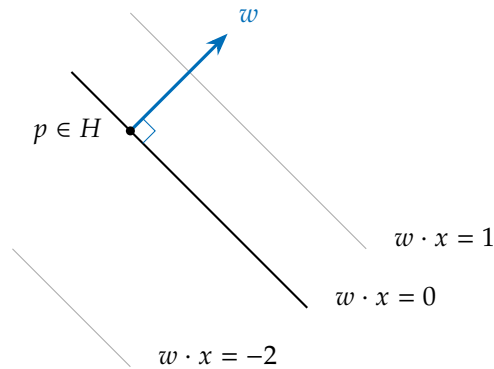


Figure 9: Parallel lines with a common normal w . Moving in the direction of w increases $w \cdot x$; moving along a line leaves it unchanged. The drawing is schematic.

The **SIGNED DISTANCE** is positive on the side toward which w points, and negative on the opposite side. For $w \neq 0$, it equals

$$s(x, H) = \frac{w \cdot x + \alpha}{\|w\|}.$$

To see this, choose a point $p \in H$ and the unit normal $u = w/\|w\|$. The perpendicular component of $x - p$ is

$$u \cdot (x - p) = \frac{w \cdot x - w \cdot p}{\|w\|} = \frac{w \cdot x + \alpha}{\|w\|}.$$

When $\|w\| = 1$, the score itself is the signed distance. Otherwise, divide both w and α by $\|w\|$ to obtain that interpretation without changing the predictions. The ordinary distance from the origin to H is $|\alpha|/\|w\|$. In particular, H passes through the origin exactly when $\alpha = 0$.

The entries of w and the offset α are the classifier's **WEIGHTS**, also called parameters or regression coefficients. Data are **LINEARLY SEPARABLE** if a hyperplane puts the two classes on opposite sides. When no such hyperplane exists, a linear classifier cannot classify all training points correctly, though it may still predict most points well.

2.5 The centroid method

A simple rule summarizes each class by its average location. Let μ_C be the mean of the training vectors in C , and μ_X the mean of those outside C . These means are the two **CENTROIDS**; both classes must contain at least one point.

The **CENTROID METHOD** places the boundary halfway between these means, perpendicular to the segment joining them. Assuming $\mu_C \neq \mu_X$, set

$$w = \mu_C - \mu_X, \quad m = \frac{\mu_C + \mu_X}{2}, \quad \alpha = -w \cdot m.$$

Its score is

$$f(x) = (\mu_C - \mu_X) \cdot x - (\mu_C - \mu_X) \cdot \frac{\mu_C + \mu_X}{2} = w \cdot (x - m).$$

The midpoint m is on the boundary because $f(m) = 0$. The normal is the direction from μ_X to μ_C , so the positive side contains μ_C .

This is also a nearest-centroid rule. Expanding the two squared distances and canceling $x \cdot x$ gives

$$\begin{aligned} \|x - \mu_X\|^2 - \|x - \mu_C\|^2 &= 2(\mu_C - \mu_X) \cdot x + \|\mu_X\|^2 - \|\mu_C\|^2 \\ &= 2w \cdot (x - m) = 2f(x). \end{aligned}$$

Thus $f(x) > 0$ means that x is closer to μ_C than to μ_X . Equal distances give score zero and use the other-class tie convention.

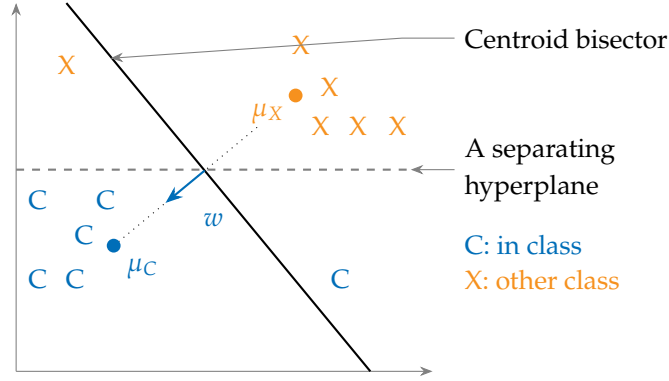


Figure 10: Centroid classification can fail on separable data. The centroid bisector misclassifies one point of each class, although the dashed horizontal line separates all points. The centroids and bisector are calculated from the schematic point positions.

Two means can hide important details of the data's arrangement. The rule can work well when the classes form two Gaussian clusters, but it need not find a separating hyperplane even when one exists. Changing only α translates the boundary without rotating it; choosing a better offset can sometimes reduce training errors. If the centroids coincide, their difference is zero and this construction does not define a separating hyperplane.

2.6 Preparing the perceptron objective

The PERCEPTRON, introduced by Frank Rosenblatt in 1957, uses numerical optimization to learn a linear classifier. Its training procedure can find a separator for linearly separable points, though it may be slow. The starting idea is to assign a penalty to violated classification constraints, then change the weights to reduce it.

Let the training vectors be $X_1, \dots, X_n \in \mathbb{R}^d$, with labels

$$y_i = \begin{cases} +1, & X_i \in C, \\ -1, & X_i \notin C. \end{cases}$$

The capital X_i identifies one observation. If observations are stored as rows of a data matrix X , its i th row is $X_i^\top$.

For now, restrict the boundary to pass through the origin: $\alpha = 0$. Then the score is $z_i = X_i \cdot w$. A positive label should have a positive score; a negative label should have a negative score. The relaxed, nonstrict constraints are

$$X_i \cdot w \geq 0 \quad (y_i = +1), \quad X_i \cdot w \leq 0 \quad (y_i = -1).$$

Multiplying by the label combines the two cases:

$$y_i X_i \cdot w \geq 0, \quad i = 1, \dots, n.$$

Each inequality is a **CONSTRAINT**. The product $y_i z_i$ checks whether the score has the desired sign. Strictly positive products put every point on the correct side; a zero product puts a point on the boundary. Also, z_i is a signed distance only if $\|w\| = 1$.

Definition: The perceptron **LOSS FUNCTION** penalizes a negative label-score product:

$$L(z, y) = \begin{cases} 0, & yz \geq 0, \\ -yz, & yz < 0, \end{cases} \quad \text{equivalently} \quad L(z, y) = \max\{0, -yz\}.$$

Correctly signed scores have zero loss. A score with the wrong sign has positive loss, which increases as it moves farther in the wrong direction.

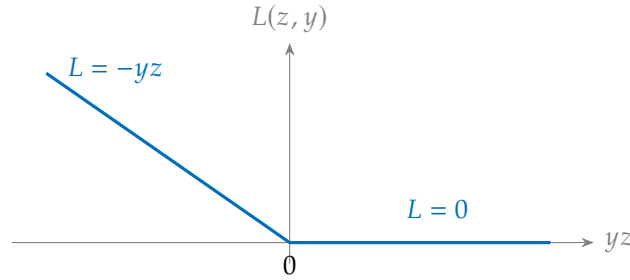


Figure 11: Loss as a function of the label-score product yz . Negative products are penalized linearly; zero and positive products have zero loss.

2.7 Risk and the optimization problem

Training combines the losses of all observations. The **RISK FUNCTION**, also called the objective or cost function, is their average:

$$R(w) = \frac{1}{n} \sum_{i=1}^n L(X_i \cdot w, y_i).$$

Let $V = \{i : y_i X_i \cdot w < 0\}$ be the indices of violated constraints. All other terms have zero loss, so

$$R(w) = -\frac{1}{n} \sum_{i \in V} y_i X_i \cdot w.$$

Each term in this sum is positive. Hence $R(w) > 0$ exactly when at least one of the nonstrict constraints is violated. The proposed optimization problem is

$$\underset{w \in \mathbb{R}^d}{\text{minimize}} \quad R(w).$$

The perceptron uses gradient-based steps to change w and reduce this objective. Here we have set up the constraints and the function to optimize.

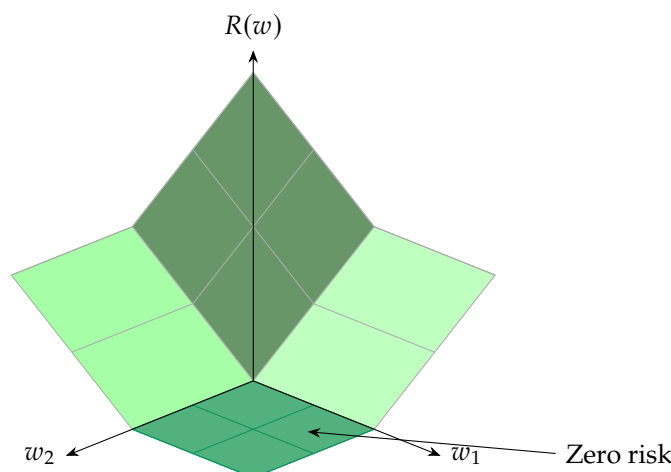


Figure 12: Schematic risk surface for two weights. Flat green regions have $R = 0$; sloping regions have positive loss. The shape depends on the training points and labels. No measured values are shown.

Note. Zero loss includes boundary ties. In particular, $w = 0$ gives $y_i X_i \cdot w = 0$ for every observation, so $R(0) = 0$, even though it does not give a useful separator. A zero score also predicts the other class under our convention, so a positive-label point on the boundary is not correctly classified. Therefore, zero risk certifies the *nonstrict inequalities*, not strict separation. Strictly positive products $y_i X_i \cdot w$ certify that all points lie on the correct sides.

Revision. The zero set of $w \cdot x + \alpha$ is a hyperplane with normal $w \neq 0$. Its signed distance is $(w \cdot x + \alpha)/\|w\|$. The centroid method uses the perpendicular bisector of the class means. The perceptron objective averages penalties for negative label-score products.

Common mistakes: treating a score as a distance without normalizing w ; forgetting the absolute value in the ordinary distance from the origin; assuming centroid classification succeeds on every separable data set; or treating zero perceptron risk as proof of strict separation.

Check yourself: Why is w perpendicular to the boundary? How does changing α move it? Why does the centroid score compare distances to the means? Why does the loss vanish at $w = 0$?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 2: Linear Classifiers and Perceptrons](#), PDF pp. 1–6 (printed pp. 7–12). Figures are redrawn. The signed-distance derivation and centroid-distance identity complete the geometric arguments. The absolute value for ordinary distance and the zero-margin caveat make the stated formulas and classification convention consistent. Boundary and risk drawings are schematic.

LECTURE 3: GRADIENT DESCENT AND HARD-MARGIN SUPPORT VECTOR MACHINES

3.1 Feature space and weight space

Finding a linear classifier means finding a hyperplane that separates the labeled observations. Optimization describes the same task by searching for a point: the vector of weights defining that hyperplane. These are two different spaces, even when both have d coordinates.

In **FEATURE SPACE**, points are inputs $x \in \mathbb{R}^d$. In **WEIGHT SPACE**, points are candidate weights $w \in \mathbb{R}^d$. For now, use the origin-passing score $f(x) = w \cdot x$ and labels $y_i \in \{+1, -1\}$. The relaxed constraints are $y_i X_i \cdot w \geq 0$. We use the total perceptron loss

$$R(w) = \sum_{i=1}^n \max\{0, -y_i X_i \cdot w\} = - \sum_{i \in V} y_i X_i \cdot w, \quad V = \{i : y_i X_i \cdot w < 0\}.$$

Dropping the earlier factor $1/n$ does not change the minimizers; it only rescales the objective and its gradient.

Feature space	Weight space
Boundary $\{z : w \cdot z = 0\}$	A chosen point w
Input point $x \neq 0$	Hyperplane $\{v : x \cdot v = 0\}$

The correspondence comes from the symmetry of the dot product:

$$x \in \{z : w \cdot z = 0\} \iff w \cdot x = 0 \iff w \in \{v : x \cdot v = 0\}.$$

Thus an input lies on the classifier's boundary exactly when the weight vector lies on that input's constraint boundary. A chosen w represents a classifier; positive multiples of w represent the same classifier.

The inequality $x \cdot w \geq 0$ puts x on the side toward w in feature space. In weight space, it puts w on the side toward x . For a negative-label point, the required side reverses. All observations together restrict w to an intersection of half-spaces.

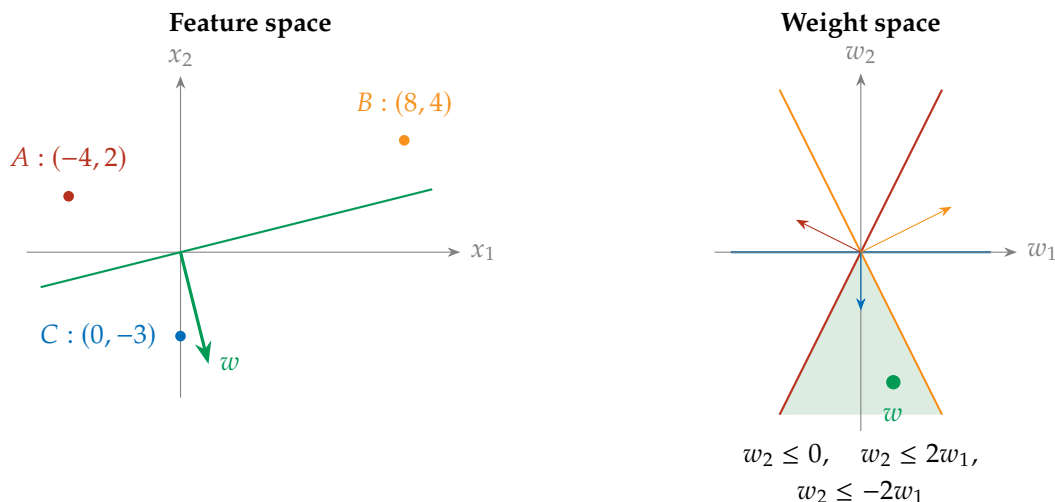


Figure 13: The same three-point arrangement in two spaces. Left: a separating line with normal $w = (1, -4)$. Right: each input defines a line through the origin. The green region satisfies all three nonstrict label constraints; its interior gives strict separation. Coordinates specify the schematic geometry.

3.2 Gradient descent on the risk

Each violated constraint contributes a linear term to R . As w crosses a constraint boundary, that term enters or leaves the sum. Consequently the risk surface consists of flat pieces joined at creases. The creases occur along the input-defined hyperplanes in weight space.

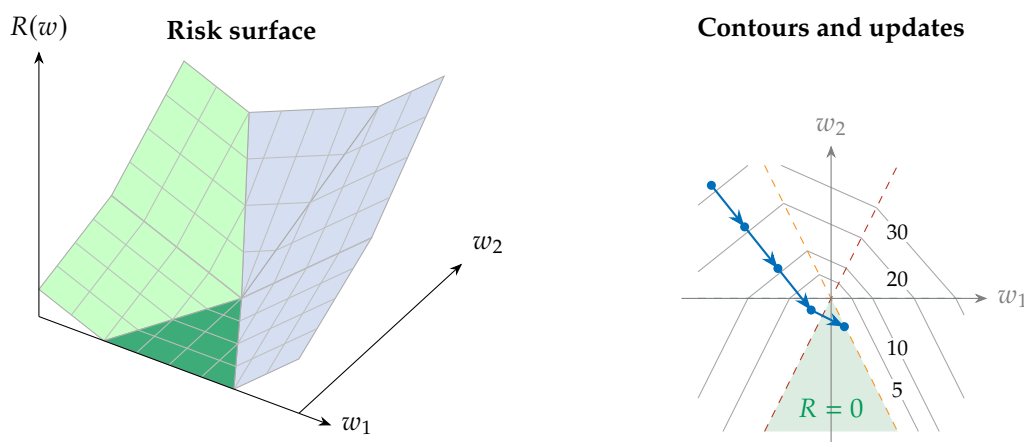


Figure 14: Risk for the three points above, viewed as a surface and as contours in weight space. The bottom wedge has zero risk. Blue arrows show batch updates toward that wedge. Contour labels are values of the total loss, not classification error rates.

GRADIENT DESCENT changes the current weights in the direction opposite the gradient. At a differentiable point, the gradient points toward the steepest local increase of the objective:

$$\nabla R(w) = \begin{bmatrix} \partial R / \partial w_1 \\ \partial R / \partial w_2 \\ \vdots \\ \partial R / \partial w_d \end{bmatrix}.$$

For a fixed vector z , differentiating $z \cdot w = \sum_j z_j w_j$ with respect to w_j gives z_j , so $\nabla_w(z \cdot w) = z$. Within a region where the violated set V is fixed,

$$\nabla R(w) = - \sum_{i \in V} y_i X_i.$$

Subtracting this gradient gives the update

$$w \leftarrow w + \epsilon \sum_{i \in V} y_i X_i,$$

where $\epsilon > 0$ is the STEP SIZE, or learning rate.

Algorithm 3.1: Batch updates for the perceptron risk

Input: Labeled vectors (X_i, y_i) ; $\epsilon > 0$; nonzero initial weights w_0 .

```

1  $w \leftarrow w_0$ 
2 while  $R(w) > 0$  do
3    $V \leftarrow \{i : y_i X_i \cdot w < 0\}$ 
4    $w \leftarrow w + \epsilon \sum_{i \in V} y_i X_i$ 
5 return  $w$ 
```

Computing all scores and summing the violated vectors takes $O(nd)$ time per iteration. A possible nonzero start is $w_0 = y_i X_i$ for any nonzero input. The learning rate is usually chosen empirically: a small step may make slow progress, while a large step can jump across a low-risk region and cause oscillation.

At a crease, the ordinary gradient need not exist. Tied points use the inactive, zero-loss branch. The batch procedure stops at nonstrict feasibility; $R = 0$ does not certify strict separation.

3.3 One-point updates: the perceptron

Instead of processing every violation at once, update using one observation. This is the STOCHASTIC GRADIENT DESCENT idea: use the gradient of one example's loss rather than the full sum. For a negative label-score product,

$$\nabla_w L(X_i \cdot w, y_i) = -y_i X_i, \quad w \leftarrow w + \epsilon y_i X_i.$$

The resulting procedure is the perceptron learning algorithm. Each weight update takes $O(d)$ time, excluding the search for a point that needs an update. A full scan to find such a point can still cost $O(nd)$.

For strict separation, treat boundary ties as unfinished as well as negative products. At a tie, the update selects the penalized branch's one-sided derivative. This also permits the convenient start $w = 0$.

Algorithm 3.2: Perceptron with boundary ties included

Input: Labeled vectors (X_i, y_i) ; fixed $\epsilon > 0$.

```

1  $w \leftarrow 0$ 
2 while there exists  $i$  with  $y_i X_i \cdot w \leq 0$  do
3   | choose an index  $i$  with  $y_i X_i \cdot w \leq 0$ 
4   |  $w \leftarrow w + \epsilon y_i X_i$ 
5 return  $w$ 
```

The correction moves toward a positive-label input and away from a negative-label input. For the selected point, its label-score product increases by

$$y_i X_i \cdot (w + \epsilon y_i X_i) - y_i X_i \cdot w = \epsilon \|X_i\|^2,$$

because $y_i^2 = 1$. Other points' scores can improve or worsen, so one correction need not finish the training problem.

3.4 Learning an offset and convergence

An origin-passing boundary is restrictive. To learn a general affine score, append a constant coordinate to every input:

$$\tilde{x} = \begin{bmatrix} x \\ 1 \end{bmatrix}, \quad \tilde{w} = \begin{bmatrix} w \\ \alpha \end{bmatrix}, \quad \tilde{w} \cdot \tilde{x} = w \cdot x + \alpha.$$

For a two-feature input, this reads

$$\begin{bmatrix} w_1 & w_2 & \alpha \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ 1 \end{bmatrix} = w_1 x_1 + w_2 x_2 + \alpha.$$

The augmented inputs lie on the plane $x_{d+1} = 1$ in $\mathbb{R}^{d+1}$. An origin-passing hyperplane there gives an affine boundary on this plane. Applying the perceptron to $(\tilde{X}_i, y_i)$ updates w and α together.

Rosenblatt's 1957 perceptron was designed for hardware image recognition. The Mark I Perceptron Machine processed 20×20 pixel images and used electric motors to adjust weights. The learning rule is also an ONLINE ALGORITHM: new observations can be added while updates continue, without restarting the weights.

For a separating hyperplane, its MARGIN is the distance to the nearest training point. A larger margin means a wider empty gap between the two classes. This distance also controls the perceptron's update bound.

Theorem 3.1 [PERCEPTRON CONVERGENCE]. *Suppose there is a unit vector u with $y_i X_i \cdot u \geq \gamma > 0$ for every i , and let $r = \max_i \|X_i\|$. Starting at zero, the tie-inclusive perceptron with a fixed positive step size finds a strict separator in $O(r^2/\gamma^2)$ updates.*

The statement requires strict linear separability in the representation being used. For an affine classifier, apply it to the augmented inputs; their lengths and margin are measured in the augmented space. It is not a guarantee for arbitrary stochastic gradient methods or nonseparable data. With a fixed nonzero initialization, the learning rate can affect the update count. With zero initialization and the same sequence of selected points, changing a constant learning rate only rescales the weights.

3.5 Maximum-margin classification

The perceptron finds a separator but does not choose the one with the widest gap. A MAXIMUM-MARGIN CLASSIFIER explicitly maximizes the distance from the boundary to the closest training observation. Assume that both classes are nonempty and strictly linearly separable.

For $f(x) = w \cdot x + \alpha$ with $w \neq 0$, the ordinary distance from X_i to the boundary is

$$\frac{|w \cdot X_i + \alpha|}{\|w\|}.$$

For a correctly separated point, this is also $y_i(w \cdot X_i + \alpha)/\|w\|$. We require

$$y_i(w \cdot X_i + \alpha) \geq 1, \quad i = 1, \dots, n.$$

Positive-label points must have score at least 1; negative-label points must have score at most -1 . Using 1 in place of 0 prevents the zero-weight solution when both classes are present.

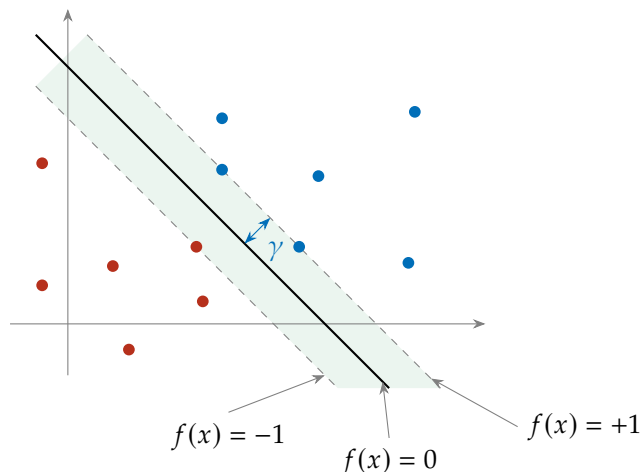


Figure 15: Blue points are in C ; red points are in the other class. A separating boundary lies halfway between the score levels -1 and $+1$. The closest points touch these two lines; the open slab between them contains no training point. The drawing is schematic.

The constraints imply the margin bound

$$\gamma(w, \alpha) = \min_i \frac{|w \cdot X_i + \alpha|}{\|w\|} \geq \frac{1}{\|w\|}.$$

Why does minimizing the norm maximize the actual margin, rather than just this lower bound? Start with any strict separator and let $\beta = \min_i y_i(w \cdot X_i + \alpha) > 0$. Dividing both w and α by β leaves the boundary unchanged and makes the smallest label-score product exactly 1. Under this normalization, $\gamma = 1/\|w\|$. Every strict separator can be represented this way.

3.6 The hard-margin quadratic program

The maximum-margin problem therefore becomes

$$\begin{aligned} & \underset{w \in \mathbb{R}^d, \alpha \in \mathbb{R}}{\text{minimize}} && \|w\|^2 \\ & \text{subject to} && y_i(X_i \cdot w + \alpha) \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

This is a **QUADRATIC PROGRAM**: the objective is quadratic and the constraints are linear. It has $d + 1$ variables and n constraints. Minimizing $\|w\|^2$ and minimizing $\|w\|$ give the same minimizers, but the squared norm is smooth everywhere, including at $w = 0$. The offset α is a variable in the constraints; it is not part of the squared-norm objective.

For finite, strictly separable data with both classes present, this unique solution defines a **HARD-MARGIN SUPPORT VECTOR MACHINE (SVM)**. Nonseparable data make the constraints infeasible.

Proposition 3.2 [MARGIN AT THE OPTIMUM]. *At the optimum, at least one constraint is an equality. Hence the margin is exactly $1/\|w\|$, and the empty slab between scores -1 and $+1$ has width $2/\|w\|$.*

Proof. If every label-score product exceeded 1, their finite minimum would be some $\beta > 1$. Replacing (w, α) by $(w/\beta, \alpha/\beta)$ would keep every constraint satisfied and reduce the objective from $\|w\|^2$ to $\|w\|^2/\beta^2$. This contradicts optimality. Thus the minimum product is 1, which gives the stated distance and slab width. $\square$

3.7 Constraints in parameter space

With an offset, the parameter space has coordinates (w_1, w_2, α) for two-feature inputs. Each training point gives a plane bounding a half-space. In the three-point arrangement used earlier,

$$X_C = (0, -3), \quad X_A = (-4, 2), \quad X_B = (8, 4),$$

with labels $+1, -1, -1$, the constraints are

$$\alpha \geq 1 + 3w_2, \quad \alpha \leq -1 + 4w_1 - 2w_2, \quad \alpha \leq -1 - 8w_1 - 4w_2.$$

The feasible parameters lie above the first plane and below the other two. Their optimal intersection is

$$(w_1, w_2, \alpha) = \left(\frac{1}{17}, -\frac{6}{17}, -\frac{1}{17} \right).$$

Substitution gives scores $+1, -1, -1$ for the three inputs, so all three constraints are equalities.

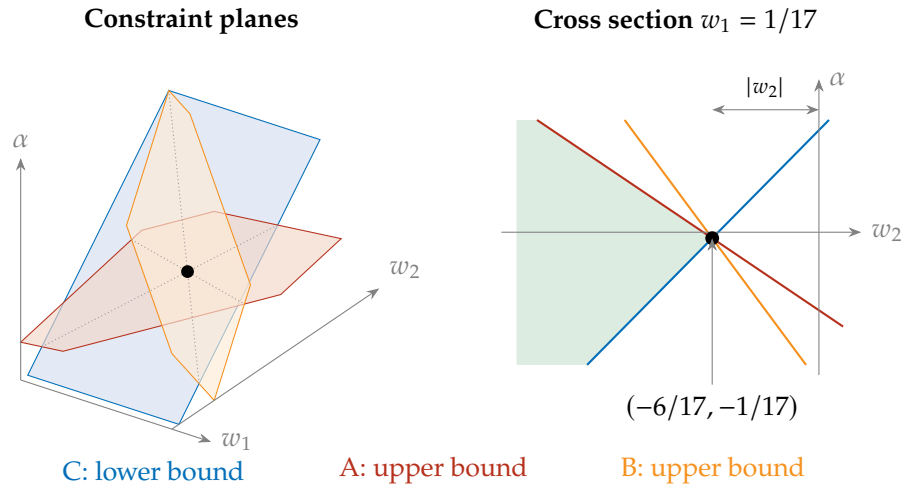


Figure 16: The three constraint planes and the cross section $w_1 = 1/17$. The green region is feasible, and the black point is the optimum. In this cross section, minimizing $w_1^2 + w_2^2$ minimizes horizontal distance to the α axis, not distance to the origin in the (w_2, α) plane.

Revision. An input point becomes a constraint hyperplane in weight space. Batch descent sums all violated vectors; the perceptron uses one point per update. Appending a constant coordinate lets the same rule learn an offset. A hard-margin SVM minimizes $\|w\|^2$ under unit label-score constraints, giving margin $1/\|w\|$ at the optimum.

Common mistakes: confusing input coordinates with weights; omitting boundary ties from a strict-separation stopping rule; counting the search for a violated point as part of an $O(d)$ update; penalizing α in the SVM norm; or confusing margin with the full slab width.

Check yourself: Why is the batch update a sum of $y_i X_i$? How does the extra coordinate update the offset? What assumptions does perceptron convergence need? Why must an optimal SVM constraint be tight, and why is its slab twice the margin?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 3: Perceptron Learning; Maximum Margin Classifiers](#), PDF pp. 1–5 (printed pp. 13–17). Figures are redrawn; the three-point coordinates specify the supplied geometric arrangement and recover its $w_1 = 1/17$ cross section. The perceptron includes boundary ties and zero initialization to make the strict-separation convergence statement precise. The normalization argument and tight-constraint proof complete the margin calculation. Parameter-space distances are clarified because the objective excludes the offset. The historical photograph and press-conference quotation are omitted.

LECTURE 4: SOFT-MARGIN SUPPORT VECTOR MACHINES AND FEATURES

4.1 Allowing margin violations

A hard-margin SVM requires every training observation to be correctly classified and outside the open margin slab. This causes two problems: nonseparable data make the constraints infeasible, and one unusual observation can move the fitted boundary substantially.

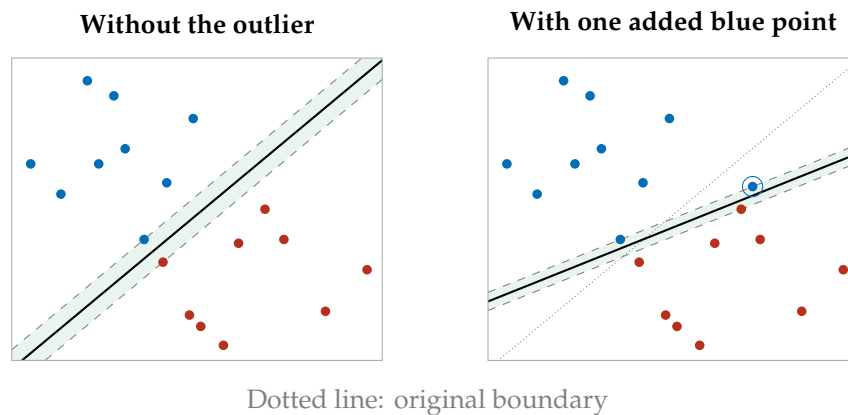


Figure 17: Schematic redraw of the outlier arrangement. The circled blue point forces a substantial change in the hard-margin boundary. The dotted line on the right shows the original boundary; the pale bands show the fitted margin slabs.

The idea of a **SOFT-MARGIN SUPPORT VECTOR MACHINE**, also called a support vector classifier, is to permit violations and charge for them. Introduce a nonnegative **SLACK VARIABLE** ξ_i for each observation. With score $f(x) = w \cdot x + \alpha$, replace the hard-margin constraint by

$$y_i f(X_i) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Slack lowers the score required of a point. It can allow a correctly classified point inside the slab, or even a misclassified point on the wrong side of the boundary.

Write $m_i = y_i f(X_i)$ for the label-score product. For fixed (w, α) , the smallest allowed slack is

$$\xi_i = \max\{0, 1 - m_i\}.$$

Once slack has a positive cost, choosing anything larger wastes objective value. Thus the following interpretations apply to the optimal slacks:

Label-score product	Slack	Position
$m_i \geq 1$	$\xi_i = 0$	On or beyond its margin plane
$0 < m_i < 1$	$0 < \xi_i < 1$	Correct side, inside the slab
$m_i = 0$	$\xi_i = 1$	On the decision boundary
$m_i < 0$	$\xi_i > 1$	Wrong side of the boundary

For $w \neq 0$, the parallel score planes $f = 0$ and $f = \pm 1$ are $1/\|w\|$ apart. For a violating point, $\xi_i/\|w\|$ is its perpendicular shortfall from its class's margin plane. A misclassified point's shortfall extends across the decision boundary.

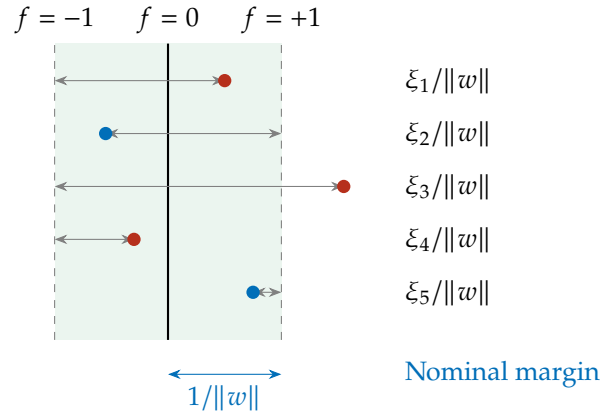


Figure 18: Five margin violations viewed along the normal direction. Blue points have positive labels; red points have negative labels. Each arrow runs from a point to its required margin plane. Rows are separated for readability; arrow lengths represent $\xi_i/\|w\|$, rather than distance to $f = 0$.

For a soft-margin classifier, *margin* means the nominal distance $1/\|w\|$ when $w \neq 0$. It is no longer the distance from the boundary to the nearest observation: points may lie anywhere inside the slab. The full nominal slab width is $2/\|w\|$.

4.2 The soft-margin objective and the choice of C

Allowing slack without charging for it would remove the incentive to fit the labels. The soft-margin optimization problem balances a small weight norm against the total amount of slack:

$$\begin{aligned}
 & \underset{w, \alpha, \xi_1, \dots, \xi_n}{\text{minimize}} && \|w\|^2 + C \sum_{i=1}^n \xi_i \\
 & \text{subject to} && y_i(X_i \cdot w + \alpha) \geq 1 - \xi_i, \quad i = 1, \dots, n, \\
 & && \xi_i \geq 0, \quad i = 1, \dots, n.
 \end{aligned}$$

This is a quadratic program with $d + n + 1$ variables and $2n$ linear constraints. The offset α remains unpenalized, and slack accommodates nonseparable labels.

The scalar $C > 0$ is a REGULARIZATION HYPERPARAMETER. It is the price per unit of slack in this objective. A small C makes violations relatively cheap, encouraging a smaller $\|w\|$ and a wider nominal margin. A large C makes violations expensive, encouraging small slack, even if fitting unusual observations narrows the margin.

	Small C	Large C
Main emphasis	Small weight norm	Small total slack
Typical risk	Underfitting	Overfitting
Outliers	More readily tolerated	Can have more influence
Nonlinear boundary	Often less flexible	Can follow the data closely

These are tendencies, rather than guarantees about test error. A linear boundary remains a hyperplane at every C ; changes in curvature require nonlinear features. Choose C by validation, rather than by training accuracy alone.

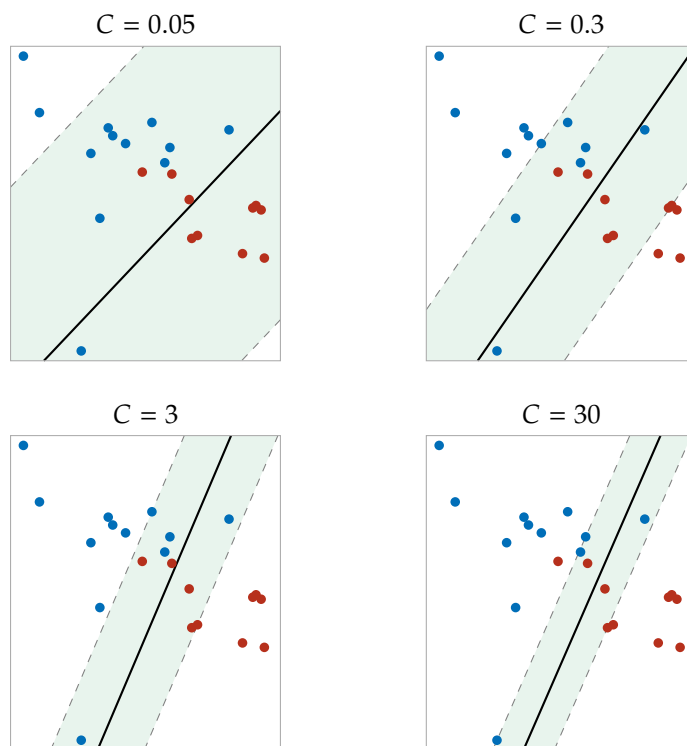


Figure 19: Increasing C makes slack more costly and narrows the fitted nominal slab in this schematic reconstruction. The same points appear in every panel. Solid lines are decision boundaries; dashed lines are margin planes. The point positions illustrate the geometry.

Think of slack as permission purchased for a point to violate its margin constraint. Greater penetration costs more. A small C accepts that cost more readily to obtain a wider margin. For separable data, the large- C limit recovers the hard-margin solution. For nonseparable data, making C large cannot make the hard-margin constraints feasible.

4.3 Nonlinear features and parabolic lifting

A hyperplane cannot separate every useful arrangement of input points. Instead of changing the linear training method, transform the inputs. A FEATURE MAP Φ computes new coordinates from x ; the classifier learns a linear score in those coordinates:

$$f(x) = w \cdot \Phi(x) + \alpha.$$

If the new coordinates depend nonlinearly on x , the boundary in the original input space can be curved. This idea also applies to classifiers that are already nonlinear.

The PARABOLIC LIFTING MAP adds one coordinate:

$$\Phi : \mathbb{R}^d \longrightarrow \mathbb{R}^{d+1}, \quad \Phi(x) = \begin{bmatrix} x \\ \|x\|^2 \end{bmatrix}.$$

The lifted points lie on the paraboloid $x_{d+1} = \|x\|^2$. Although the added feature is determined by the original coordinates, it expands the possible decision boundaries. A plane cutting the paraboloid can induce a sphere boundary in input space.

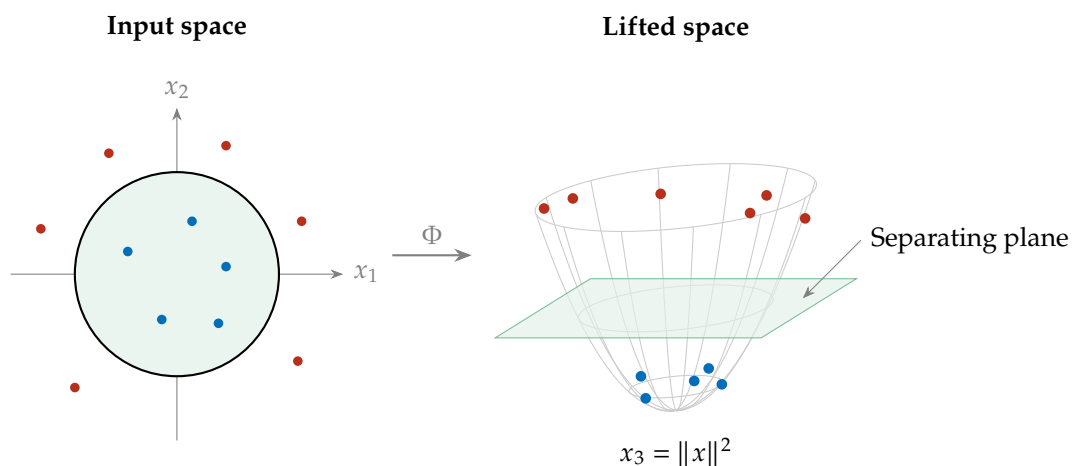


Figure 20: A circular boundary becomes a linear separation after lifting. Blue points inside the circle lie below the separating plane; red points outside it lie above. The drawing shows the geometry schematically, without changing the labels.

Theorem 4.1 [SEPARATION AFTER PARABOLIC LIFTING]. *For a finite labeled data set with both classes present, the lifted points $\Phi(X_1), \dots, \Phi(X_n)$ are strictly linearly separable if and only if the original points are strictly separable by a hypersphere or a hyperplane. Either class may occupy the inside of the sphere.*

Proof. The key is that the squared-distance formula becomes linear in the lifted coordinates. For a sphere with center c and radius ρ ,

$$\|x - c\|^2 < \rho^2 \iff \|x\|^2 - 2c \cdot x + \|c\|^2 - \rho^2 < 0.$$

Equivalently,

$$\begin{bmatrix} -2c^\top & 1 \end{bmatrix} \Phi(x) < \rho^2 - \|c\|^2.$$

Inside and outside therefore become opposite sides of a hyperplane in lifted space. An original hyperplane also lifts to a hyperplane whose last weight is zero.

Conversely, write a separating score in lifted space as

$$g(x) = a \cdot x + b\|x\|^2 + \alpha.$$

If $b = 0$, its input-space boundary is already a hyperplane. If $b \neq 0$, completing the square gives

$$\frac{g(x)}{b} = \left\| x + \frac{a}{2b} \right\|^2 - \left(\frac{\|a\|^2}{4b^2} - \frac{\alpha}{b} \right).$$

This is a sphere score with center $-a/(2b)$ and squared radius $\rho^2 = \|a\|^2/(4b^2) - \alpha/b$. Both signs occur among the observations, so $\rho^2 > 0$. Dividing by a negative b exchanges which class is inside, while preserving separation. $\square$

The hyperplane case is sometimes described as a sphere of infinite radius. Algebraically, it is precisely the case in which the weight on the squared-norm feature vanishes.

4.4 Quadratic features

Parabolic lifting gives all squared coordinates the same coefficient. To obtain axis-aligned ellipsoids and hyperboloids, give each square its own feature. In three dimensions, the boundary has the form

$$Ax_1^2 + Bx_2^2 + Cx_3^2 + Dx_1 + Ex_2 + Fx_3 + \alpha = 0.$$

Here the capital letters are scalar coefficients; this C is a coefficient, rather than the regularization parameter. Use

$$\Phi(x) = \begin{bmatrix} x_1^2 & \cdots & x_d^2 & x_1 & \cdots & x_d \end{bmatrix}^\top \in \mathbb{R}^{2d}.$$

For $d = 3$, the weights are (A, B, C, D, E, F) . The resulting score is linear in the $2d$ features even though it is quadratic in the inputs.

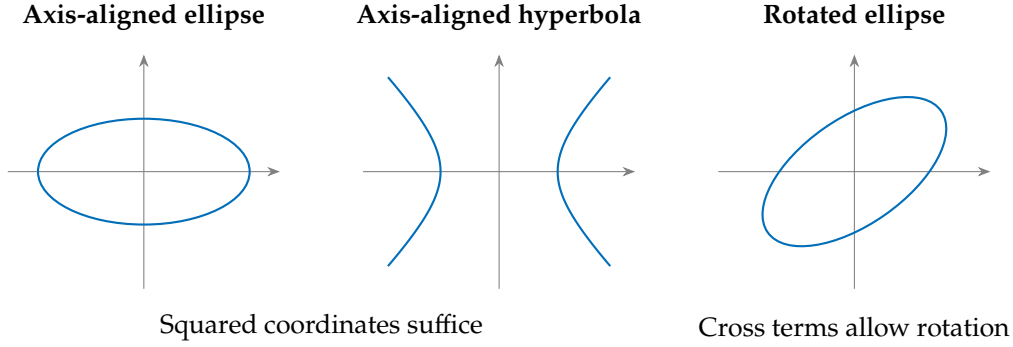


Figure 21: Quadratic decision boundaries in two dimensions, drawn schematically. Separate squared-coordinate features allow axis-aligned ellipses and hyperbolas. Cross terms allow a rotated ellipse.

To allow general orientations, add cross products of distinct input coordinates. In three dimensions, the full quadratic boundary is

$$Ax_1^2 + Bx_2^2 + Cx_3^2 + Dx_1x_2 + Ex_2x_3 + Fx_3x_1 + Gx_1 + Hx_2 + Ix_3 + \alpha = 0.$$

There are d squares, $\binom{d}{2}$ distinct cross products, and d linear terms. The feature dimension is therefore

$$d + \binom{d}{2} + d = \frac{d^2 + 3d}{2}.$$

For $d = 3$, this gives nine features. The constant term is supplied by the offset α , so it is not counted again.

The zero set of a quadratic polynomial is a **QUADRIC**; in two dimensions it is a **CONIC SECTION**. Depending on the coefficients, this includes ellipsoids, hyperboloids, and degenerate cases. The number of features now grows quadratically with the input dimension, so the extra flexibility also increases storage and computation.

4.5 Higher-degree polynomial features

A **MONOMIAL** is a product of powers of input coordinates; its total degree is the sum of their exponents. For a degree- p polynomial score, include every nonconstant monomial of total degree at most p . A cubic score in two dimensions uses

$$\Phi(x) = [x_1^3 \ x_1^2x_2 \ x_1x_2^2 \ x_2^3 \ x_1^2 \ x_1x_2 \ x_2^2 \ x_1 \ x_2]^T.$$

Each feature receives its own weight. Together with α , these weights specify every polynomial of degree at most three.

The general dimension, excluding the constant, is

$$\binom{d+p}{p} - 1.$$

For fixed p , this is $O(d^p)$ as d grows. With $d = 100$ and $p = 4$, it gives 4,598,125 features per observation. Forming and storing such vectors can be expensive. The `KERNEL TRICK` is a way to work with large feature spaces without explicitly forming these vectors; its construction is deferred.

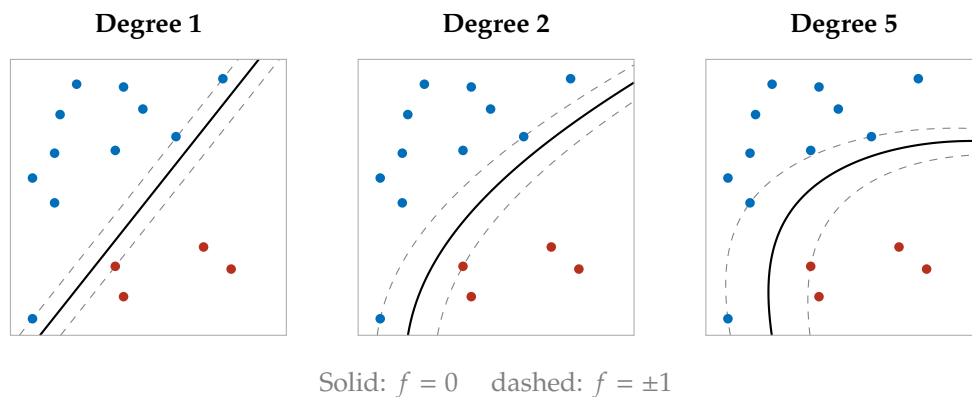


Figure 22: Schematic redraw of polynomial classifiers on the same arrangement. Higher-degree features allow curved boundaries. Each panel fits a hard-margin separator to its displayed feature map. The $f = \pm 1$ contours need not be a constant distance from the input-space boundary; SVM margin is measured in feature space.

Increasing the degree can make formerly nonseparable observations separable after lifting. It can also produce a larger margin in the chosen feature representation. Neither effect guarantees better test predictions: excessive flexibility can fit accidental details of the training sample.

4.6 Choosing features without overfitting

Polynomial degree is another hyperparameter. A degree that is too small can underfit; one that is too large can overfit. Training error alone does not select the right degree, because a more flexible family can fit the same observations more closely.

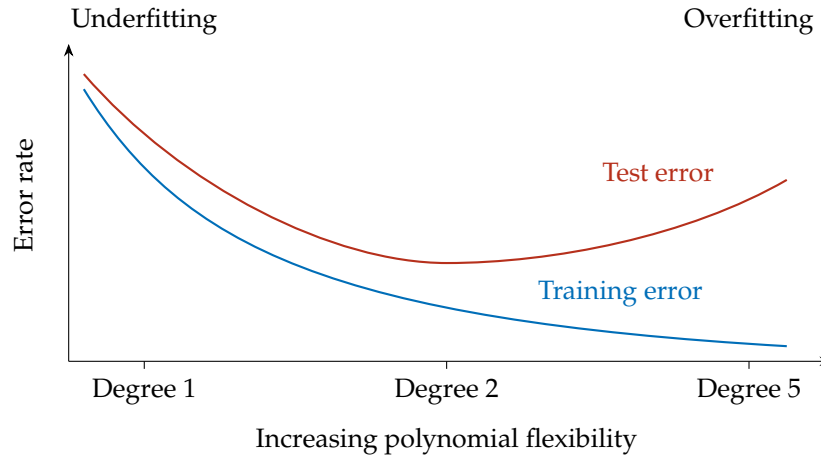


Figure 23: Illustrative training and test error as polynomial flexibility increases. Degree two gives the lowest test error in this schematic picture, while training error continues downward. The curves are conceptual, rather than measured results.

Choose the degree using validation. If polynomial features are combined with a soft-margin SVM, both degree and C require selection. Changing the degree changes the representation and can change the best C , so validate C again for each degree under consideration.

4.7 Image features from edge orientations

Features need not be polynomials. For images, useful features can describe local structure rather than raw pixel values. An EDGE DETECTOR approximates image intensity or color gradients. Examples include a tap filter, a Sobel filter, and an oriented Gaussian derivative filter. Because an image is sampled on a discrete grid, these are approximations to continuous derivatives.

Collect local line orientations into histograms, using twelve orientation bins per region. The histogram entries become the features used for classification. Regions retain local information; the bins describe which orientations are prominent there.

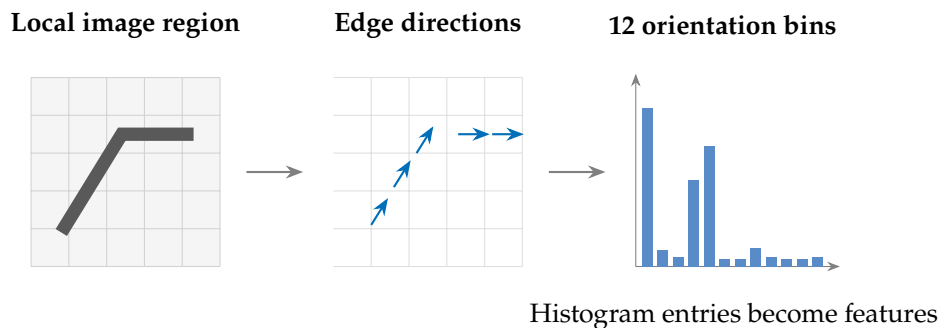


Figure 24: Local edges are summarized by an orientation histogram. Repeat the construction across image regions and collect the histogram entries into a feature vector. Bar heights illustrate the representation; they are schematic values.

In the handwriting-recognition comparison attributed to Maji and Malik (2009), a linear SVM's test error was 15.38% with raw pixels and 2.64% with histogram features. The training method remained linear; the representation made the difference. This motivates choosing features that capture structure useful for the particular task.

Revision. Slack relaxes the unit label-score constraints, and C charges for its total amount. The soft-margin distance $1/\|w\|$ is nominal, not the distance to the nearest point. A feature map makes a linear score in transformed coordinates nonlinear in the original inputs. Parabolic lifting gives sphere boundaries; squares and cross terms give quadratic boundaries. Higher-degree features add flexibility and cost, while local image histograms offer task-specific structure.

Common mistakes: treating every nonzero slack as a wrong label; reversing the effect of C in this objective; counting the constant feature twice; comparing feature-space margin with input-space distance; or selecting degree using training error alone.

Check yourself: What does $\xi_i > 1$ mean? Why is the soft-margin problem always feasible? How does a sphere become a hyperplane after lifting? Why are there $(d^2 + 3d)/2$ quadratic features? Why should C be validated again when the degree changes?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 4: Soft-Margin Support Vector Machines; Features](#), PDF pp. 1–7 (printed pp. 18–24). Figures are original vector redraws; fitted panels use schematic reconstructions of the supplied point arrangements. The smallest-slack calculation, converse lifting argument, and exact monomial count complete the supplied formulas. Here C is the slack penalty coefficient, as in the displayed objective, rather than a slack budget. The image photograph is replaced by a feature-pipeline diagram; the reported image-classification errors are retained from the supplied material. Administration and future kernel details are omitted.

LECTURE 5: MACHINE LEARNING ABSTRACTIONS AND OPTIMIZATION

5.1 Four levels of a learning system

A learning system contains several choices that answer different questions. The data determine the task. The model determines which answers are permitted. The optimization problem describes how to fit that model, and the optimization algorithm computes a fit. Keeping these choices separate makes both the mathematics and the software easier to change.

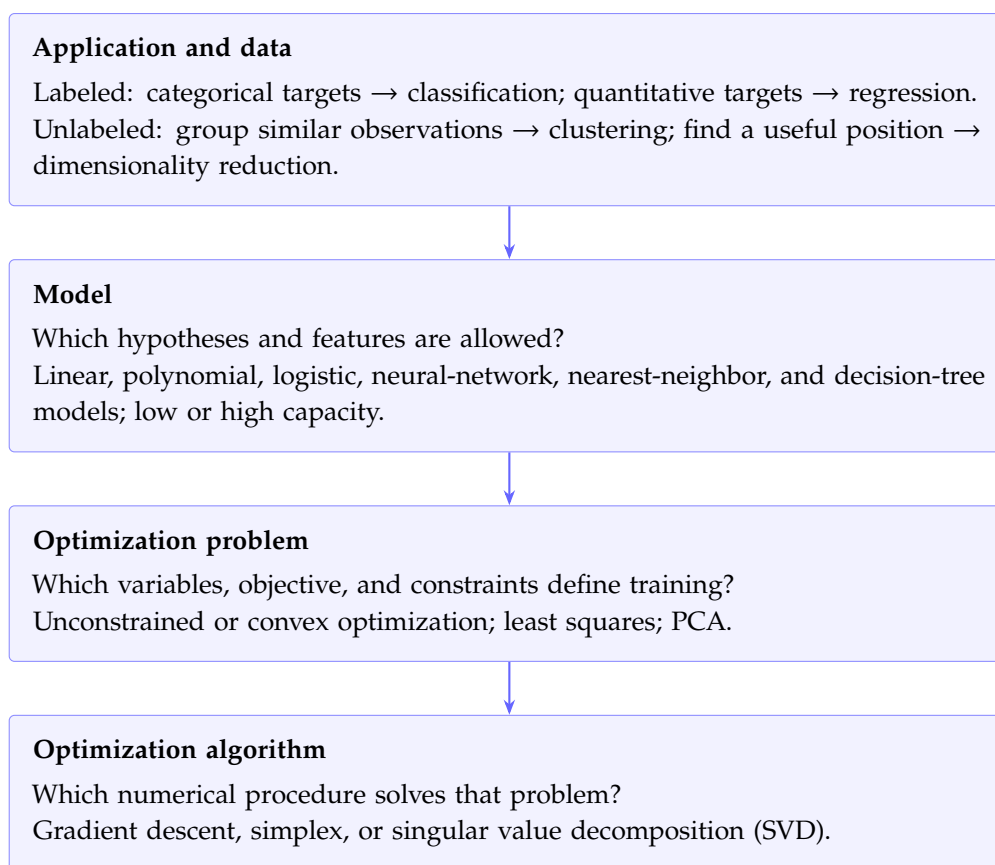


Figure 25: Four abstraction levels. Each level specifies a different part of the learning system; changing an earlier choice can require changes to the levels below it.

The **MODEL** specifies a family of hypotheses and the features they use. Its capacity affects underfitting and overfitting, while its structure affects interpretability. Useful features should reflect the data's structure. A model also needs a training problem that can be solved with the available resources.

The OPTIMIZATION PROBLEM specifies variables, an objective, and any constraints. The OPTIMIZATION ALGORITHM is the procedure used to solve that problem. A linear decision function is a model choice; minimizing a loss is a problem choice; taking gradient steps is an algorithm choice. Replacing a linear classifier with a neural network can change both the objective's geometry and the solver that is appropriate for it.

This decomposition is useful even though some learning methods do not fit it exactly. Recognizing a familiar optimization problem is a major part of translating an application into a workable training method.

5.2 Unconstrained optimization and convexity

In UNCONSTRAINED OPTIMIZATION, every parameter vector is allowed. For a continuous OBJECTIVE FUNCTION $f : \mathbb{R}^d \rightarrow \mathbb{R}$, the minimization problem is

$$\underset{w \in \mathbb{R}^d}{\text{minimize}} f(w).$$

Maximization can be handled by minimizing $-f$. A function is SMOOTH here if it is differentiable and its gradient is continuous.

Definition 5.1 [GLOBAL AND LOCAL MINIMA]. A point w^* is a GLOBAL MINIMIZER if $f(w^*) \leq f(v)$ for every $v \in \mathbb{R}^d$. It is a LOCAL MINIMIZER if there is a radius $r > 0$ such that

$$f(w^*) \leq f(v) \quad \text{whenever } \|v - w^*\| < r.$$

The minimum value is $f(w^*)$; the minimizer is the parameter vector where that value is attained.

A local minimizer has no better point in a sufficiently small neighborhood. A global minimizer has no better point anywhere. Finding a local minimum is usually more manageable than establishing global optimality for a general objective.

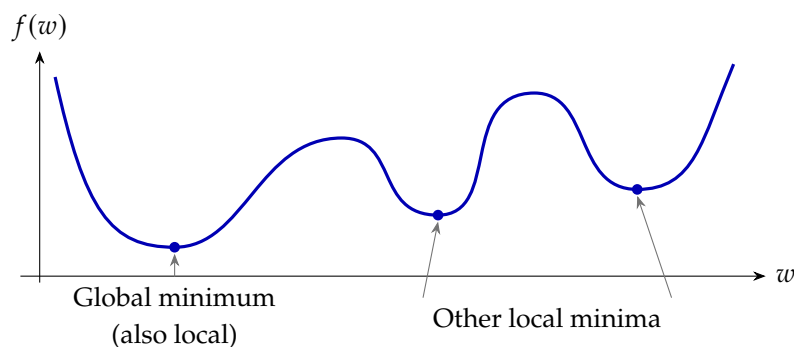


Figure 26: A schematic nonconvex objective. All three marked points are locally lowest in their neighborhoods, but only the leftmost has the lowest value over the whole domain.

Convexity gives a useful exception: a locally best point is also globally best. Geometrically, the graph never rises above a chord joining two points on it.

Definition 5.2 [CONVEX FUNCTION]. A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is CONVEX if, for every $x, y \in \mathbb{R}^d$ and $t \in [0, 1]$,

$$f((1-t)x + ty) \leq (1-t)f(x) + tf(y).$$

The left side is the function value at a point between x and y ; the right side is the height of the chord at that point.

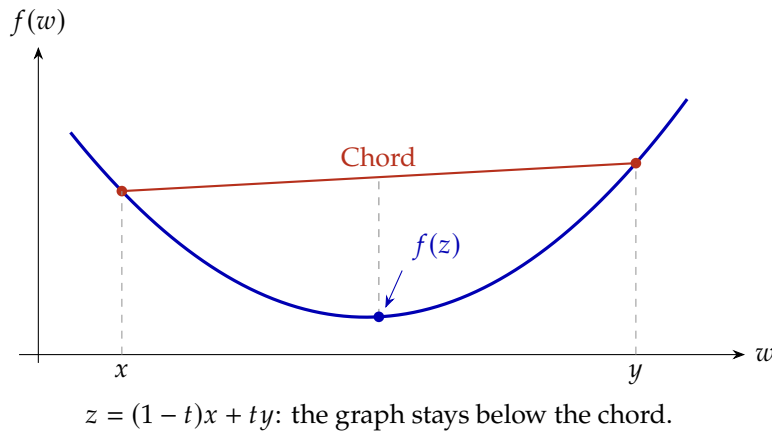


Figure 27: The convexity inequality in one dimension. At every intermediate input, the curve lies on or below the straight chord.

Proposition 5.3 [MINIMIZERS OF A CONVEX FUNCTION]. Every local minimizer of a convex function is global. If two points minimize the function, every point on their connecting segment also minimizes it, with the same objective value.

Proof. The central idea is to move a short distance toward any purportedly better point. Suppose x is a local minimizer but $f(y) < f(x)$. For a sufficiently small $t > 0$, the point $z = (1-t)x + ty$ lies in the neighborhood where x is minimal. Yet convexity gives

$$f(z) \leq (1-t)f(x) + tf(y) < f(x),$$

a contradiction. Thus x is global.

Now suppose $f(x) = f(y) = m$, the global minimum value. Convexity gives $f((1-t)x + ty) \leq m$, while global minimality gives the reverse inequality. The value is therefore exactly m along the segment. $\square$

A convex function may have no attained minimum, one minimizer, or a connected set of minimizers. On an unbounded domain, even continuity and a finite lower bound do not guarantee attainment.

The perceptron risk is convex but nonsmooth:

$$R(w) = \sum_{i=1}^n \max\{0, -y_i w \cdot X_i\}.$$

Each term is the maximum of two affine functions and is convex; adding the convexity inequalities shows that their sum is convex. Corners can occur where a term changes between its two affine pieces. Whenever the risk has several minimizers, they form a connected, equal-value set. Neural-network training objectives, in contrast, can be nonconvex and have many local minima.

5.3 Numerical methods and line search

For smooth f , the gradient points in the direction of greatest local increase. GRADIENT DESCENT moves in the opposite direction:

$$w \leftarrow w - \varepsilon \nabla f(w), \quad \varepsilon > 0.$$

The LEARNING RATE ε determines the step size. A fixed, predetermined step does not check how far along that direction the objective decreases.

Algorithm 5.1: Fixed-step gradient descent

Input: Smooth objective f , initial vector w_0 , learning rate $\varepsilon > 0$, iteration count T

```

1  $w \leftarrow w_0$ 
2 for  $t \leftarrow 1$  to  $T$  do
3    $w \leftarrow w - \varepsilon \nabla f(w)$ 
4 return  $w$ 
```

STOCHASTIC GRADIENT DESCENT instead computes an update from one training observation or a small batch at a time. This reduces the amount of data processed per update. With very large data sets, these inexpensive updates are often useful for neural-network training.

Other smooth optimization methods include NEWTON'S METHOD and NONLINEAR CONJUGATE GRADIENT. Newton's method uses the HESSIAN MATRIX, whose (j, k) entry is $\partial^2 f / \partial w_j \partial w_k$. In d dimensions it is a $d \times d$ matrix, often dense. Forming and using it can be expensive when d is large. Nonlinear conjugate gradient uses a sequence of search directions with line searches rather than only fixed steps along the negative gradient.

A LINE SEARCH chooses a step by solving a one-dimensional problem along a chosen direction p :

$$\underset{\eta \geq 0}{\text{minimize}} \phi(\eta), \quad \phi(\eta) = f(w + \eta p).$$

The goal is a suitable local minimum or decreasing step along this line, not a search over all parameter vectors. Secant and Newton–Raphson methods use derivative information for smooth line objectives. Newton–Raphson can require second derivatives, hence Hessian information. Direct methods such as golden-section search compare function values and can be used without smoothness on a bracketed interval where the function decreases to a minimum and then increases.

Note. At a nonsmooth corner, the ordinary gradient may not exist. Gradient-style methods must account for this; blindly applying the smooth formula at every point is not well defined. BFGS (Broyden–Fletcher–Goldfarb–Shanno) is another method sometimes applied to nonsmooth objectives, but smooth-method guarantees do not automatically apply. These methods seek local improvement; none of these names alone guarantees a local or global minimum for every objective and step rule.

5.4 Smooth equality constraints

A constraint restricts the allowed parameter vectors. With smooth equality constraints, the problem has the form

$$\begin{aligned} & \underset{w}{\text{minimize}} && f(w) \\ & \text{subject to} && g(w) = 0. \end{aligned}$$

The function g may be scalar or vector valued. A single equality describes a level set; several equalities require their common intersection. Improvement must respect this restricted geometry.

LAGRANGE MULTIPLIERS incorporate the equalities into a LAGRANGIAN. For $g : \mathbb{R}^d \rightarrow \mathbb{R}^m$ and $\lambda \in \mathbb{R}^m$, write

$$L(w, \lambda) = f(w) + \sum_{j=1}^m \lambda_j g_j(w).$$

At a constrained local optimum where the constraint gradients are linearly independent, the first-order conditions are

$$\nabla f(w) + \sum_{j=1}^m \lambda_j \nabla g_j(w) = 0, \quad g(w) = 0.$$

These stationarity equations balance the objective gradient against constraint normals. They do not jointly minimize L or, by themselves, prove optimality.

5.5 Linear programs and feasible regions

Linear objectives with linear constraints have a particularly clear geometry. A LINEAR PROGRAM can be written

$$\begin{aligned} & \underset{w \in \mathbb{R}^d}{\text{maximize}} && c \cdot w \\ & \text{subject to} && Aw \leq b, \end{aligned}$$

where $A \in \mathbb{R}^{n \times d}$ and $b \in \mathbb{R}^n$. The inequality is componentwise: row A_i imposes $A_i \cdot w \leq b_i$. Minimization has the same form with the objective direction reversed.

The FEASIBLE REGION is $F = \{w : Aw \leq b\}$. It is the intersection of the halfspaces allowed by all the inequalities.

Definition 5.4 [CONVEX SET]. A set P is CONVEX if, for every $p, q \in P$ and every $t \in [0, 1]$, the point $(1 - t)p + tq$ also belongs to P .

For $p, q \in F$,

$$A((1 - t)p + tq) = (1 - t)Ap + tAq \leq (1 - t)b + tb = b.$$

Thus F is convex. An intersection of finitely many halfspaces is a POLYHEDRON; a bounded one is a POLYTOPE, the higher-dimensional counterpart of a polygon. A feasible region can also be empty or unbounded.

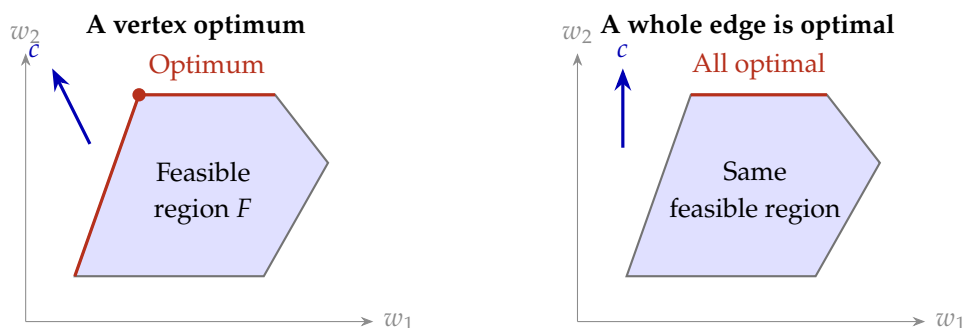


Figure 28: Schematic maximization of $c \cdot w$. Moving in the direction c favors the upper-left vertex in the first panel. Pointing straight up makes the entire top edge optimal in the second. Red edges show constraints tight at the indicated optimum or optima.

When a finite optimum is attained, it is as far as possible in the objective direction c . A constraint is ACTIVE at a point when it holds with equality, $A_i \cdot w = b_i$. The first panel has two active constraints at its optimum. An optimum need not be unique: every point on a face can have the same objective value. The optimal set is convex, since a segment between two feasible optimizers remains feasible and preserves the linear objective value.

For SVMs, active margin constraints explain why observations that touch a margin plane or violate the nominal slab can determine the fit. With positive slack cost, a soft-margin observation with nonzero slack has a tight label-score constraint $y_i(w \cdot X_i + \alpha) = 1 - \xi_i$.

Example: Finding any separating linear classifier is a feasibility problem:

$$\begin{aligned} & \underset{w, \alpha}{\text{maximize}} && 0 \\ & \text{subject to} && y_i(w \cdot X_i + \alpha) \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

Every feasible pair (w, α) is equally optimal for this constant objective, and every such pair correctly classifies the observations.

Proposition 5.5 [SEPARABILITY AND FEASIBILITY]. *A finite labeled data set is strictly linearly separable if and only if the unit label-score constraints above are feasible.*

Proof. Feasibility gives $y_i(w \cdot X_i + \alpha) \geq 1 > 0$, so it gives strict separation. Conversely, for any strict separator, the finite minimum

$$m = \min_i y_i(w \cdot X_i + \alpha)$$

is positive. Replacing (w, α) by $(w/m, \alpha/m)$ makes every label-score product at least one. Hence the constraints are feasible. $\square$

The SIMPLEX METHOD, introduced by George Dantzig in 1947, searches along the polyhedron's edges from vertex to vertex. INTERIOR-POINT METHODS provide another major family of linear-programming algorithms. Which constraints become active is a central part of the search; enumerating all candidate combinations is not practical in general.

The constant objective finds a separator but does not select the one with the largest margin. Changing to a squared-norm objective leads to a quadratic program.

5.6 Quadratic programs and the SVM objective

A QUADRATIC PROGRAM minimizes a quadratic objective subject to linear constraints:

$$\begin{aligned} & \underset{w}{\text{minimize}} && f(w) = w^T Q w + c^T w \\ & \text{subject to} && A w \leq b, \end{aligned}$$

where Q is symmetric. Its curvature determines whether this is a convex optimization problem.

Definition 5.6 [POSITIVE DEFINITE AND SEMIDEFINITE MATRICES]. A symmetric matrix Q is POSITIVE DEFINITE if $v^T Q v > 0$ for every nonzero v . It is POSITIVE SEMIDEFINITE if $v^T Q v \geq 0$ for every v . It is INDEFINITE if the quadratic form takes both positive and negative values.

For $z = (1 - t)x + ty$, expansion gives

$$(1 - t)f(x) + tf(y) - f(z) = t(1 - t)(x - y)^T Q (x - y).$$

If Q is positive semidefinite, the right side is nonnegative, so f is convex. If Q is positive definite, it is positive whenever $x \neq y$ and $0 < t < 1$. This **STRICT CONVEXITY** rules out two distinct minimizers on a convex feasible region: their midpoint would have a strictly smaller value.

For a nonempty closed polyhedral feasible region and positive definite Q , the quadratic grows without bound as $\|w\| \rightarrow \infty$, so a minimum is attained and is unique. With positive semidefinite Q , an attained minimum may be unique or may have infinitely many minimizers. An indefinite Q gives a nonconvex objective; global quadratic optimization is NP-hard in general, although particular instances can be easier.

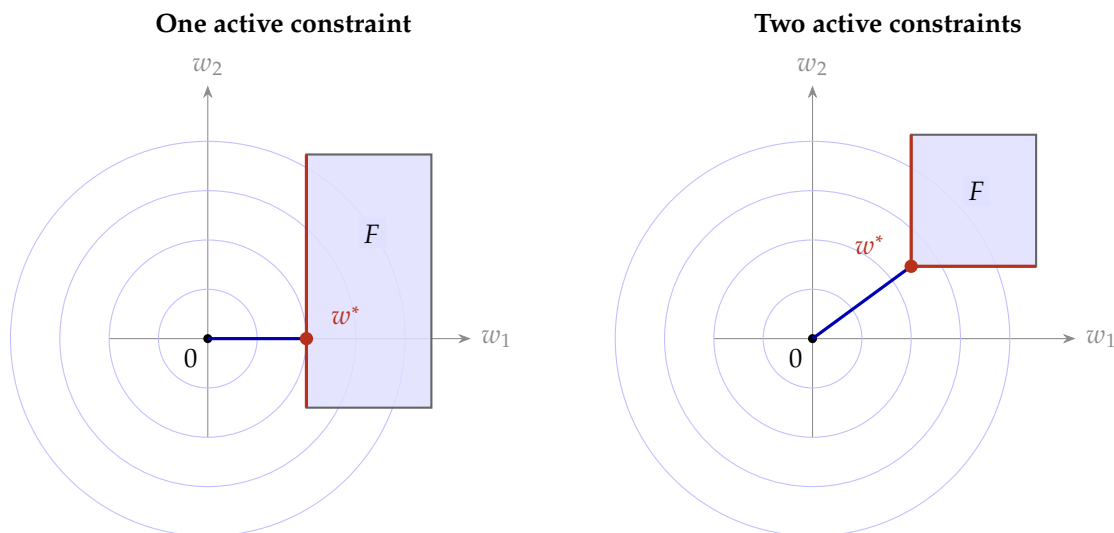


Figure 29: Schematic minimization of $\|w\|^2$ over two polygons. Circles are equal-value contours. The smallest circle meeting the feasible region touches an edge in the first panel and a vertex in the second. Red boundaries are active at w^* .

The hard-margin SVM uses this squared-norm geometry:

$$\begin{aligned} & \underset{w, \alpha}{\text{minimize}} && \|w\|^2 \\ & \text{subject to} && y_i(w \cdot X_i + \alpha) \geq 1, \quad i = 1, \dots, n. \end{aligned}$$

Its feasible region is the same one used to find any separator. The objective now chooses the smallest weight norm, hence the largest margin. The distance is measured in the w coordinates: α is not penalized. In the full variable vector (w, α) , the quadratic matrix has a zero entry in the offset direction and is positive semidefinite, rather than positive definite. Thus the positive-definite uniqueness argument cannot simply be applied to all SVM variables.

Quadratic-programming algorithms include simplex-like methods, **SEQUENTIAL MINIMAL OPTIMIZATION** (SMO, used by LIBSVM), and **COORDINATE DESCENT** (used by LIBLINEAR). The latter two exploit SVM structure. SMO changes a pair of coordinates at a time; this allows a cheap line optimization using only the relevant curvature information, rather than processing a full dense Hessian or always moving in the steepest-descent direction.

Revision. Separate the task, model, training problem, and solver. A local minimum only compares nearby points; a global minimum compares the entire domain. Convexity makes every local minimum global and makes the minimizer set convex. Linear programs optimize a linear objective over halfspaces. Convex quadratic programs add nonnegative quadratic curvature; SVMs use it to prefer a small weight norm.

Common mistakes: confusing a model with its solver; assuming convexity guarantees an attained or unique minimum; using an ordinary gradient at a corner; treating Lagrange stationarity as a proof of optimality; or assuming the SVM offset is penalized.

Check yourself: Why is a local minimum global for a convex function? What does a line search optimize? Why is a linear program's feasible region convex? How does rescaling prove the separability criterion? Why does positive definite Q imply uniqueness, while positive semidefinite Q need not?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 5: Machine Learning Abstractions and Numerical Optimization](#), PDF pp. 1–5 (printed pp. 25–29). Figures are original vector redraws of the supplied hierarchy and optimization geometry. The convex-minimizer, feasibility, and quadratic-curvature arguments complete the supplied claims. Attainment, nonsmooth updates, equality-constraint stationarity, and the unpenalized SVM offset are qualified to avoid unconditional optimality or uniqueness claims. BFGS's nonsmooth qualification was checked against Lewis and Overton, [Nonsmooth Optimization via BFGS](#). Course administration is omitted.

LECTURE 6: DECISION THEORY AND GENERATIVE MODELS

6.1 From class frequencies to posterior probabilities

Identical features need not imply identical outcomes. Two borrowers can each earn \$30,000 and owe \$15,000, yet only one defaults. A rule that uses only those features must give both borrowers the same prediction. Probability describes the uncertainty that the features leave unresolved.

Write X for a random feature vector and $Y \in \{-1, 1\}$ for its random class. Here uppercase letters denote random variables, rather than a data matrix. Lowercase x and y denote observed values. The PRIOR PROBABILITY $\mathbb{P}(Y = y)$ gives a class's frequency before observing the features. The CLASS-CONDITIONAL DISTRIBUTION describes X within that class. The POSTERIOR PROBABILITY $\mathbb{P}(Y = y \mid X = x)$ describes the class after observing x .

Example: Consider a fictional population in which 10% have cancer and 90% do not. Let $Y = 1$ mean cancer, $Y = -1$ mean no cancer, and group daily calorie intake into three categories. These invented numbers are only a probability example.

	Below 1,200	1,200–1,600	Above 1,600
$\mathbb{P}(X \text{ in category} \mid Y = 1)$	0.20	0.50	0.30
$\mathbb{P}(X \text{ in category} \mid Y = -1)$	0.01	0.10	0.89

Each row sums to one. The entries say how calorie intake is distributed *within* a class; they do not give the chance of cancer for someone in a particular category.

For the middle category, write $B = \{1,200 \leq X \leq 1,600\}$. The law of total probability gives

$$\mathbb{P}(B) = \mathbb{P}(B \mid Y = 1)\mathbb{P}(Y = 1) + \mathbb{P}(B \mid Y = -1)\mathbb{P}(Y = -1) = 0.50(0.10) + 0.10(0.90) = 0.14.$$

Bayes' theorem then reverses the conditioning:

$$\begin{aligned}\mathbb{P}(Y = 1 \mid B) &= \frac{\mathbb{P}(B \mid Y = 1)\mathbb{P}(Y = 1)}{\mathbb{P}(B)} = \frac{0.05}{0.14} = \frac{5}{14}, \\ \mathbb{P}(Y = -1 \mid B) &= \frac{0.09}{0.14} = \frac{9}{14}.\end{aligned}$$

The posteriors sum to one. A person eating 1,400 calories belongs to B ; the table supplies information about that category, rather than an exact-value probability. Within this model, the chance of cancer in the category is about 36%, even though its conditional frequency is five times greater in the cancer group. The larger no-cancer prior reverses the comparison.

6.2 Loss and the risk of a decision rule

Choosing the more likely class maximizes the chance of being correct, but different errors can have different consequences. In screening, missing a case can be costlier than sending someone for an unnecessary follow-up test. In spam filtering, hiding a legitimate message can be costlier than leaving spam in the inbox.

Definition 6.1 [LOSS AND DECISION RULE]. A LOSS FUNCTION $L(z, y)$ assigns a cost to predicting class z when the true class is y . A DECISION RULE, or classifier, is a function $r : \mathbb{R}^d \rightarrow \{-1, 1\}$ that chooses a class from the features.

A FALSE POSITIVE predicts 1 when $Y = -1$; a FALSE NEGATIVE predicts -1 when $Y = 1$. The ZERO-ONE LOSS is $L(z, y) = \mathbf{1}\{z \neq y\}$. It treats both types of error equally. For the screening example, instead use the asymmetric loss

$$L(z, y) = \begin{cases} 1, & z = 1, y = -1, \\ 5, & z = -1, y = 1, \\ 0, & z = y. \end{cases}$$

At the middle category, predicting 1 has expected loss $1(9/14) = 9/14$. Predicting -1 has expected loss $5(5/14) = 25/14$. Thus the lower-cost action is to predict 1 and seek follow-up screening, although -1 is the more likely class.

Definition 6.2 [RISK]. The RISK of a decision rule is its expected loss over the joint distribution of features and classes:

$$R(r) = \mathbb{E}[L(r(X), Y)].$$

It averages over all possible observations, rather than only the training points or a single feature value.

For discrete features, conditioning first on X gives

$$R(r) = \sum_x \left[\sum_{y \in \{-1, 1\}} L(r(x), y) \mathbb{P}(Y = y \mid X = x) \right] \mathbb{P}(X = x).$$

The expression in brackets is the expected loss at x . Using $\mathbb{P}(Y = y \mid X = x) \mathbb{P}(X = x) = \mathbb{P}(X = x \mid Y = y) \mathbb{P}(Y = y)$ also gives the class-by-class form

$$R(r) = \sum_{y \in \{-1, 1\}} \mathbb{P}(Y = y) \sum_x L(r(x), y) \mathbb{P}(X = x \mid Y = y).$$

Both expressions average the same joint outcomes, in different orders.

6.3 The Bayes decision rule

To minimize overall risk, minimize the expected loss separately at each feature value. Define

$$q_y(x) = \mathbb{P}(Y = y \mid X = x), \quad \rho(z \mid x) = \sum_{y \in \{-1,1\}} L(z, y) q_y(x).$$

Here $\rho(z \mid x)$ is the **CONDITIONAL RISK** of predicting z after seeing x .

Theorem 6.3 [BAYES DECISION RULE]. A BAYES DECISION RULE, or BAYES CLASSIFIER, chooses $r^*(x) \in \arg \min_{z \in \{-1,1\}} \rho(z \mid x)$ wherever the posterior is defined. It minimizes $R(r)$ among decision rules.

Proof. Each choice affects only the loss at its own feature value. For every x , the minimizing choice satisfies $\rho(r^*(x) \mid x) \leq \rho(r(x) \mid x)$. Multiplying by the nonnegative weight $\mathbb{P}(X = x)$ and summing proves $R(r^*) \leq R(r)$ for every r . The same argument uses an integral for continuous features. Ties may be resolved either way; feature values with zero probability do not affect the risk. $\square$

Assume correct predictions cost zero. Set $c_{FP} = L(1, -1)$ and $c_{FN} = L(-1, 1)$. Then

$$\rho(1 \mid x) = c_{FP} q_{-1}(x), \quad \rho(-1 \mid x) = c_{FN} q_1(x).$$

The error cost for predicting a class multiplies the probability of the *other* class. Choosing -1 in a tie gives

$$r^*(x) = \begin{cases} 1, & c_{FN} q_1(x) > c_{FP} q_{-1}(x), \\ -1, & \text{otherwise.} \end{cases}$$

Since $q_{-1}(x) = 1 - q_1(x)$, for positive error costs this is equivalent to predicting 1 when

$$q_1(x) > \frac{c_{FP}}{c_{FP} + c_{FN}}.$$

Equal error costs give a threshold of $1/2$: choose the class with larger posterior probability. Making false negatives more expensive lowers the threshold for predicting 1.

6.4 Optimal risk in the screening example

The BAYES RISK, or OPTIMAL RISK, is $R(r^*)$. Obtaining the rule that minimizes it is RISK MINIMIZATION. For the asymmetric screening loss, the posterior threshold is $1/6$. We can avoid dividing by the category probability by comparing the prior-weighted error contributions directly:

Category	Predict 1	Predict -1	Lower-cost prediction
Below 1,200	$0.90(0.01) = 0.009$	$5(0.10)(0.20) = 0.100$	1
1,200–1,600	$0.90(0.10) = 0.090$	$5(0.10)(0.50) = 0.250$	1
Above 1,600	$0.90(0.89) = 0.801$	$5(0.10)(0.30) = 0.150$	-1

Each entry is a category's *unconditional contribution* to risk, not the conditional risk given that category. Both entries in a row would have the same denominator when converted to conditional risks, so their ordering is unchanged. The optimal rule predicts 1 for intake at most 1,600 and -1 above 1,600. Its risk is

$$\begin{aligned} R(r^*) &= 0.009 + 0.090 + 0.150 \\ &= 0.90(0.01 + 0.10) + 0.10(5 \cdot 0.30) = 0.249. \end{aligned}$$

The first part is the cost of false positives in the first two categories. The second is the cost of false negatives in the last category. No rule based on these categories has lower risk under the specified probabilities and loss. In applications, those probabilities must usually be estimated, so the resulting rule need not attain the true Bayes risk.

6.5 Continuous features and probability density

For continuous features, a single exact value has probability zero. Probabilities come from areas under a density, not from the curve's height at one point.

Definition 6.4 [PROBABILITY DENSITY]. A PROBABILITY DENSITY FUNCTION f for a scalar random variable X is nonnegative and satisfies

$$\int_{-\infty}^{\infty} f(x) dx = 1, \quad \mathbb{P}(x_1 \leq X \leq x_2) = \int_{x_1}^{x_2} f(x) dx.$$

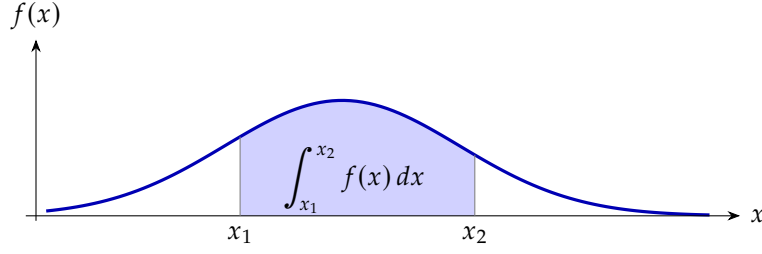


Figure 30: Probability is shaded area. The height $f(x)$ is a density; the integral between x_1 and x_2 is a probability. The curve is schematic.

Expectations weight each value by its density:

$$\mathbb{E}[g(X)] = \int_{-\infty}^{\infty} g(x)f(x) dx.$$

The MEAN and VARIANCE are

$$\mu = \mathbb{E}[X] = \int_{-\infty}^{\infty} x f(x) dx, \quad \sigma^2 = \mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - \mu^2.$$

The last identity follows by expanding $(X - \mu)^2 = X^2 - 2\mu X + \mu^2$ and using $\mathbb{E}[X] = \mu$. These expressions assume the required expectations are finite.

6.6 Continuous Bayes classification and risk

Write $f_y(x) = f_{X|Y=y}(x)$ for the density within class y and $\pi_y = \mathbb{P}(Y = y)$ for its prior. Each f_y integrates to one, but the marginal density is the prior-weighted mixture

$$f_X(x) = \pi_1 f_1(x) + \pi_{-1} f_{-1}(x).$$

At points with $f_X(x) > 0$, Bayes' theorem gives

$$q_y(x) = \mathbb{P}(Y = y | X = x) = \frac{\pi_y f_y(x)}{f_X(x)}.$$

This conditional probability is defined through densities, rather than by dividing probabilities of the zero-probability event $\{X = x\}$. The denominator is shared by both classes.

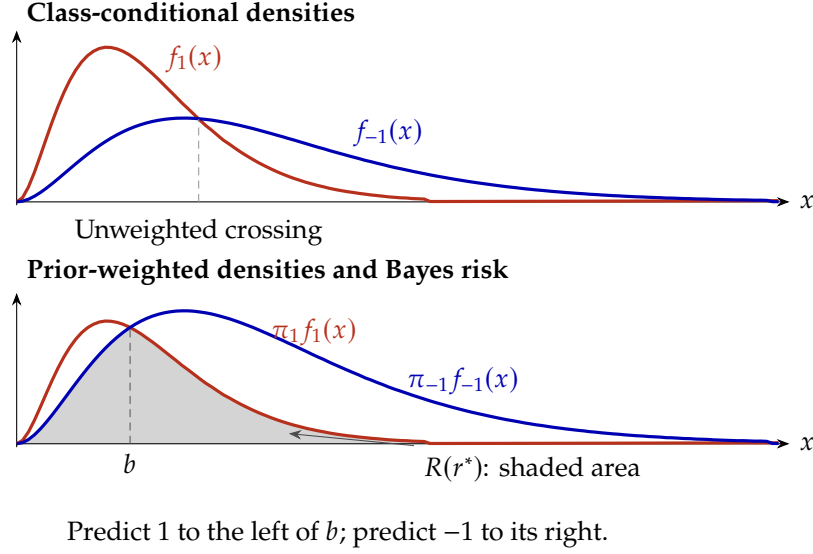


Figure 31: Schematic continuous class distributions. Top: both class-conditional densities have total area one. Bottom: with priors $\pi_1 = 1/3$ and $\pi_{-1} = 2/3$, the weighted curves cross at the zero-one Bayes boundary b . The shifted boundary accounts for class frequencies. The shaded area under their lower envelope is the Bayes risk.

Under zero-one loss, compare $\pi_1 f_1(x)$ with $\pi_{-1} f_{-1}(x)$. Comparing the unweighted densities alone ignores class frequencies. The weighted curves are joint densities for features and class; they are not posterior probabilities. Their areas are π_1 and π_{-1} , respectively.

Risk now averages over x by integration:

$$R(r) = \sum_{y \in \{-1, 1\}} \pi_y \int L(r(x), y) f_y(x) dx.$$

For d -dimensional features, the integral is over $\mathbb{R}^d$. With zero loss for correct predictions, the Bayes risk is

$$R(r^*) = \int \min\{c_{FN}\pi_1 f_1(x), c_{FP}\pi_{-1} f_{-1}(x)\} dx.$$

At each x , predicting -1 incurs the first contribution and predicting 1 incurs the second. Taking the smaller one gives the lower envelope. Asymmetric losses scale the curves by their error costs before taking this minimum.

For zero-one loss, $L(r(X), Y)$ is the indicator of a wrong prediction, so

$$R(r) = \mathbb{P}(r(X) \neq Y).$$

The Bayes risk is then the smallest achievable probability of error using the observed features.

6.7 Decision boundaries in several dimensions

With zero-one loss, a binary Bayes rule can use $q_1(x)$ as its decision function. Its equal-posterior threshold set is

$$\{x : q_1(x) = 1/2\} = \{x : \pi_1 f_1(x) = \pi_{-1} f_{-1}(x)\},$$

where $f_X(x) > 0$. When the class comparison changes sign across this set, it forms a decision boundary. A tie by itself need not separate two prediction regions.

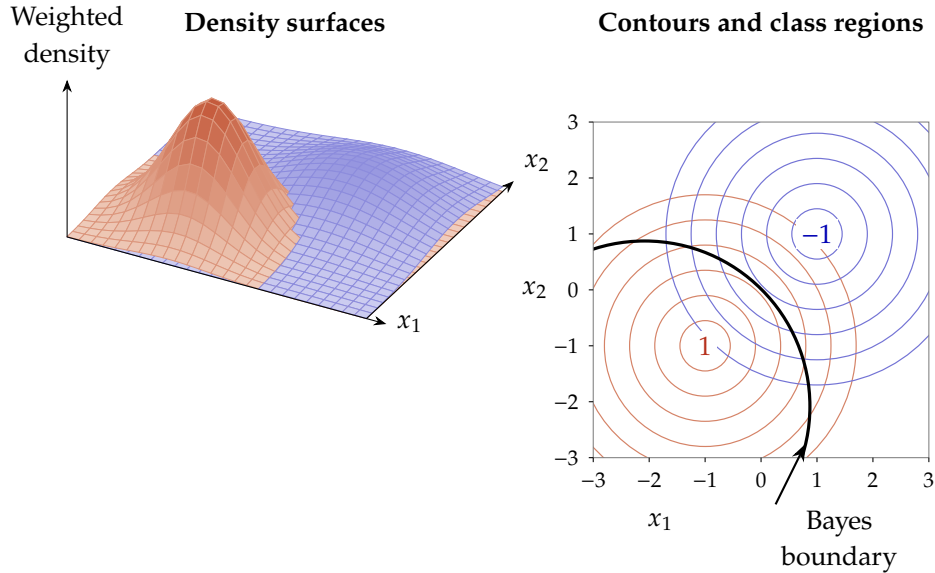


Figure 32: Two views of the same schematic, prior-weighted two-dimensional Gaussian densities. Left: overlapping density surfaces. Right: contours and the equality curve of the two densities. Different spreads can give a curved Bayes boundary. The shapes illustrate the geometry, rather than fitted measurements.

Errors in probability estimates matter most near the boundary, where a small change can reverse the class comparison. Far from it, small errors can leave the selected class unchanged.

For a finite set of classes C , zero-one loss gives conditional risk $1 - \mathbb{P}(Y = C \mid X = x)$ for predicting C . Thus the same argument yields

$$r^*(x) \in \arg \max_{C \in C} \mathbb{P}(Y = C \mid X = x).$$

Only ties for the *largest* posterior are relevant to the multiclass boundary. Equality between two low-probability classes need not change the prediction.

6.8 Three ways to build a classifier

A classifier can model the joint distribution, model the posterior, or directly specify which class to predict. These choices determine what information the fitted model can provide.

Definition 6.5 [GENERATIVE AND DISCRIMINATIVE MODELS]. A **GENERATIVE MODEL** specifies a probabilistic model for all the variables, here the joint distribution of (X, Y) . A **DISCRIMINATIVE MODEL** specifies the target's conditional distribution, here Y given X , without specifying the distribution of the features themselves.

1. *Fit a generative model.* Choose a form for each class's feature distribution and fit its parameters to that class's training points. Estimate the class priors and use Bayes' theorem to obtain posteriors. Linear discriminant analysis (LDA) is an example. Under zero-one loss, predict the class maximizing $\pi_C f_C(x)$, equivalently its posterior.
2. *Fit a discriminative probability model.* Model $\mathbb{P}(Y | X)$ directly. Logistic regression is an example. Apply the loss-dependent decision rule to the fitted posteriors.
3. *Fit the decision rule directly.* Model $r(x)$ or a decision boundary without providing posterior probabilities. Support vector machines are an example.

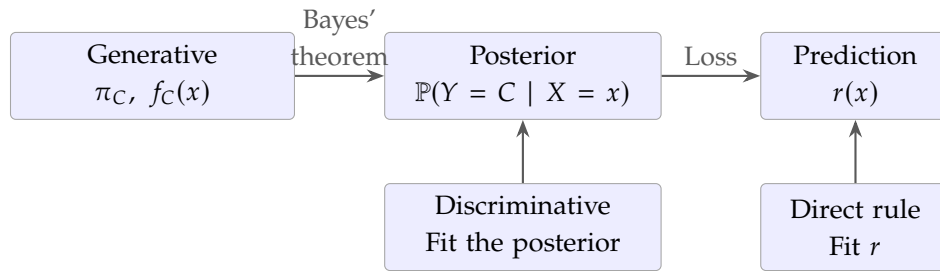


Figure 33: The three modeling routes supply different information. Both probabilistic routes use the loss to turn posterior probabilities into a prediction. A direct decision rule already supplies the prediction.

Both probabilistic approaches provide a model-based chance that a prediction is wrong: for a predicted class C , it is $1 - \mathbb{P}(Y = C | X = x)$. An ordinary SVM decision score alone does not provide this probability. Its accuracy as an uncertainty estimate depends on the quality of the probability model.

A generative model additionally supplies $f_X(x) = \sum_C \pi_C f_C(x)$, which can flag an observation lying in a low-density region as a possible **OUTLIER**. This is a density comparison, not the probability of an exact continuous value. The difficulty is fitting the feature distributions accurately; real data may not match the assumed family. Approximately normal class distributions and ample training data can make this modeling choice useful. Estimated posteriors and priors remain subject to error.

Revision. Risk is expected loss. The Bayes rule minimizes conditional expected loss at each input. Under zero-one loss, choose the largest posterior; with asymmetric errors, compare cost-weighted posteriors. Continuous probabilities are integrals of densities, and the Bayes risk is the integral of the smaller error contribution.

Common mistakes: confusing $\mathbb{P}(X | Y)$ with $\mathbb{P}(Y | X)$; ignoring priors; attaching an error cost to the wrong class probability; treating density height as probability; or treating an estimated Bayes rule as if the true distribution were known.

Check yourself: Why is the middle-category cancer posterior 5/14? Why does the asymmetric loss still predict 1 there? How does the pointwise minimization prove optimality? What is shaded in the continuous risk figure? Which quantities does each modeling route estimate?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 6: Decision Theory; Generative and Discriminative Models](#), PDF pp. 1–5 (printed pp. 30–34). Figures are original vector redraws of the supplied density, weighted-density, Gaussian, and modeling geometry. The conditional-risk proof, posterior threshold algebra, category risk calculations, and variance expansion complete the supplied arguments. Continuous conditioning and equal-posterior boundary sets are qualified where needed. Schematic distributions do not represent medical measurements; administration is omitted.

LECTURE 7: GAUSSIAN DISCRIMINANT ANALYSIS AND MAXIMUM LIKELIHOOD

7.1 Gaussian class models and discriminant scores

A generative classifier fits a feature distribution to each class, then combines those distributions with class frequencies. GAUSSIAN DISCRIMINANT ANALYSIS assumes a normal, or GAUSSIAN, distribution within each class. The fitted parameters determine both predictions and posterior probabilities.

An ISOTROPIC GAUSSIAN has the same spread in every direction. Its density in d dimensions is

$$f(x) = \frac{1}{(\sqrt{2\pi} \sigma)^d} \exp\left(-\frac{\|x - \mu\|^2}{2\sigma^2}\right), \quad \sigma > 0.$$

The input $x \in \mathbb{R}^d$ and mean $\mu \in \mathbb{R}^d$ are vectors; σ is a scalar standard deviation, and σ^2 is the variance of each coordinate. Density decreases with distance from μ , so its equal-density surfaces are spheres. For $d = 1$, this is the ordinary bell-shaped normal density.

For each class C , let f_C have mean μ_C and variance σ_C^2 , and let $\pi_C = \mathbb{P}(Y = C)$ be the prior. Under zero-one loss, predict the class maximizing $\pi_C f_C(x)$. Assume positive priors and variances. Taking the logarithm preserves the ordering and turns the products into sums. A common constant can also be removed without changing the prediction.

Definition 7.1 [QUADRATIC DISCRIMINANT SCORE]. For an isotropic Gaussian class model, define

$$\begin{aligned} Q_C(x) &= \ln((\sqrt{2\pi})^d \pi_C f_C(x)) \\ &= -\frac{\|x - \mu_C\|^2}{2\sigma_C^2} - d \ln \sigma_C + \ln \pi_C. \end{aligned}$$

The DISCRIMINANT SCORE compares classes. The first term penalizes distance from the class mean; the second accounts for its spread; the last accounts for its frequency.

For two classes C, D with zero cost for correct predictions, an asymmetric loss can be included in this comparison. Predict C when

$$L(D, C)\pi_C f_C(x) > L(C, D)\pi_D f_D(x).$$

For positive error costs, this compares $Q_C(x) + \ln L(D, C)$ with $Q_D(x) + \ln L(C, D)$. The cost of predicting D wrongly weights the evidence for C , and conversely. With several classes, a general loss requires minimizing $\sum_C L(z, C)\mathbb{P}(Y = C \mid X = x)$ over predictions z ; a single class-dependent multiplier need not express that loss.

7.2 Quadratic discriminant analysis

QUADRATIC DISCRIMINANT ANALYSIS (QDA) allows each class its own variance. With two classes, use the score difference $\Delta(x) = Q_C(x) - Q_D(x)$:

$$r(x) = \begin{cases} C, & \Delta(x) > 0, \\ D, & \Delta(x) \leq 0. \end{cases}$$

The zero set of Δ gives the equal-score threshold; it is a decision boundary where the preferred class changes across it.

Expanding the squared distances makes its geometry explicit:

$$\begin{aligned} \Delta(x) = & \left(\frac{1}{2\sigma_D^2} - \frac{1}{2\sigma_C^2} \right) \|x\|^2 + \left(\frac{\mu_C}{\sigma_C^2} - \frac{\mu_D}{\sigma_D^2} \right) \cdot x \\ & - \frac{\|\mu_C\|^2}{2\sigma_C^2} + \frac{\|\mu_D\|^2}{2\sigma_D^2} - d \ln \frac{\sigma_C}{\sigma_D} + \ln \frac{\pi_C}{\pi_D}. \end{aligned}$$

This is a polynomial of degree at most two. In one dimension, a nonzero quadratic has at most two real roots. Depending on the parameters, there can be two thresholds, one, or none. A repeated root need not separate different predictions. In several dimensions, the equal-score set is a QUADRIC; in two dimensions it is a conic or a degenerate case. For isotropic models, the quadratic coefficient multiplies $\|x\|^2$, so the nondegenerate curved sets are spheres. A general rotated or stretched quadric requires a more general Gaussian model.

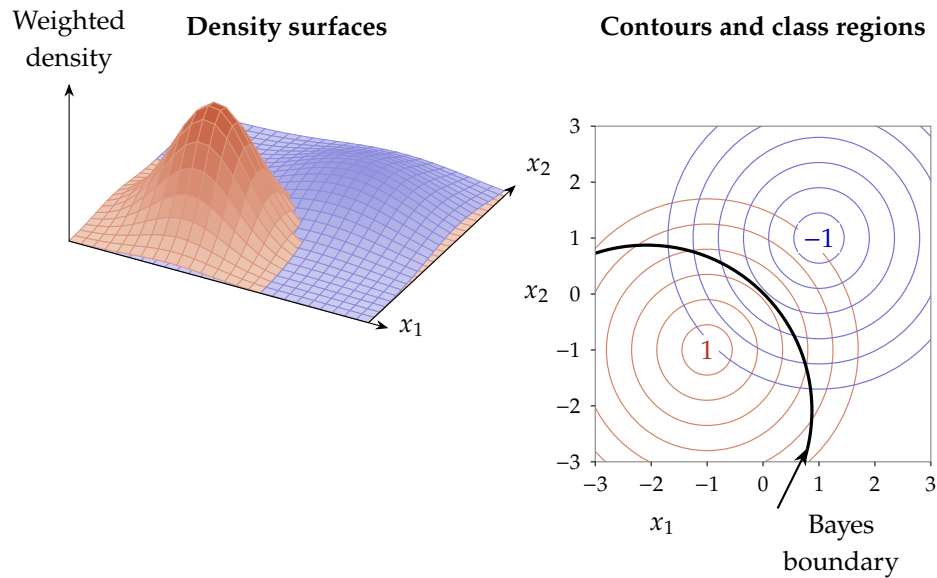


Figure 34: The same two schematic Gaussian models viewed as weighted density surfaces and contours. Different class spreads produce a curved equal-score boundary. Black marks the boundary; the colored contours describe the class densities.

For multiple classes, choose the largest $Q_C(x)$ at each input. Pairwise equal-score curves contribute to the boundary only where those classes tie for the largest score. Curved boundaries can meet at junctions. Since maximizing Q_C is equivalent to minimizing

$$\frac{\|x - \mu_C\|^2}{2\sigma_C^2} + d \ln \sigma_C - \ln \pi_C,$$

the partition is a MULTIPLICATIVELY AND ADDITIVELY WEIGHTED VORONOI DIAGRAM: the variance rescales distance, while spread and prior also add class-dependent offsets.

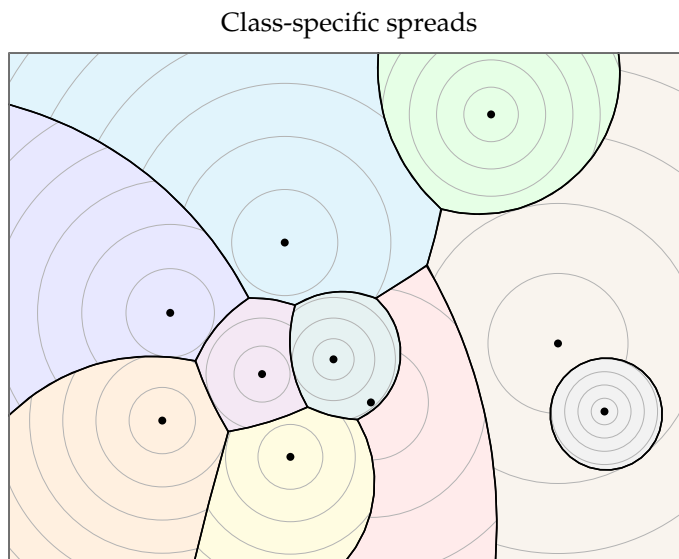


Figure 35: Schematic multiclass QDA with class-specific spreads. Dots mark class means, black curves separate winning classes, and circles show density contours within each prediction region. Some classes occupy small bounded regions.

7.3 Recovering posterior probabilities

The scores contain more information than the winning class. For two classes, Bayes' theorem gives

$$\begin{aligned} \mathbb{P}(Y = C \mid X = x) &= \frac{\pi_C f_C(x)}{\pi_C f_C(x) + \pi_D f_D(x)} \\ &= \frac{e^{Q_C(x)}}{e^{Q_C(x)} + e^{Q_D(x)}} = \frac{1}{1 + e^{-\Delta(x)}}. \end{aligned}$$

The common density-normalization factor cancels. Thus the posterior is a fixed increasing function of the discriminant difference.

Definition 7.2 [LOGISTIC FUNCTION]. The LOGISTIC FUNCTION, also called the SIGMOID, is

$$s(t) = \frac{1}{1 + e^{-t}}.$$

It satisfies $s(0) = 1/2$, approaches zero as $t \rightarrow -\infty$, and approaches one as $t \rightarrow \infty$. Its derivative $s'(t) = s(t)(1 - s(t)) > 0$ shows that it is strictly increasing.

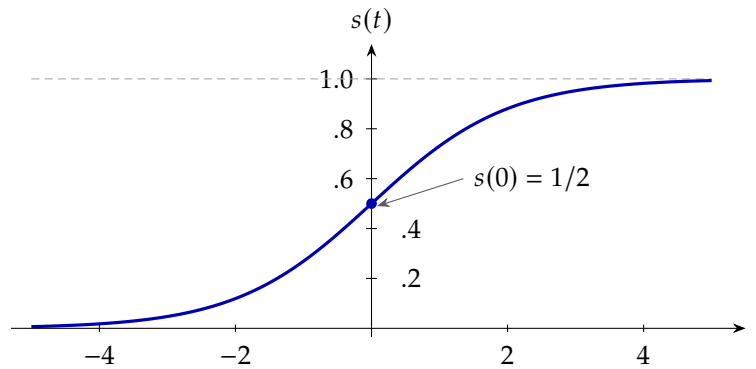


Figure 36: The logistic function converts a real discriminant difference into a class posterior. Zero difference means equal posterior probabilities; positive difference favors class C.

Under the two-class Gaussian model, $\mathbb{P}(Y = C \mid X = x) = s(\Delta(x))$ and $\mathbb{P}(Y = D \mid X = x) = 1 - s(\Delta(x))$. At a zero-one decision boundary, both equal $1/2$. An asymmetric-loss threshold generally does not correspond to posterior $1/2$. These are model-based probabilities; estimation error or a poor Gaussian assumption can make them inaccurate.

7.4 Equal variances give linear discriminants

LINEAR DISCRIMINANT ANALYSIS (LDA) uses the same variance σ^2 for every class. It constrains the model more than QDA, which can reduce overfitting, although the restriction can also fit some class distributions poorly.

Proposition 7.3 [CANCELLATION OF QUADRATIC TERMS]. *With common isotropic variance σ^2 , the two-class score difference has the affine form $\Delta(x) = w \cdot x + \alpha$, where*

$$w = \frac{\mu_C - \mu_D}{\sigma^2}, \quad \alpha = -\frac{\|\mu_C\|^2 - \|\mu_D\|^2}{2\sigma^2} + \ln \frac{\pi_C}{\pi_D}.$$

Proof. Both classes have the same $\|x\|^2$ and log-spread terms. Expanding gives

$$Q_C(x) = -\frac{\|x\|^2}{2\sigma^2} + \frac{\mu_C \cdot x}{\sigma^2} - \frac{\|\mu_C\|^2}{2\sigma^2} - d \ln \sigma + \ln \pi_C.$$

Subtract the corresponding expression for D . The terms $-\|x\|^2/(2\sigma^2)$ and $-d \ln \sigma$ cancel, leaving the stated w and α . $\square$

For any number of classes, remove those shared terms and choose C maximizing the LINEAR DISCRIMINANT FUNCTION

$$\delta_C(x) = \frac{\mu_C \cdot x}{\sigma^2} - \frac{\|\mu_C\|^2}{2\sigma^2} + \ln \pi_C.$$

The name “linear” includes an offset. With two distinct means, $w \neq 0$, the boundary is the hyperplane $w \cdot x + \alpha = 0$, and

$$\mathbb{P}(Y = C \mid X = x) = s(w \cdot x + \alpha).$$

In one dimension this scales and translates the sigmoid. In higher dimensions, the posterior varies only through the scalar $w \cdot x + \alpha$: moving parallel to the boundary leaves it unchanged. If $w = 0$, the comparison is constant rather than a separating hyperplane.

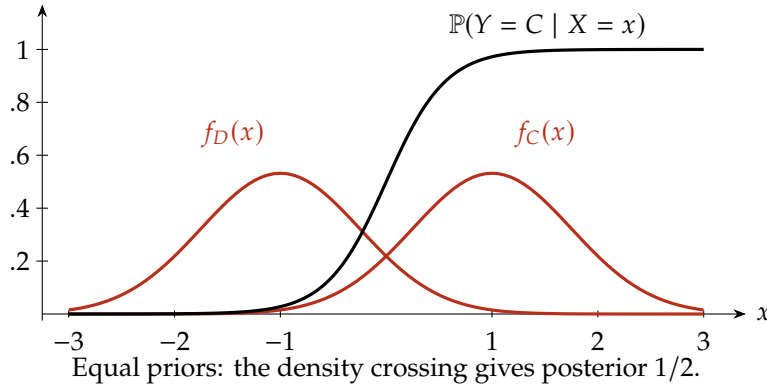


Figure 37: Schematic one-dimensional LDA. The equal-width red Gaussians have equal priors. The black posterior is the right-hand density divided by the sum of both densities. Red density heights and black posterior probabilities are different quantities.

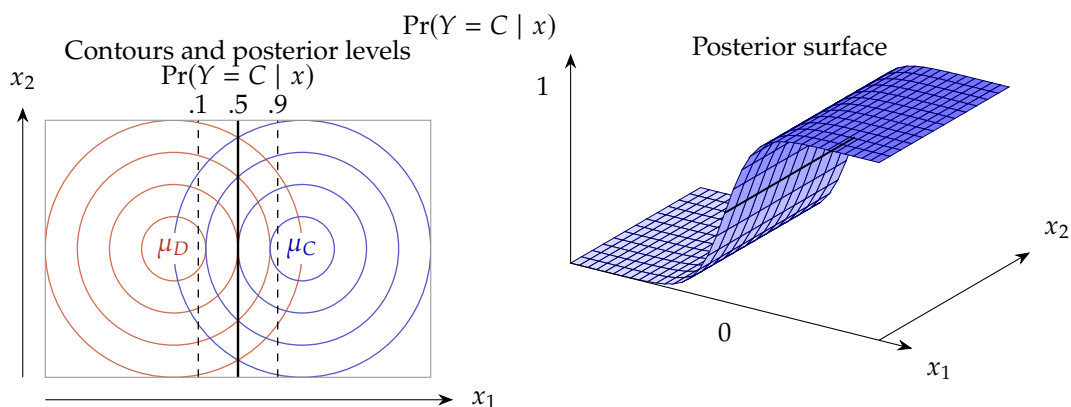


Figure 38: The same LDA structure in two dimensions. Left: Gaussian contours and parallel constant-posterior lines. Right: the posterior surface is a sigmoid in the normal direction and constant along the boundary direction. The geometry is schematic.

7.5 Equal priors recover the centroid method

With equal variances and equal priors, maximizing the discriminant is equivalent to minimizing $\|x - \mu_C\|^2$. Thus every input is assigned to its nearest class mean. For many classes, the prediction regions form an ordinary **VORONOI DIAGRAM**: each cell contains the points nearest to one mean.

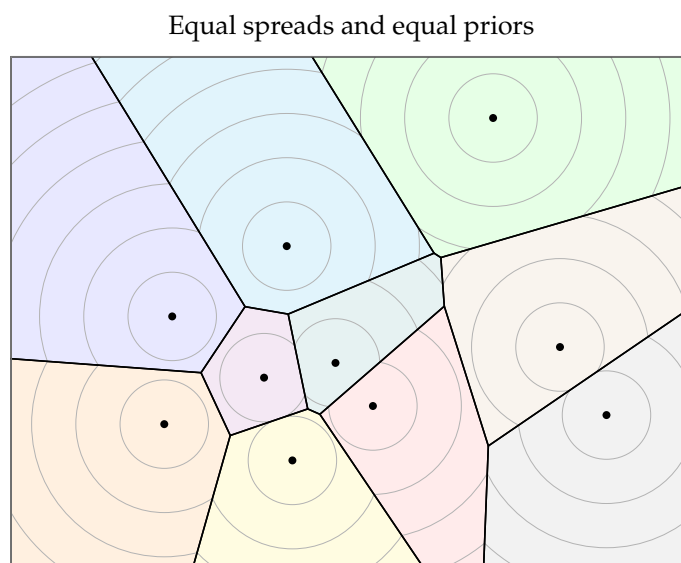


Figure 39: The class means from the QDA partition, now with equal spreads and equal priors. Nearest-mean regions have straight edges. Only the edge portions where the tied means are nearest contribute to the multiclass boundary.

For two classes with $\pi_C = \pi_D = 1/2$, the log-prior ratio vanishes. Multiplying the equal-score equation by σ^2 gives

$$\begin{aligned} 0 &= (\mu_C - \mu_D) \cdot x - \frac{\|\mu_C\|^2 - \|\mu_D\|^2}{2} \\ &= (\mu_C - \mu_D) \cdot \left(x - \frac{\mu_C + \mu_D}{2} \right). \end{aligned}$$

The second equality uses $\|\mu_C\|^2 - \|\mu_D\|^2 = (\mu_C - \mu_D) \cdot (\mu_C + \mu_D)$. For distinct means this is their perpendicular bisector, exactly the centroid classifier. Unequal priors change the offset and move the boundary; they do not rotate its normal when means and variance are held fixed.

7.6 Maximum likelihood and class priors

Gaussian classifiers need fitted means, variances, and priors. MAXIMUM LIKELIHOOD ESTIMATION (MLE) chooses parameters under which the observed data have the largest likelihood. The observations are held fixed while the parameters vary.

Example: A biased coin lands heads with probability p . In ten independent flips, eight are heads and two are tails. If H counts heads, its binomial distribution gives

$$\mathbb{P}(H = h) = \binom{n}{h} p^h (1 - p)^{n-h}.$$

For the observed count $h = 8$ and $n = 10$, the LIKELIHOOD FUNCTION is

$$\mathcal{L}(p) = \binom{10}{8} p^8 (1 - p)^2 = 45 p^8 (1 - p)^2, \quad 0 \leq p \leq 1.$$

The parameter estimate maximizes this function of p . It does not assign a probability distribution to p itself.

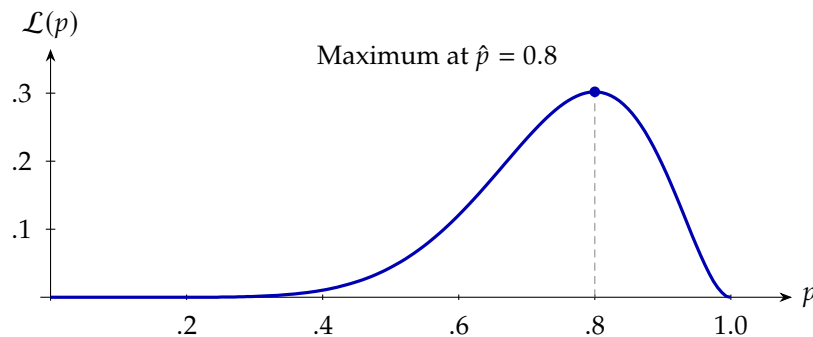


Figure 40: The probability of eight heads in ten flips as a function of the coin parameter. This likelihood is maximized at $p = 0.8$; its horizontal axis is a parameter, not a possible data outcome.

Differentiating and factoring yields

$$\begin{aligned}\mathcal{L}'(p) &= 360p^7(1-p)^2 - 90p^8(1-p) \\ &= 90p^7(1-p)(4-5p).\end{aligned}$$

For $0 < p < 1$, the derivative is positive below 0.8 and negative above it. The endpoints give zero likelihood, so the global maximum is $\hat{p} = 0.8$. Also $\mathcal{L}''(0.8) \approx -18.9 < 0$, confirming local curvature. A hat marks an estimated parameter.

The same calculation estimates a class prior: if eight of ten training labels are C , then $\hat{\pi}_C = 0.8$. For class counts n_C and total $n = \sum_D n_D$, the estimates are

$$\hat{\pi}_C = \frac{n_C}{n}.$$

They describe the class frequencies in the sampled population. Fitting a probability density from observations is called **DENSITY ESTIMATION**; MLE is one way to fit a chosen family.

7.7 Maximum likelihood for an isotropic Gaussian

Suppose $X_1, \dots, X_n \in \mathbb{R}^d$ are independent observations from the same isotropic Gaussian. The likelihood is the joint *density* at those observations:

$$\mathcal{L}(\mu, \sigma; X_1, \dots, X_n) = \prod_{i=1}^n f_{\mu, \sigma}(X_i).$$

Each exact continuous observation has probability zero. Using its density in the likelihood is well defined and does not claim that an exact-point probability is positive.

The **LOG LIKELIHOOD** is $\ell = \ln \mathcal{L}$. Since the logarithm is strictly increasing, it has the same maximizers. Expanding the product into a sum gives

$$\begin{aligned}\ell(\mu, \sigma) &= \sum_{i=1}^n \left[-\frac{\|X_i - \mu\|^2}{2\sigma^2} - \frac{d}{2} \ln(2\pi) - d \ln \sigma \right] \\ &= -\frac{1}{2\sigma^2} \sum_{i=1}^n \|X_i - \mu\|^2 - \frac{nd}{2} \ln(2\pi) - nd \ln \sigma.\end{aligned}$$

First optimize the mean. Its gradient is

$$\nabla_{\mu} \ell = \frac{1}{\sigma^2} \sum_{i=1}^n (X_i - \mu).$$

Setting it to zero gives the SAMPLE MEAN

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n X_i.$$

For any fixed $\sigma > 0$, this is the unique maximizing mean:

$$\sum_i \|X_i - \mu\|^2 = \sum_i \|X_i - \hat{\mu}\|^2 + n\|\mu - \hat{\mu}\|^2.$$

To verify the identity, expand around $\hat{\mu}$. The cross term vanishes because $\sum_i (X_i - \hat{\mu}) = 0$.

Now differentiate with respect to σ :

$$\frac{\partial \ell}{\partial \sigma} = \frac{\sum_i \|X_i - \mu\|^2 - nd\sigma^2}{\sigma^3}.$$

Substituting the fitted mean and setting the numerator to zero gives

$$\hat{\sigma}^2 = \frac{1}{dn} \sum_{i=1}^n \|X_i - \hat{\mu}\|^2.$$

The factor d averages over coordinates as well as observations. For $d = 1$, the denominator is n ; this is the MLE variance, not the unbiased estimate with denominator $n - 1$. If the residual sum is positive, the derivative is positive below $\hat{\sigma}$ and negative above it, proving global maximality for $\sigma > 0$ after fitting μ .

Note. The Gaussian density requires positive variance. If all observations coincide, the residual sum is zero and the likelihood grows without bound as $\sigma \rightarrow 0$ at that common mean. There is then no finite maximum with $\sigma > 0$; the formal zero variance cannot be substituted into an ordinary Gaussian density.

7.8 Fitting QDA and LDA to labeled data

For labeled observations (X_i, y_i) , let $I_C = \{i : y_i = C\}$ and $n_C = |I_C|$. QDA fits each class separately:

$$\hat{\mu}_C = \frac{1}{n_C} \sum_{i \in I_C} X_i, \quad \hat{\sigma}_C^2 = \frac{1}{dn_C} \sum_{i \in I_C} \|X_i - \hat{\mu}_C\|^2, \quad \hat{\pi}_C = \frac{n_C}{n}.$$

These formulas require observations in each fitted class and positive class residual sums to give nondegenerate Gaussian densities. Insert the estimates into $Q_C(x)$ and compare the scores.

LDA uses the same fitted means and priors, but a single POOLED WITHIN-CLASS VARIANCE:

$$\hat{\sigma}^2 = \frac{1}{dn} \sum_C \sum_{i \in I_C} \|X_i - \hat{\mu}_C\|^2.$$

The common-variance log likelihood contains precisely this total within-class residual sum. Its derivative gives the same $S/(dn)$ formula as for one Gaussian. Pooling combines information from all classes about their shared spread.

Each observation is compared with its *own class mean*, rather than with the global mean. Distances between class centers should not count as within-class spread. Equivalently,

$$\hat{\sigma}^2 = \sum_C \frac{n_C}{n} \hat{\sigma}_C^2.$$

Thus the pooled variance averages the class-specific MLE variances, weighted by their sample sizes. It requires a positive total within-class residual sum.

Revision. Gaussian discriminants combine distance, spread, and prior. QDA allows class-specific spreads; LDA's shared spread cancels the quadratic terms. A binary posterior is the sigmoid of the score difference. Equal spreads and priors recover nearest-centroid classification. MLE fits priors by frequencies, means by averages, and variances by average squared residuals. *Common mistakes:* dropping the spread or prior term from QDA; calling a density a probability; interpreting likelihood as a posterior over parameters; using $n - 1$ for the MLE variance; forgetting the factor d ; or pooling around the global mean.

Check yourself: Which terms cancel in LDA? Why does the posterior equal 1/2 at an equal-score boundary? How do equal priors yield the centroid bisector? Why is 0.8 the global coin-likelihood maximum? Why are LDA residuals measured from class-specific means?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 7: Gaussian Discriminant Analysis, including QDA and LDA](#), PDF pp. 1–5 (printed pp. 35–39). Figures are original vector redraws of the supplied Gaussian, classification-partition, logistic, LDA, and likelihood geometry. The discriminant expansion, cancellation proof, centroid identity, derivative signs, and Gaussian residual identity complete the supplied algebra and optimization claims. Boundary roots, isotropic geometry, independence, continuous likelihood, and zero-variance cases are qualified for correctness. Schematic shapes and class means are not empirical measurements; administration is omitted.

LECTURE 8: EIGENDECOMPOSITION, QUADRATIC FORMS, AND ANISOTROPIC GAUSSIANS

8.1 Eigenvectors preserve a line

A matrix usually changes both a vector's length and its direction. Some directions are special: multiplying a vector in that direction keeps it on the same line. These directions make the matrix's geometry easier to understand.

Definition 8.1 [EIGENVECTOR AND EIGENVALUE]. For a real square matrix A , a nonzero real vector v is an **EIGENVECTOR** with **EIGENVALUE** λ when, for $\lambda \in \mathbb{R}$,

$$Av = \lambda v.$$

If $\lambda > 0$, the direction is preserved; if $\lambda < 0$, it is reversed. If $\lambda = 0$, the vector maps to zero. Its length is multiplied by $|\lambda|$.

Every nonzero multiple of v is another eigenvector with the same value, since $A(cv) = cAv = \lambda(cv)$. Thus an eigenvector specifies a direction, rather than a particular length.

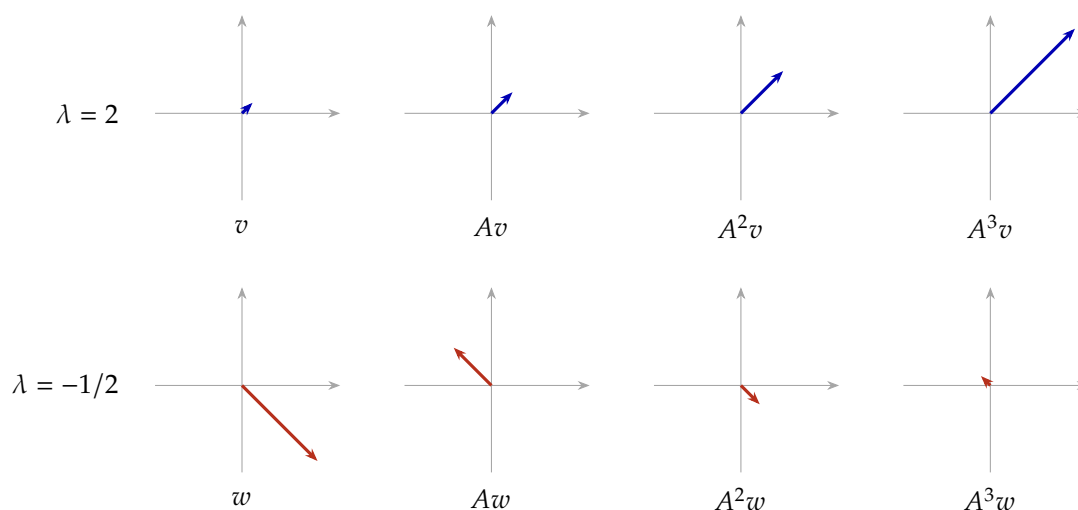


Figure 41: Repeated multiplication in two eigendirections. An eigenvalue of 2 doubles length at each step. An eigenvalue of $-1/2$ halves length and reverses direction at each step. Each row uses a common display scale.

Proposition 8.2 [POWERS AND INVERSES]. *If $Av = \lambda v$, then $A^k v = \lambda^k v$ for every positive integer k . If A is invertible, then $\lambda \neq 0$ and $A^{-1}v = \lambda^{-1}v$.*

Proof. Applying A once more multiplies the same vector by another λ : if $A^k v = \lambda^k v$, then $A^{k+1}v = A(\lambda^k v) = \lambda^{k+1}v$. The case $k = 1$ starts this induction.

If A is invertible, $Av = 0$ would imply $v = 0$, so an eigenvalue cannot be zero. Multiplying $Av = \lambda v$ by A^{-1} gives $v = \lambda A^{-1}v$, hence $A^{-1}v = v/\lambda$. $\square$

Squaring preserves the chosen eigenvectors and squares their values. Inverting preserves them and takes reciprocal values. Squaring can also merge distinct eigenvalues, such as λ and $-\lambda$, so the squared matrix may have additional eigenvector directions.

8.2 Symmetric matrices and quadratic forms

Symmetry gives a complete set of perpendicular directions in which the matrix acts by scalar multiplication.

Theorem 8.3 [SPECTRAL THEOREM]. *Every real symmetric $n \times n$ matrix has real eigenvalues and an orthonormal basis $v_1, \dots, v_n$ of eigenvectors:*

$$Av_i = \lambda_i v_i, \quad v_i^\top v_j = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

ORTHONORMAL means mutually perpendicular and unit length. The basis need not be unique. If $Av = \lambda v$ and $Aw = \lambda w$, every nonzero linear combination of v, w is also an eigenvector with value λ . A repeated eigenvalue can therefore have infinitely many eigendirections within its EIGENSPACE. Choose an orthonormal basis there. The eigenvalues, including their multiplicities, are determined by the matrix even when the basis choices differ.

A QUADRATIC FORM of a symmetric matrix M is the scalar $x^\top M x$. For the identity matrix it is $z^\top z = \|z\|^2$, whose positive level sets are spheres. For symmetric, invertible A ,

$$\|A^{-1}x\|^2 = x^\top (A^{-1})^\top A^{-1} x = x^\top A^{-2} x.$$

Symmetry justifies the second equality. This measures squared length after undoing the transformation $x = Az$.

Example: Consider the matrix

$$A = \begin{bmatrix} 3/4 & 5/4 \\ 5/4 & 3/4 \end{bmatrix}, \quad v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad v_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Direct multiplication gives $Av_1 = 2v_1$ and $Av_2 = -v_2/2$. It stretches the first eigendirection by 2 and reverses and shrinks the second by 1/2. The quadratic form of A^{-2} has eigenvalues 1/4 and 4, so it rises slowly along v_1 and quickly along v_2 .

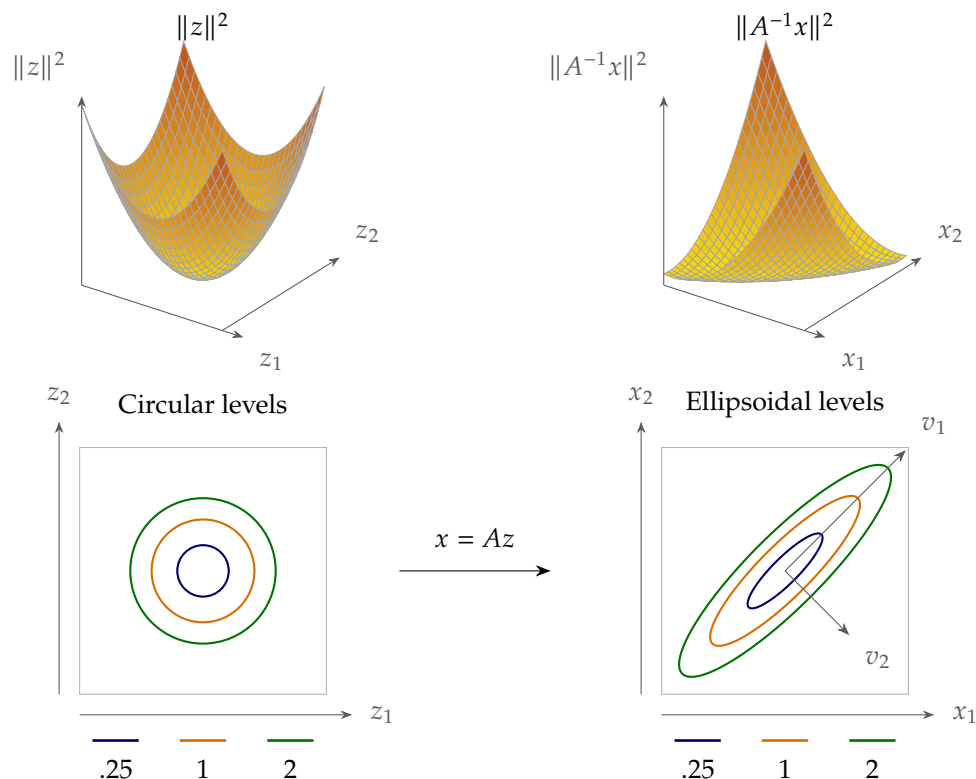


Figure 42: The transformation $x = Az$ sends circular squared-length contours to ellipses. Top: the corresponding quadratic bowls, with separate height display scales. Bottom: the level sets $\|z\|^2 = c$ and $\|A^{-1}x\|^2 = c$. The long ellipse axis is parallel to v_1 ; the short axis is parallel to v_2 .

For an orthonormal eigenbasis of symmetric, invertible A , write $x = \sum_i a_i v_i$. Then

$$\|A^{-1}x\|^2 = \sum_{i=1}^n \frac{a_i^2}{\lambda_i^2}.$$

Consequently the unit level is an **ELLIPSOID** with axes v_i and semiaxis lengths $|\lambda_i|$. In particular, $x = \lambda_i v_i$ lies on that level because $A^{-1}x = v_i$. An eigenvalue's sign changes orientation under A , but does not make a geometric radius negative.

If A is diagonal, the coordinate vectors are an eigenbasis and the ellipsoid is axis-aligned. Conversely, a symmetric matrix whose coordinate vectors are eigenvectors is diagonal. Repeated eigenvalues allow other eigenbasis choices as well.

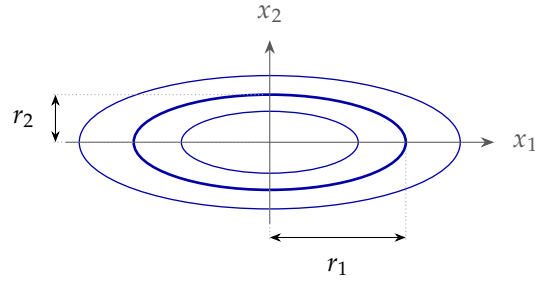


Figure 43: Schematic level sets for a diagonal transformation. Each ellipsoid axis follows a coordinate axis. At the unit level, the semiaxis lengths are the absolute diagonal entries of A .

8.3 Eigenvalue signs determine curvature

The eigenbasis turns a quadratic form into a sum of independent coordinate contributions. If $Mv_i = \eta_i v_i$ and $x = \sum_i a_i v_i$, orthonormality gives

$$x^T M x = \sum_{i=1}^n \eta_i a_i^2.$$

A positive value makes the graph rise in that direction, a negative value makes it fall, and a zero value leaves it flat.

Property of symmetric M	Quadratic-form condition	Eigenvalue condition
Positive definite	$x^T M x > 0$ for $x \neq 0$	All $\eta_i > 0$
Positive semidefinite	$x^T M x \geq 0$ for all x	All $\eta_i \geq 0$
Indefinite	Takes positive and negative values	Both signs occur
Invertible	Has no nonzero null vector	No $\eta_i = 0$

These equivalences follow directly from the sum. If all eigenvalues are positive, every nonzero x has a positive contribution. Conversely, choosing $x = v_i$ isolates η_i . The same reasoning gives the semidefinite and indefinite cases. A zero eigenvalue gives a null vector; with no zero eigenvalues, each eigenbasis coordinate can be recovered by division.

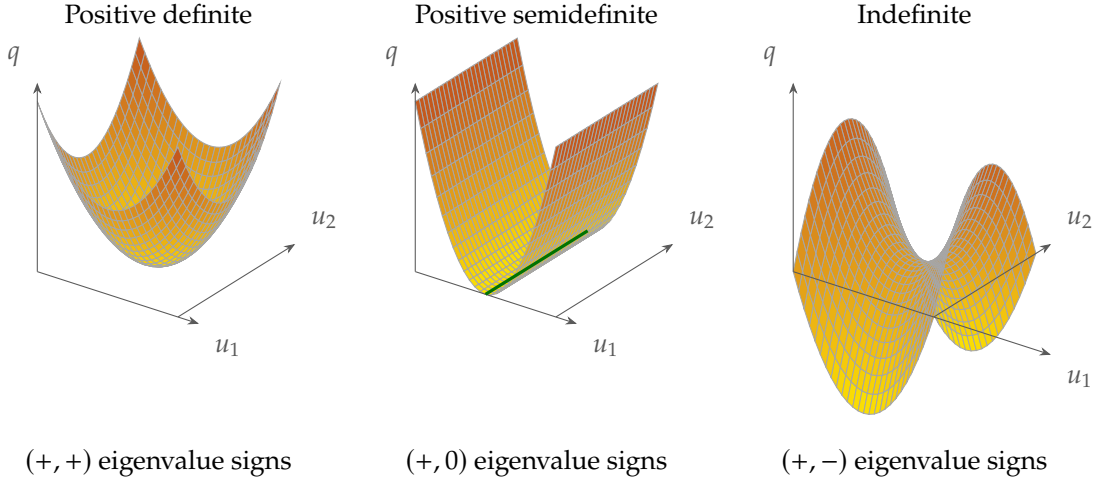


Figure 44: Quadratic-form geometry in eigenvector coordinates. Positive definite: a bowl rises in both directions. Positive semidefinite with one zero eigenvalue: a trough is flat along the green line. Indefinite: a saddle rises in one direction and falls in the other. Heights are schematic.

For symmetric, invertible A , the eigenvalues of A^{-2} are $1/\lambda_i^2 > 0$. Thus A^{-2} is positive definite even if A has negative eigenvalues.

More generally, for positive definite M , the level $x^T M x = c$ with $c > 0$ has semiaxis lengths

$$r_i = \sqrt{\frac{c}{\eta_i}}.$$

To see this, set all eigenbasis coordinates except a_i to zero; then $\eta_i a_i^2 = c$. At $c = 1$, the radius is $1/\sqrt{\eta_i}$. A larger eigenvalue of the quadratic-form matrix therefore means a *shorter* axis.

If M is positive semidefinite with zero eigenvalues, the quadratic form ignores those coordinates. When positive eigenvalues exist, a positive level is an ellipsoidal cylinder extending along the zero-eigenvalue directions. Its zero level is the nullspace. If $M = 0$, no positive level exists.

8.4 Constructing a matrix from its axes

We can also start with desired perpendicular axes and scaling factors, then build the symmetric matrix that has them. Let $v_1, \dots, v_n$ be an orthonormal basis and set

$$V = \begin{bmatrix} v_1 & v_2 & \cdots & v_n \end{bmatrix}, \quad \Lambda = \text{diag}(\lambda_1, \dots, \lambda_n).$$

The (i, j) entry of $V^T V$ is $v_i^T v_j$, so $V^T V = I$. Since V is square, $V^{-1} = V^T$ and $V V^T = I$. Such an ORTHOGONAL MATRIX preserves lengths and angles; it acts as a rotation or reflection.

Theorem 8.4 [EIGENDECOMPOSITION]. *The symmetric matrix with the chosen eigenvectors and eigenvalues is*

$$A = V\Lambda V^T = \sum_{i=1}^n \lambda_i v_i v_i^T.$$

Conversely, every real symmetric matrix admits this EIGENDECOMPOSITION.

Proof. The eigenvector equations together are $AV = V\Lambda$. Multiply on the right by V^T to obtain $A = V\Lambda V^T$. Conversely, a matrix defined by that expression satisfies

$$Av_j = \sum_i \lambda_i v_i (v_i^T v_j) = \lambda_j v_j,$$

and is symmetric. The spectral theorem provides the required orthonormal basis for every real symmetric matrix. $\square$

The OUTER PRODUCT $v_i v_i^T$ is an $n \times n$ rank-one matrix: it maps x to $v_i(v_i^T x)$, the component of x along v_i . Each weighted outer product contributes one eigendirection. The factor V^T first expresses a vector in that basis, Λ scales each coordinate, and V returns it to the original coordinates. In particular, $V^T A V = \Lambda$ diagonalizes A .

For chosen positive ellipsoid radii r_i , take $A = V \text{diag}(r_i) V^T$. Then

$$x^T A^{-2} x = \sum_i \frac{(v_i^T x)^2}{r_i^2},$$

whose unit level has exactly those axes and radii. The equation specifies a level set; the graph of the full quadratic form is a paraboloid.

8.5 Matrix powers and a symmetric square root

Adjacent factors $V^T V$ cancel, leaving

$$A^2 = V\Lambda^2 V^T, \quad A^{-2} = V\Lambda^{-2} V^T \quad \text{if } A \text{ is invertible.}$$

Matrix powers act on the diagonal eigenvalues, while retaining the chosen eigenvectors. This also gives a way to take a square root.

If a symmetric positive semidefinite matrix Σ has

$$\Sigma = V \text{diag}(\rho_1, \dots, \rho_n) V^\top, \quad \rho_i \geq 0,$$

define its symmetric positive semidefinite **MATRIX SQUARE ROOT** by

$$\Sigma^{1/2} = V \text{diag}(\sqrt{\rho_1}, \dots, \sqrt{\rho_n}) V^\top.$$

Squaring this expression gives Σ . The nonnegative roots select the positive semidefinite square root; arbitrary sign choices need not have that property.

Algorithm 8.1: Symmetric positive semidefinite square root

Input: A real symmetric positive semidefinite matrix $\Sigma \in \mathbb{R}^{n \times n}$.

- 1 Compute an orthonormal eigendecomposition $\Sigma = V \text{diag}(\rho_1, \dots, \rho_n) V^\top$
 - 2 **for** $i \leftarrow 1$ **to** n **do**
 - 3 $b_i \leftarrow \sqrt{\rho_i}$
 - 4 $B \leftarrow V \text{diag}(b_1, \dots, b_n) V^\top$
 - 5 **return** B
-

Finding the eigenvectors and eigenvalues is the main computational step. Taking scalar square roots and reassembling the matrix are simpler. If Σ is positive definite, its inverse square root is also defined:

$$\Sigma^{-1/2} = V \text{diag}(\rho_1^{-1/2}, \dots, \rho_n^{-1/2}) V^\top.$$

8.6 Anisotropic Gaussian densities

An isotropic Gaussian has the same variance in every direction. An **ANISOTROPIC GAUSSIAN** permits different spreads along different perpendicular directions, including tilted axes. For $X \in \mathbb{R}^d$, write $X \sim \mathcal{N}(\mu, \Sigma)$, where $\mu \in \mathbb{R}^d$ and Σ is a symmetric positive definite $d \times d$ matrix. Its density is

$$f(x) = \frac{1}{\sqrt{(2\pi)^d \det \Sigma}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right).$$

The determinant is positive. The matrix Σ is the **COVARIANCE MATRIX**; its inverse Σ^{-1} is the **PRECISION MATRIX**. The covariance interpretation will be verified through the coordinate representation below.

The density combines two functions:

$$q(x) = (x - \mu)^\top \Sigma^{-1}(x - \mu), \quad n(t) = \frac{e^{-t/2}}{\sqrt{(2\pi)^d \det \Sigma}}, \quad f(x) = n(q(x)).$$

The first is a quadratic bowl centered at μ ; the second is a strictly decreasing exponential. Indeed $n'(t) = -n(t)/2 < 0$ and $n''(t) = n(t)/4 > 0$. It reverses the height ordering without changing the level-set geometry.

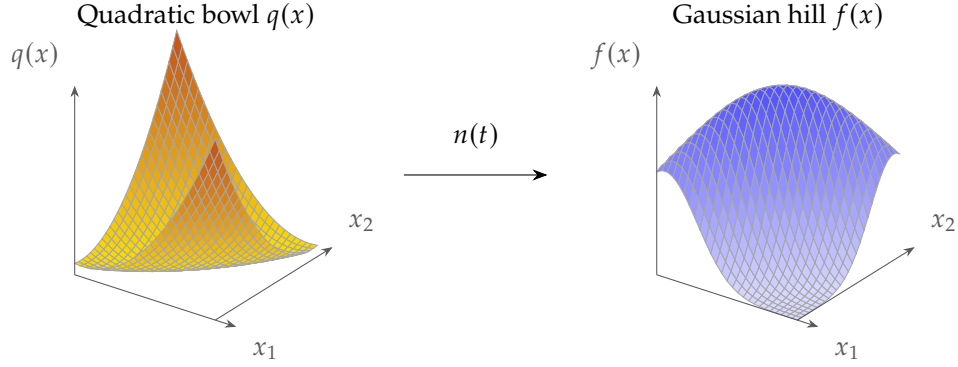


Figure 45: The quadratic bowl becomes a Gaussian hill under $f = n \circ q$. These illustrations use $\mu = 0$ and $\Sigma = A^2$ for the matrix A above. The mean minimizes q and maximizes f . The surfaces have separate height display scales.

Because n is one-to-one,

$$f(x) = n(c) \iff q(x) = c.$$

Thus quadratic and Gaussian contours are the same ellipsoids with different labels. For $\Sigma v_i = \rho_i v_i$, the contour $q(x) = c$ has radii $\sqrt{c\rho_i}$ along v_i . At $c = 1$, the squared radii are the eigenvalues of Σ , while the eigenvalues of Σ^{-1} are their reciprocals.

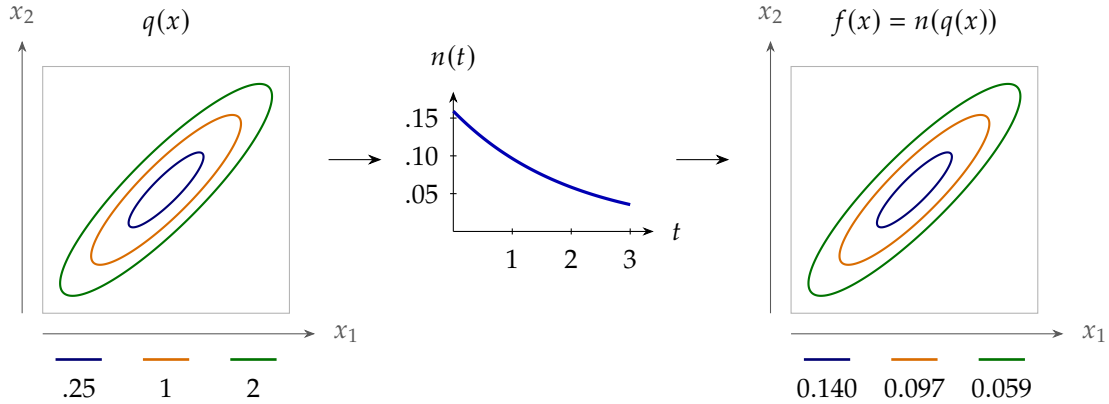


Figure 46: Corresponding contours for the same two-dimensional model. Equal colors pair a quadratic level c with density $n(c)$; labels below the plots list their values. The middle curve is $n(t) = e^{-t/2}/(2\pi)$, since $\det(A^2) = 1$. Density labels are rounded.

Since $q(x) \geq 0$ and vanishes only at μ , the Gaussian has its unique maximum there. Positive definiteness also gives $q(x) \geq \|x - \mu\|^2/\rho_{\max}$, so $f(x) \rightarrow 0$ as $\|x - \mu\| \rightarrow \infty$. If $\Sigma = \sigma^2 I$, the determinant is σ^{2d} and the expression reduces to the isotropic density.

8.7 Precision as a transformed distance

The inverse square root converts the ellipsoidal distance into an ordinary Euclidean distance:

$$q(x) = \|\Sigma^{-1/2}(x - \mu)\|^2,$$

$$d_\Sigma(x, \mu) = \sqrt{q(x)} = \|\Sigma^{-1/2}x - \Sigma^{-1/2}\mu\|.$$

The equality uses symmetry and $(\Sigma^{-1/2})^\top \Sigma^{-1/2} = \Sigma^{-1}$. Along an eigenvector with variance ρ_i , this transformation divides distances by $\sqrt{\rho_i}$. A given displacement therefore counts less along a direction in which the distribution is wider.

Using the same expression between any two points defines a **METRIC**: it is nonnegative, symmetric, zero only for equal points, and obeys the triangle inequality. These properties follow from the Euclidean norm and the invertibility of $\Sigma^{-1/2}$. The precision matrix specifies this warped distance.

8.8 Covariance and Gaussian independence

For random column vectors R, S with finite second moments, write $\mu_R = \mathbb{E}[R]$ and $\mu_S = \mathbb{E}[S]$. Their **COVARIANCE** is

$$\begin{aligned} \text{Cov}(R, S) &= \mathbb{E}[(R - \mu_R)(S - \mu_S)^\top] \\ &= \mathbb{E}[RS^\top] - \mu_R \mu_S^\top. \end{aligned}$$

Expanding the product gives the second identity; the two cross terms each have expectation $-\mu_R \mu_S^\top$, while the constant term restores one copy. For scalar variables the transpose has no effect. Define $\text{Var}(R) = \text{Cov}(R, R)$. For a d -vector this is the matrix

$$\text{Var}(R) = \begin{bmatrix} \text{Var}(R_1) & \text{Cov}(R_1, R_2) & \cdots & \text{Cov}(R_1, R_d) \\ \text{Cov}(R_2, R_1) & \text{Var}(R_2) & \cdots & \text{Cov}(R_2, R_d) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(R_d, R_1) & \text{Cov}(R_d, R_2) & \cdots & \text{Var}(R_d) \end{bmatrix}.$$

It is symmetric: the scalar off-diagonal entries are unchanged when the indices are swapped. Diagonal entries are individual coordinate variances; off-diagonal entries describe covariation.

Independent scalar variables have zero covariance, because $\mathbb{E}[R_i R_j] = \mathbb{E}[R_i] \mathbb{E}[R_j]$. The reverse implication fails for general distributions. For *jointly Gaussian* variables, however, zero covariance between two coordinates implies their independence. The assumption concerns their joint distribution; Gaussian marginals alone do not suffice.

If $X \sim \mathcal{N}(\mu, \Sigma)$ and $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$, then

$$q(x) = \sum_{j=1}^d \frac{(x_j - \mu_j)^2}{\sigma_j^2}, \quad \det \Sigma = \prod_{j=1}^d \sigma_j^2.$$

Substitute these expressions into the density and split the exponential of a sum into a product:

$$f(x) = \prod_{j=1}^d \left[\frac{1}{\sqrt{2\pi} \sigma_j} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right) \right].$$

Each factor is a univariate Gaussian density, so the coordinates are mutually independent. For a joint Gaussian, diagonal covariance, axis-aligned contours, and this product form are equivalent.

A tilted Gaussian has the same product structure in its eigenvector coordinates. Write $\Sigma = V \text{diag}(\rho_i) V^\top$ and define $Y = V^\top(X - \mu)$. Then

$$q(x) = \sum_i \frac{y_i^2}{\rho_i}, \quad \det \Sigma = \prod_i \rho_i.$$

The orthogonal change of coordinates has absolute determinant 1, so it does not change the volume element. The density of Y factors into independent $\mathcal{N}(0, \rho_i)$ densities. Consequently $\mathbb{E}[Y] = 0$ and $\text{Var}(Y) = \text{diag}(\rho_i)$. Since $X = \mu + VY$,

$$\mathbb{E}[X] = \mu, \quad \text{Var}(X) = V \text{Var}(Y) V^\top = V \text{diag}(\rho_i) V^\top = \Sigma.$$

This confirms that the matrix in the Gaussian density is its covariance. The original coordinates can be dependent even though the eigenvector coordinates are independent.

Revision. An eigenvector stays on one line under a matrix. Symmetric matrices have an orthonormal eigenbasis. In that basis, quadratic forms are sums of eigenvalues times squared coordinates, and $A = V\Lambda V^T$. Gaussian contours are quadratic-form ellipsoids: covariance eigenvalues give squared unit-contour radii; precision eigenvalues give their reciprocals.

Common mistakes: assigning a negative geometric radius; forgetting symmetry in $\|A^{-1}x\|^2 = x^T A^{-2}x$; using eigenvalues rather than inverse square roots for the radii of $x^T Mx = 1$; treating a merely monotone function as one-to-one; or inferring independence from zero covariance without a joint Gaussian assumption.

Check yourself: What happens to eigenvalues under powers and inverses? Why is A^{-2} positive definite? Which coordinates are ignored by a zero eigenvalue? How does an eigendecomposition produce a matrix square root? Why do q and $n \circ q$ have the same contours? In which coordinates does a tilted Gaussian factor?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 8: Eigenvectors and the Anisotropic Multivariate Normal Distribution](#), PDF pp. 1–6 (printed pp. 40–45). Figures are original vector redraws of the supplied eigenvector, quadratic-form, axis-aligned, curvature, and Gaussian geometry. Induction, inverse and orthonormal-coordinate arguments, eigenvalue-sign conditions, eigendecomposition, covariance expansion, and Gaussian factorization complete the supplied calculations. The spectral theorem is stated without its general proof. Absolute radii, possible extra eigendirections after squaring, semidefinite degeneracies, and strict monotonicity qualify the claims where needed. Gaussian figures consistently use $\Sigma = A^2$ for the supplied matrix; schematic display scales are not empirical measurements.

LECTURE 9: ANISOTROPIC GAUSSIANS, QDA, AND LDA

9.1 Covariance, widths, and precision

A Gaussian can spread farther in one direction than another, and its axes can be tilted. Three related matrices describe that geometry. Let Σ be symmetric positive definite, with $\Sigma = V\Lambda V^\top$ and $\Lambda = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$. The columns v_i of V form an orthonormal eigenbasis.

Matrix	Eigenvalues along v_i	Role
$\Sigma = V\Lambda V^\top$	σ_i^2	Covariance: directional variances
$\Sigma^{1/2} = V\Lambda^{1/2}V^\top$	σ_i	Widths: sphere to ellipsoid
$\Sigma^{-1} = V\Lambda^{-1}V^\top$	$1/\sigma_i^2$	Precision: quadratic distance

The transformation $x = \mu + \Sigma^{1/2}z$ maps $\|z\| = 1$ to an ellipsoid centered at μ , with semiaxis σ_i along v_i . Its inverse removes both the location and the unequal widths: $z = \Sigma^{-1/2}(x - \mu)$.

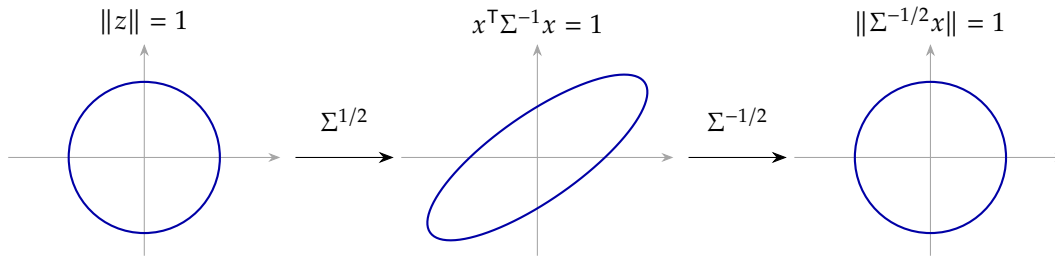


Figure 47: Covariance square roots stretch a sphere into an ellipsoid; inverse square roots undo that stretch. The drawing uses $\mu = 0$. Widths are square roots of covariance eigenvalues.

For $X \sim \mathcal{N}(\mu, \Sigma)$, the density is

$$f(x) = n(q(x)), \quad q(x) = (x - \mu)^\top \Sigma^{-1} (x - \mu), \quad n(t) = \frac{e^{-t/2}}{\sqrt{(2\pi)^d \det \Sigma}}.$$

The precision matrix defines the quadratic bowl. The strictly decreasing function n converts it into a Gaussian hill. The contours are unchanged: $f(x) = n(c)$ exactly when $q(x) = c$. The matrix Σ^{-1} is also called a **METRIC TENSOR** here because it specifies the squared transformed distance.

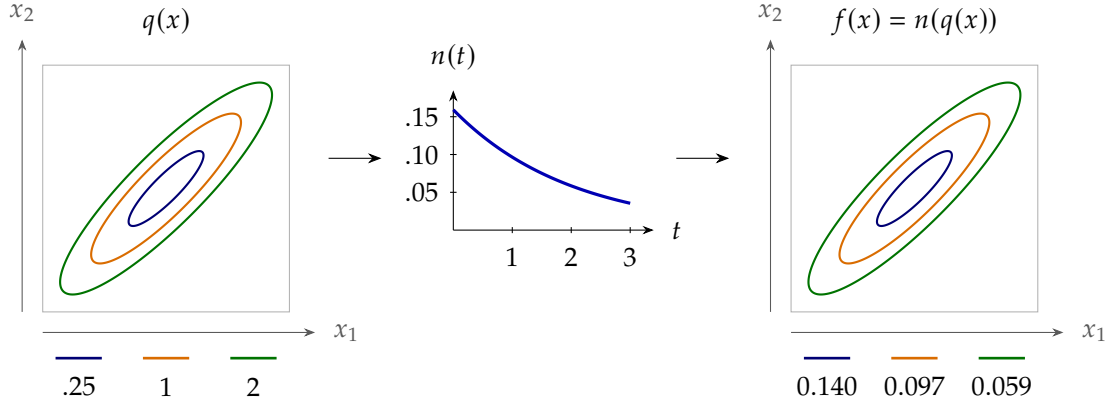


Figure 48: The same ellipsoids have quadratic labels on the left and density labels on the right. This two-dimensional illustration uses $\det \Sigma = 1$, so $n(t) = e^{-t/2}/(2\pi)$. Equal colors identify the same contour; density values are rounded.

9.2 Fitting full covariance matrices

Given column vectors $X_1, \dots, X_n \in \mathbb{R}^d$ and class labels $y_1, \dots, y_n$, fit a Gaussian within each class. Write $I_C = \{i : y_i = C\}$ and $n_C = |I_C| > 0$. The maximum-likelihood prior and mean remain

$$\hat{\pi}_C = \frac{n_C}{n}, \quad \hat{\mu}_C = \frac{1}{n_C} \sum_{i \in I_C} X_i.$$

The new parameter is a full covariance matrix, rather than one scalar variance. For quadratic discriminant analysis (QDA),

$$\hat{\Sigma}_C = \frac{1}{n_C} \sum_{i \in I_C} (X_i - \hat{\mu}_C)(X_i - \hat{\mu}_C)^\top.$$

Each residual outer product is a $d \times d$ matrix. Its (j, k) entry multiplies residual coordinates j and k ; averaging therefore records variances on the diagonal and covariances off it. The denominator is n_C , as required for maximum likelihood.

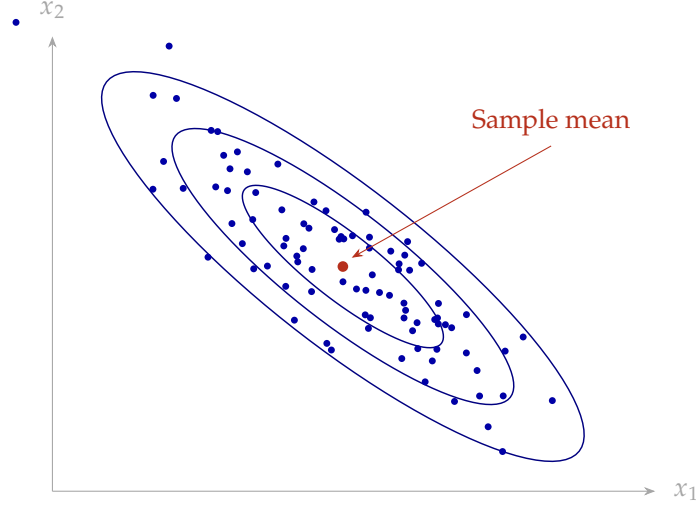


Figure 49: A schematic redraw of fitting a Gaussian to a slanted cloud. The fitted center follows the sample mean, and the contour axes follow the sample covariance. Points illustrate the geometry; they do not reproduce the original measurements.

Proposition 9.1 [SAMPLE COVARIANCE CAN BE SINGULAR]. $\hat{\Sigma}_C$ is positive semidefinite. It is positive definite exactly when the centered observations span $\mathbb{R}^d$.

Proof. Test the matrix in any direction u :

$$u^\top \hat{\Sigma}_C u = \frac{1}{n_C} \sum_{i \in I_C} (u^\top (X_i - \hat{\mu}_C))^2 \geq 0.$$

This is zero precisely when u is perpendicular to every residual. A nonzero such u exists exactly when their span is smaller than $\mathbb{R}^d$. Because the residuals sum to zero, their rank is at most $n_C - 1$; in particular, $n_C \leq d$ forces singularity. $\square$

A zero eigenvalue is a direction with no observed spread. The ordinary Gaussian density requires an inverse and a positive determinant, so it cannot use a singular estimate. Removing zero-variance directions is one possible response, with the Gaussian then defined in the retained space.

Linear discriminant analysis (LDA) instead fits one shared POOLED WITHIN-CLASS COVARIANCE:

$$\hat{\Sigma} = \frac{1}{n} \sum_C \sum_{i \in I_C} (X_i - \hat{\mu}_C)(X_i - \hat{\mu}_C)^\top = \sum_C \frac{n_C}{n} \hat{\Sigma}_C.$$

The residual for every observation uses its own class mean. Subtracting the global mean would add variation between class centers, which is not the shared within-class spread. The pooled matrix must also be positive definite to use the ordinary LDA density.

9.3 Quadratic discriminants and posterior probabilities

Assume $\pi_C > 0$ and $\Sigma_C > 0$. Under zero-one loss, choose the class maximizing $\pi_C f_C(x)$. Taking its logarithm and removing the common Gaussian constant gives the QUADRATIC DISCRIMINANT

$$Q_C(x) = -\frac{1}{2}(x - \mu_C)^\top \Sigma_C^{-1}(x - \mu_C) - \frac{1}{2} \ln \det \Sigma_C + \ln \pi_C.$$

The first term measures distance using the class's precision. The determinant term accounts for the density's normalization; the prior term accounts for class frequency. For any number of classes, select a class with largest $Q_C(x)$.

For two classes C, D , write $\Delta(x) = Q_C(x) - Q_D(x)$. Expansion gives

$$\begin{aligned}\Delta(x) &= x^\top Mx + b^\top x + \alpha, \\ M &= \frac{1}{2}(\Sigma_D^{-1} - \Sigma_C^{-1}), \\ b &= \Sigma_C^{-1}\mu_C - \Sigma_D^{-1}\mu_D, \\ \alpha &= -\frac{1}{2}\mu_C^\top \Sigma_C^{-1}\mu_C + \frac{1}{2}\mu_D^\top \Sigma_D^{-1}\mu_D - \frac{1}{2} \ln \frac{\det \Sigma_C}{\det \Sigma_D} + \ln \frac{\pi_C}{\pi_D}.\end{aligned}$$

The two cross terms in each symmetric quadratic form combine into $\mu_C^\top \Sigma_C^{-1}x$, which explains b . Although both precision matrices are positive definite, their difference can be indefinite. The zero set $\Delta(x) = 0$ is a quadric, possibly degenerate. It is a decision boundary where the preferred class changes across that set. Hyperbolic boundaries are possible with unequal anisotropic covariance matrices.

Bayes' theorem converts the score difference into a posterior:

$$\mathbb{P}(Y = C \mid X = x) = \frac{\pi_C f_C(x)}{\pi_C f_C(x) + \pi_D f_D(x)} = \frac{1}{1 + e^{-\Delta(x)}} = s(\Delta(x)).$$

Here $s(t) = 1/(1 + e^{-t})$ is the logistic function. The contour $\Delta = 0$ is also the posterior contour $1/2$. Comparing scores suffices for classification; applying s provides model-based probabilities. They are accurate only to the extent that the fitted model describes the data.

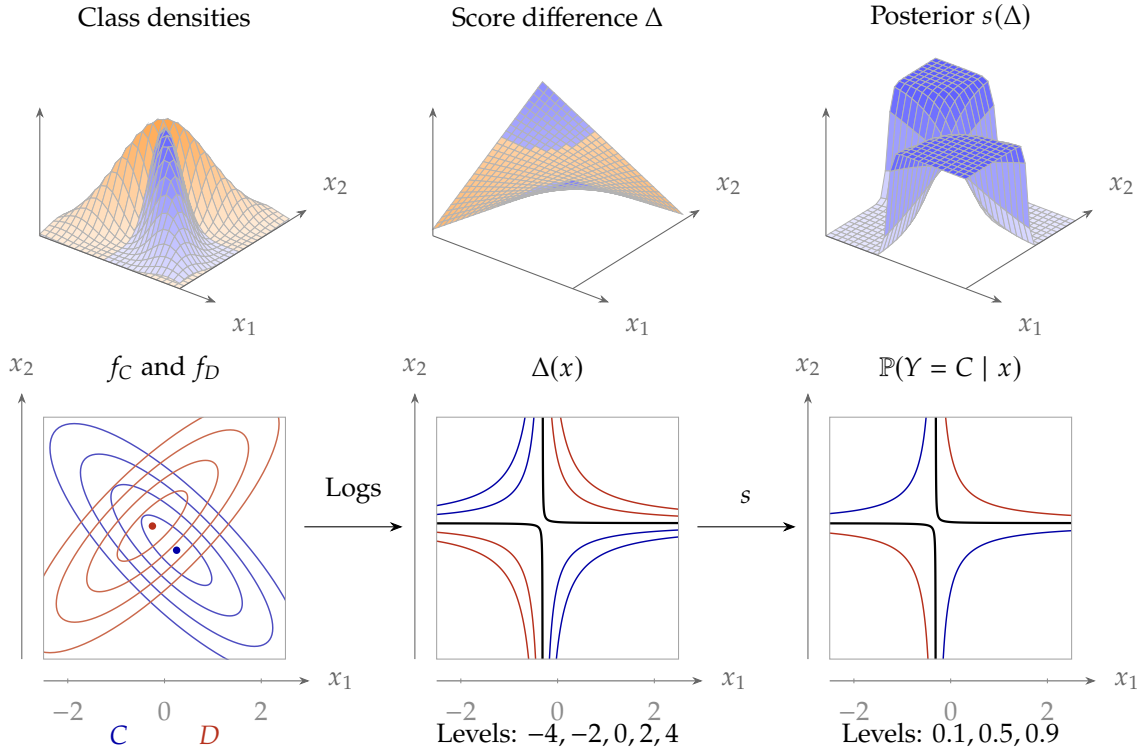


Figure 50: Schematic anisotropic QDA: two tilted Gaussian densities, an indefinite score difference, and its logistic posterior. Bottom panels show the corresponding contours. Black marks $\Delta = 0$, equivalently posterior $1/2$; colored posterior contours show 0.1 and 0.9. The hyperbola separates regions of opposite sign. Surface heights are shifted and scaled for display.

9.4 Multiclass QDA regions

With several classes, each point belongs to the class with greatest quadratic score. Pairwise equality curves contribute to the boundary only where the tied scores are also maximal. The result is an **ANISOTROPIC VORONOI DIAGRAM**: every class uses its own ellipsoidal distance, together with determinant and prior offsets. A class region can be disconnected. No single common Euclidean nearest-mean rule describes the general partition.

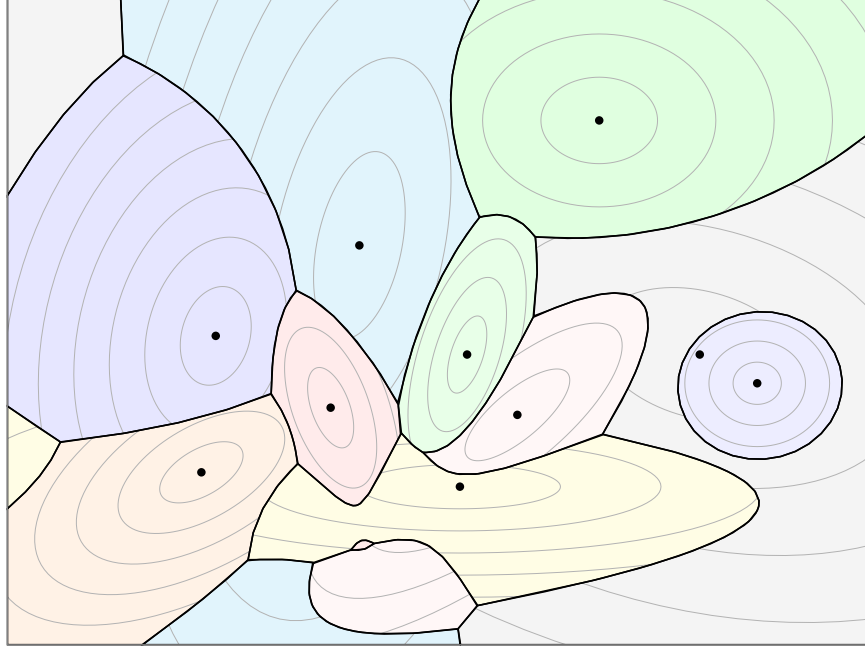


Figure 51: A schematic multiclass QDA partition. Dots mark means, ellipses show class-density contours, and black curves separate winning classes. The same class can win in separated parts of the plane. Shapes illustrate anisotropic geometry rather than measured data.

9.5 Shared covariance gives linear discriminants

LDA uses the same $\Sigma > 0$ for every class. Its precision can stretch or rotate distances, but the common quadratic term cancels when comparing classes.

Proposition 9.2 [ANISOTROPIC LDA CANCELLATION]. *For two classes with shared covariance,*

$$Q_C(x) - Q_D(x) = w^\top x + \alpha,$$

where

$$w = \Sigma^{-1}(\mu_C - \mu_D),$$

$$\alpha = -\frac{1}{2}(\mu_C^\top \Sigma^{-1} \mu_C - \mu_D^\top \Sigma^{-1} \mu_D) + \ln \frac{\pi_C}{\pi_D}.$$

Proof. Expand each discriminant:

$$Q_C(x) = -\frac{1}{2}x^\top \Sigma^{-1}x + \mu_C^\top \Sigma^{-1}x - \frac{1}{2}\mu_C^\top \Sigma^{-1}\mu_C - \frac{1}{2}\ln \det \Sigma + \ln \pi_C.$$

The terms $-x^\top \Sigma^{-1}x/2$ and $-\ln \det \Sigma/2$ are common to every class and disappear in the difference. The remaining linear and constant terms give w and α . $\square$

For any number of classes, maximize the LINEAR DISCRIMINANT

$$L_C(x) = \mu_C^\top \Sigma^{-1} x - \frac{1}{2} \mu_C^\top \Sigma^{-1} \mu_C + \ln \pi_C.$$

It is affine, despite the conventional name “linear.” For two distinct means, $w \neq 0$, so $w^\top x + \alpha = 0$ is a hyperplane and the posterior is $s(w^\top x + \alpha)$. If $w = 0$, the comparison is constant and does not define a separating hyperplane.

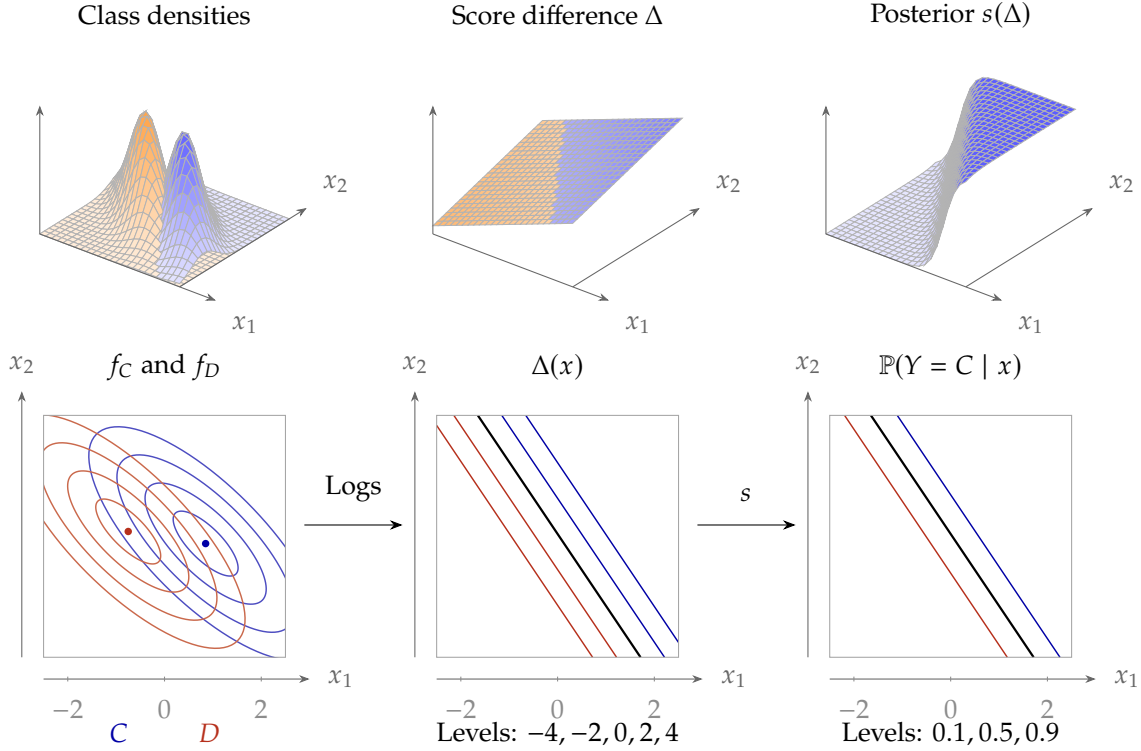


Figure 52: Schematic anisotropic LDA. The two Gaussians have the same ellipsoidal shape. Their score difference is a plane, with parallel straight contours; the posterior is a sigmoid varying only along the normal direction. Black marks posterior 1/2, and the colored posterior contours mark 0.1 and 0.9. Surface heights are shifted and scaled for display.

There is no need to form an explicit inverse for repeated predictions. Solve $\Sigma a_C = \mu_C$ once for each class, then store $b_C = -\mu_C^\top a_C / 2 + \ln \pi_C$. A test point requires only $L_C(x) = a_C^\top x + b_C$. This precomputation separates fitting from fast prediction.

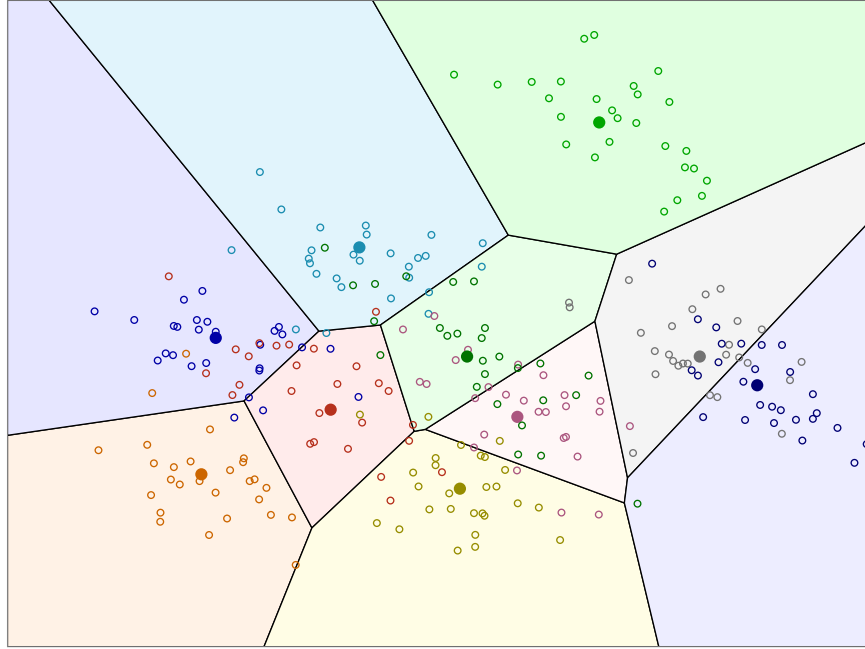


Figure 53: Schematic multiclass LDA on overlapping, imperfect clusters. Black boundaries are straight pieces of pairwise hyperplanes; dots show class means and small circles show observations. Overlap permits classification errors even with simple, useful decision regions. No error rate is inferred from the drawing.

9.6 Projection and model flexibility

For a two-class LDA rule with $w \neq 0$, put $u = w/\|w\|$. Its score depends on the point only through the scalar projection $u^\top x$:

$$w^\top x + \alpha = \|w\| \left(u^\top x + \frac{\alpha}{\|w\|} \right).$$

Projecting onto the normal direction and comparing with the threshold $-\alpha/\|w\|$ therefore gives the same prediction. Moving parallel to the hyperplane does not change the score or posterior.

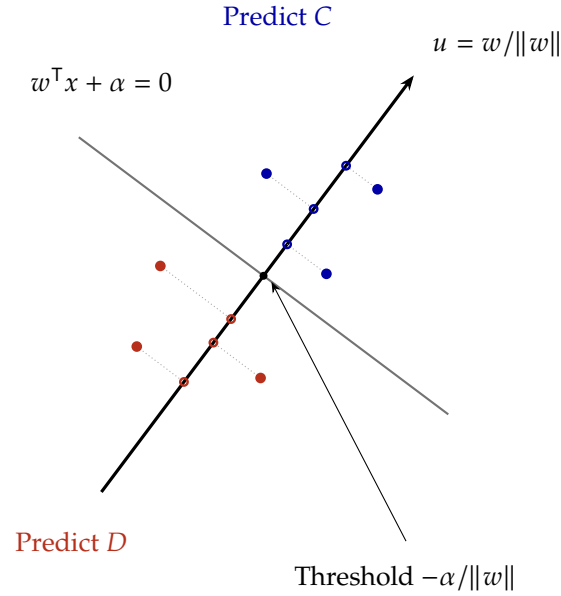


Figure 54: An LDA decision depends on the coordinate along its normal. Dotted perpendicular segments project observations onto that line. The threshold is where the normal line meets the decision boundary; points on the same side of the threshold share a prediction.

In d dimensions, an affine two-class score has $d + 1$ coefficients: d entries of w and one offset. A general quadratic score has

$$\frac{d(d+1)}{2} + d + 1 = \frac{d(d+3)}{2} + 1$$

coefficients, since its quadratic matrix is symmetric. These count decision-function coefficients, before accounting for the fact that positive rescaling preserves its sign. They are not counts of all parameters in two separately fitted Gaussian densities. QDA's flexibility grows quadratically with dimension, so it can fit curvature but is more susceptible to overfitting.

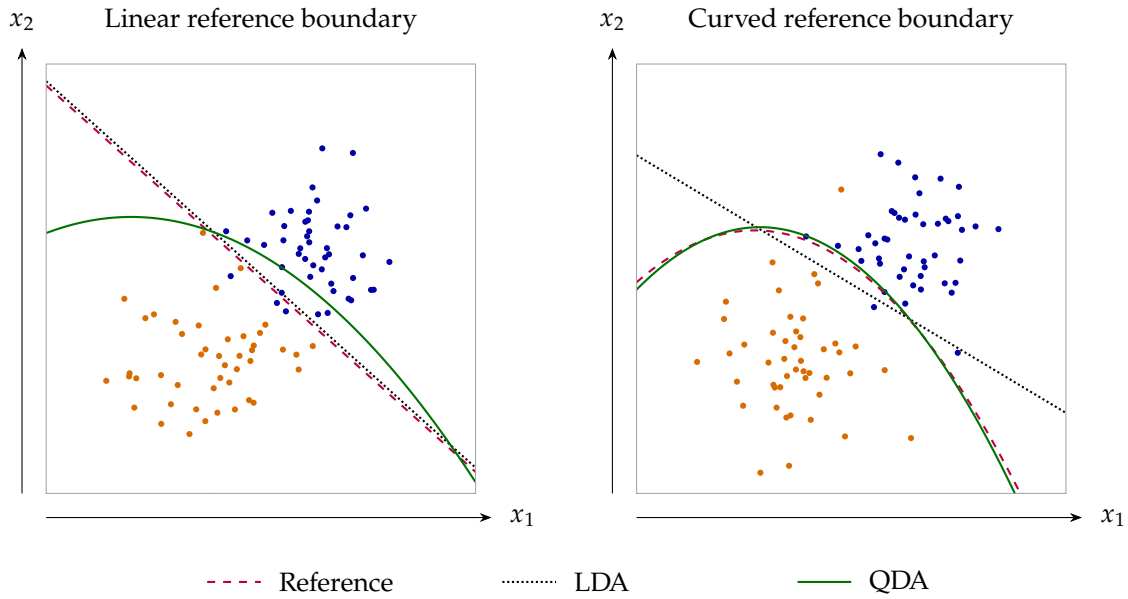


Figure 55: Schematic LDA–QDA comparison. Purple dashed curves are reference Bayes boundaries, black dotted lines represent LDA, and green curves represent QDA. A linear reference can favor the stability of LDA; a curved reference can favor QDA’s flexibility. The drawings do not report fitted parameters or error rates.

Adding nonlinear features changes this statement in the original input space: LDA can then give nonlinear boundaries, and QDA can give nonquadratic ones. A fitted Gaussian classifier need not be the true Bayes classifier, because finite data introduce estimation error and real distributions need not be Gaussian. Simple boundaries can still be useful under imperfect distributional assumptions. A reported STATLOG comparison placed LDA or QDA among the top three classifiers on 10 of 22 datasets; this is a historical comparison, not a guarantee for a new dataset.

9.7 Priors, losses, and posterior contours

Changing a prior adds a constant to its discriminant. In the two-class case it changes the offset in Δ , leaving the other coefficients fixed when the means and covariances are held fixed. LDA hyperplanes move without rotating; QDA changes the chosen level of its quadratic score.

For a posterior threshold $p \in (0, 1)$,

$$\mathbb{P}(Y = C \mid X = x) > p \iff \Delta(x) > \ln \frac{p}{1-p}.$$

Thus posterior contours 0.1, 0.5, and 0.9 correspond to score levels $-\ln 9$, 0, and $\ln 9$. This holds for both QDA and LDA.

Let C be the positive class. With zero loss for a correct prediction, false-positive cost c_{FP} and false-negative cost c_{FN} give the rule

$$\text{predict } C \text{ when } \mathbb{P}(Y = C \mid X = x) > \frac{c_{FP}}{c_{FP} + c_{FN}}.$$

Taking $c_{FP} = p$ and $c_{FN} = 1 - p$ therefore implements the threshold p . Equivalently compare $(1 - p)\pi_C f_C(x)$ with $p\pi_D f_D(x)$. These weights can be absorbed into normalized effective priors

$$\tilde{\pi}_C = \frac{(1 - p)\pi_C}{(1 - p)\pi_C + p\pi_D}, \quad \tilde{\pi}_D = \frac{p\pi_D}{(1 - p)\pi_C + p\pi_D}.$$

When the baseline priors are equal, these become $1 - p$ and p . For unequal baseline priors, the original prior factors must remain. Changing the prediction threshold or loss does not itself refit the Gaussian densities.

9.8 Design matrices and empirical covariance

A `DESIGN MATRIX` stores observations as rows: $X \in \mathbb{R}^{n \times d}$ has row i equal to $X_i^\top$. Here X denotes a data matrix, rather than a random vector. Let $\mu = n^{-1} \sum_i X_i$ be the global sample mean. `CENTERING` subtracts $\mu^\top$ from every row, giving $\dot{X}$ with rows $(X_i - \mu)^\top$. Each column of $\dot{X}$ has mean zero.

If R is uniformly distributed over the n observed points, its covariance is

$$S = \text{Var}(R) = \frac{1}{n} \dot{X}^\top \dot{X}.$$

Matrix multiplication sums the residual outer products: $\dot{X}^\top \dot{X} = \sum_i (X_i - \mu)(X_i - \mu)^\top$. Hence S is $d \times d$, not $n \times n$. This is an empirical covariance with denominator n . For the class-only matrix X_C , centered at $\hat{\mu}_C$, the same formula gives $\hat{S}_C = n_C^{-1} \dot{X}_C^\top \dot{X}_C$. Class centering for QDA and global centering of the full dataset serve different purposes.

9.9 Decorrelation, sphering, and whitening

Let $S = V\Lambda V^\top$, with V orthogonal and Λ diagonal. `DECORRELATING` the centered data means expressing its rows in the eigenvector coordinates:

$$Z = \dot{X}V.$$

For a column observation, the same change is $z_i = V^\top(X_i - \mu)$; transposing gives the row formula. The empirical covariance becomes diagonal:

$$\begin{aligned} \frac{1}{n} Z^\top Z &= \frac{1}{n} V^\top \dot{X}^\top \dot{X} V \\ &= V^\top S V = V^\top V \Lambda V^\top V = \Lambda. \end{aligned}$$

The axes are now uncorrelated, but their variances need not be equal. Decorrelation alone does not make a general distribution independent.

Assume now that $S \succ 0$. SPHERING applies its symmetric inverse square root to centered rows:

$$W = \dot{X}S^{-1/2} = \dot{X}V\Lambda^{-1/2}V^T = Z\Lambda^{-1/2}V^T.$$

The eigenvector coordinates are scaled by the reciprocal standard deviations, then rotated back. WHITENING means centering followed by sphering. Its empirical covariance is the identity:

$$\frac{1}{n}W^TW = S^{-1/2} \left(\frac{1}{n}\dot{X}^T\dot{X} \right) S^{-1/2} = S^{-1/2}SS^{-1/2} = I.$$

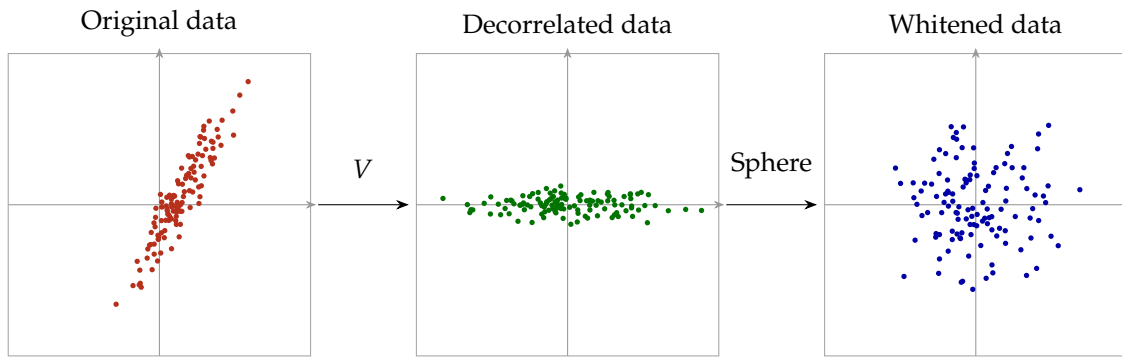


Figure 56: The same schematic observations before transformation, after rotation into covariance eigenvectors, and after whitening. Rotation removes off-diagonal covariance; sphering also equalizes variances. The point cloud illustrates geometry, not measured data.

Algorithm 9.1: Symmetric empirical whitening

Input: An $n \times d$ design matrix X whose empirical covariance is positive definite.

- 1 $\mu \leftarrow n^{-1} \sum_{i=1}^n X_i$
 - 2 Form $\dot{X}$ by subtracting μ^T from each row of X
 - 3 $S \leftarrow n^{-1} \dot{X}^T \dot{X}$
 - 4 Compute $S = V \text{diag}(\rho_1, \dots, \rho_d) V^T$ with V orthogonal
 - 5 $B \leftarrow V \text{diag}(\rho_1^{-1/2}, \dots, \rho_d^{-1/2}) V^T$
 - 6 $W \leftarrow \dot{X}B$
 - 7 **return** W
-

Note. If S has a zero eigenvalue, $S^{-1/2}$ is not defined in the ordinary sense. Zero-variance directions must be removed before using this formula; identity covariance then refers to the retained space.

Using the true mean and covariance of a Gaussian gives exact population statements: eigenvector coordinates have law $\mathcal{N}(0, \Lambda)$, and whitened coordinates have law $\mathcal{N}(0, I)$. Using estimates from the same finite dataset instead gives the exact empirical covariance identities above. It does not guarantee an exactly Gaussian population law or independence for arbitrary data.

Whitening can be useful before SVMs or neural networks because features measured in different units alter the geometry of a norm penalty. It puts empirical principal directions on a common variance scale. Discriminant analysis already incorporates this adjustment through its precision-weighted distances. Covariance eigenvectors also provide ingredients for principal components analysis.

Revision. Covariance eigenvalues are variances; their square roots are widths. QDA fits a covariance per class and compares quadratic scores. LDA pools within-class covariance, so its shared quadratic term cancels. Posterior thresholds are score levels. Centering and rotation diagonalize empirical covariance; sphering changes it to I .

Common mistakes: using a singular covariance in an ordinary Gaussian density; pooling residuals around the global mean; counting decision coefficients as all Gaussian parameters; dropping existing priors when changing a threshold; confusing $\dot{X}^T \dot{X}$ with $\dot{X} \dot{X}^T$; or equating decorrelation with independence for arbitrary data.

Check yourself: Why can a sample covariance be singular? Which terms cancel in LDA? Why can QDA produce a hyperbola? What score level corresponds to posterior 0.9? Why does $Z = \dot{X}V$ have covariance Λ ? What extra step makes whitened covariance the identity?

Source. Jonathan Shewchuk, University of California, Berkeley, CS 189/289A, *Introduction to Machine Learning*, Spring 2022, [Lecture 9: Anisotropic Gaussians, Maximum Likelihood Estimation, QDA, and LDA](#), PDF pp. 1–7 (printed pp. 46–52). Figures are original vector reconstructions of the supplied covariance, fitting, QDA, LDA, projection, model-comparison, and whitening geometry; the repeated quadratic-to-density contours reuse the original redraw from Lecture 8. Illustrative points and display scales are schematic. The sample-covariance positivity and rank argument, discriminant expansions, prior-threshold algebra, and covariance-transform proofs complete or qualify the supplied claims. Covariance MLE formulas are stated without a general optimization derivation. Coefficient counts, zero-variance cases, and empirical versus population whitening are distinguished for correctness. The STATLOG comparison is retained as reported in the supplied material.

LECTURE 10: LINEAR AND LOGISTIC REGRESSION

10.1 Regression has three interchangeable parts

REGRESSION predicts a numerical value from an input x . Classification predicts a class label. A probability prediction is numerical too: QDA and LDA both perform regression when they estimate posterior probabilities, even if the final output is a class.

To define a regression method, choose a family of hypotheses $h(x; p)$, a loss for one prediction, and a cost that combines the training losses. The parameters p are what training changes.

Definition: A REGRESSION FUNCTION specifies the shape of the predictor. Three common choices are

$$\begin{aligned} \text{linear:} \quad & h(x; w, \alpha) = w \cdot x + \alpha, \\ \text{polynomial:} \quad & \text{linear in parameters after adding polynomial features,} \\ \text{logistic:} \quad & h(x; w, \alpha) = s(w \cdot x + \alpha), \quad s(t) = \frac{1}{1 + e^{-t}}. \end{aligned}$$

The logistic choice restricts predictions to $(0, 1)$, which is useful for probabilities. A polynomial predictor can curve in the original input space while remaining linear in its feature weights.

Definition: A LOSS FUNCTION $L(z, y)$ measures the error of one prediction $z = h(x; p)$ against its target y :

$$\begin{aligned} \text{squared error} \quad & L(z, y) = (z - y)^2, & z, y \in \mathbb{R}, \\ \text{absolute error} \quad & L(z, y) = |z - y|, & z, y \in \mathbb{R}, \\ \text{logistic loss} \quad & L(z, y) = -y \ln z - (1 - y) \ln(1 - z), & y \in [0, 1], z \in (0, 1). \end{aligned}$$

Logistic loss is also called CROSS-ENTROPY.

Definition: A COST FUNCTION, or objective, combines losses over the training set. Write $L_i = L(h(X_i; p), y_i)$. Common choices are

$$J_{\text{mean}} = \frac{1}{n} \sum_{i=1}^n L_i, \quad J_{\text{max}} = \max_{1 \leq i \leq n} L_i, \quad J_{\text{weighted}} = \sum_{i=1}^n \omega_i L_i, \quad \omega_i \geq 0.$$

Weights give some observations more influence. A REGULARIZED cost adds a penalty, such as $\lambda \|w\|^2$ or $\lambda \|w\|_1$, with $\lambda \geq 0$ and $\|w\|_1 = \sum_j |w_j|$. Risk is a cost expressed as expected loss.

Dropping $1/n$ does not change the minimizers of an unpenalized mean loss. It does rescale the gradient, so the same numerical step size gives a different update. With a penalty present, keep its strength consistent with whether losses are summed or averaged.

Method	Predictor	Loss / aggregation	Optimization
Least squares	Linear	Squared / sum	Quadratic, calculus
Weighted least squares	Linear	Squared / weighted	Quadratic, calculus
Ridge	Linear	Squared / sum + $\lambda \ w\ ^2$	Quadratic, calculus
Lasso	Linear	Squared / sum + $\lambda \ w\ _1$	Quadratic program
Logistic regression	Logistic	Cross-entropy / sum	Convex, gradient descent
Least absolute deviations	Linear	Absolute / sum	Linear program
Chebyshev criterion	Linear	Absolute / maximum	Linear program

These choices can be exchanged, but changing one can change the optimization problem and the algorithm needed to solve it.

10.2 Least squares and the design matrix

LEAST-SQUARES LINEAR REGRESSION chooses a linear predictor and minimizes the sum of squared errors:

$$\min_{w, \alpha} \sum_{i=1}^n (w \cdot X_i + \alpha - y_i)^2.$$

For two input features, the prediction surface is a plane. Each residual is the vertical difference between an observed target and the prediction at the same input, rather than the shortest distance from the point to the plane.

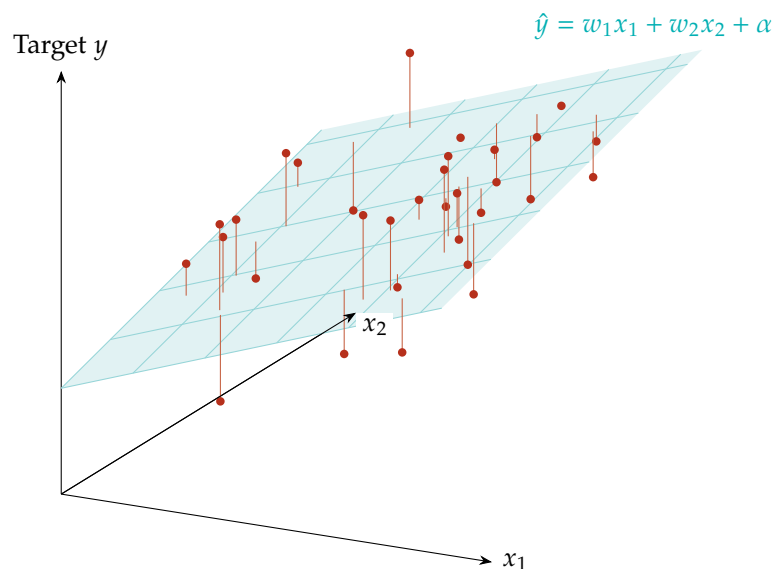


Figure 57: A schematic redraw of fitting a plane to observations with two input features. Red segments measure errors in the target coordinate. Least squares minimizes the sum of their squared lengths.

Initially, the DESIGN MATRIX $X \in \mathbb{R}^{n \times d}$ has one observation per row and one feature per column:

$$X = \begin{bmatrix} X_{11} & \cdots & X_{1d} \\ X_{21} & \cdots & X_{2d} \\ \vdots & & \vdots \\ X_{n1} & \cdots & X_{nd} \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

Row i is $X_i^\top$, while column j contains feature j across all observations. Usually $n > d$, but that is not required.

Absorb the intercept into a fictitious feature equal to 1. From here on, X and X_i include that extra coordinate, and w includes α as its last entry. With $p = d + 1$,

$$X \in \mathbb{R}^{n \times p}, \quad w \in \mathbb{R}^p, \quad h(x) = \begin{bmatrix} x^\top & 1 \end{bmatrix} w.$$

The last column of X is all ones. All training predictions form the vector Xw . The RESIDUAL SUM OF SQUARES is therefore

$$\text{RSS}(w) = \|Xw - y\|^2.$$

10.3 The normal equations

Expand the squared norm before differentiating:

$$\begin{aligned} \text{RSS}(w) &= (Xw - y)^\top (Xw - y) \\ &= w^\top X^\top Xw - 2y^\top Xw + y^\top y, \\ \nabla_w \text{RSS}(w) &= 2X^\top Xw - 2X^\top y. \end{aligned}$$

Setting the gradient to zero gives the NORMAL EQUATIONS:

$$\boxed{X^\top Xw = X^\top y.}$$

The matrix on the left is $p \times p$ and the right-hand side is a p -vector. The data X, y are known; the unknown is w .

Proposition 10.1 [WHEN THE SOLUTION IS UNIQUE]. $X^T X$ is positive semidefinite. It is positive definite, and the least-squares minimizer is unique, exactly when X has linearly independent columns.

Proof. For any $v \in \mathbb{R}^p$,

$$v^T X^T X v = \|Xv\|^2 \geq 0.$$

This is positive for every nonzero v exactly when $Xv = 0$ has only the zero solution. At a solution w_* of the normal equations, the cross term vanishes, giving

$$\text{RSS}(w_* + v) = \text{RSS}(w_*) + \|Xv\|^2.$$

Thus the stationary point is a global minimum, and it is unique exactly when no nonzero v leaves the predictions unchanged. $\square$

If $X^T X$ is singular, some nonzero coefficient vector v satisfies $Xv = 0$. With the intercept feature included, this means all original observations lie on a common affine hyperplane: $v_1 X_{i1} + \dots + v_d X_{id} + v_p = 0$ for every i . The fitting problem is underconstrained: w_* and $w_* + tv$ have identical predictions for every t . Having many observations alone does not guarantee independent columns.

Computation. When X has full column rank, write

$$w_* = (X^T X)^{-1} X^T y$$

to describe the solution mathematically. In an implementation, solve the linear system; do not explicitly form the inverse. The inverse formula does not apply when $X^T X$ is singular.

10.4 Pseudoinverse, hat matrix, and projection

Under the full-column-rank assumption, the PSEUDOINVERSE is

$$X^+ = (X^T X)^{-1} X^T \in \mathbb{R}^{p \times n}.$$

It is a left inverse because

$$X^+ X = (X^T X)^{-1} X^T X = I_p.$$

Although a nonsquare X cannot have an ordinary inverse, every matrix has a pseudoinverse. The displayed formula is the full-column-rank case. Its role here is to show that, for fixed X , the fitted weights are a linear transformation of the targets: $w_* = X^+ y$.

The fitted target vector is

$$\hat{y} = X w_* = X X^+ y = H y, \quad H = X X^+ = X (X^T X)^{-1} X^T \in \mathbb{R}^{n \times n}.$$

The HAT MATRIX H puts a hat on y .

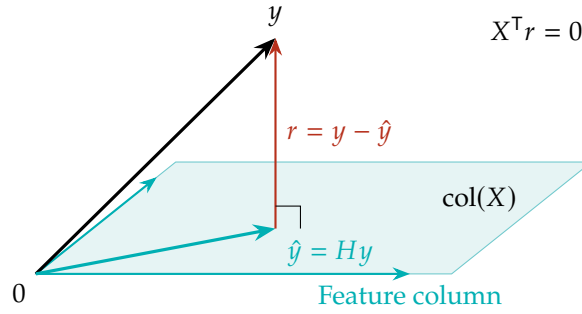


Figure 58: Least squares projects the target vector onto the column space of X . This schematic lives in target space $\mathbb{R}^n$. The residual $r = y - \hat{y}$ is perpendicular to every feature column.

Proposition 10.2 [LEAST SQUARES IS ORTHOGONAL PROJECTION]. *$\hat{y}$ is the orthogonal projection of y onto $\text{col}(X)$. In the full-column-rank case, $H^T = H$ and $H^2 = H$.*

Proof. The vector $\hat{y} = Xw_*$ lies in the column space. The normal equations say

$$X^T(y - \hat{y}) = 0,$$

so the residual is orthogonal to every column and hence to their span. For any other attainable prediction $\tilde{y} = X\tilde{w}$,

$$\|y - \tilde{y}\|^2 = \|y - \hat{y}\|^2 + \|\hat{y} - \tilde{y}\|^2.$$

This proves that $\hat{y}$ is the unique closest attainable prediction. Also, H is symmetric because $(X^T X)^{-1}$ is symmetric, and

$$H^2 = X(X^+ X)X^+ = XX^+ = H.$$

Projecting a prediction already in the column space does nothing. □

When $n > p$ and X has full column rank, H has rank $p < n$ and cannot be I_n . A perfect fit is still possible for particular targets: it occurs exactly when $y \in \text{col}(X)$. Even with dependent columns, the closest prediction vector remains unique, although its coefficient representation may not be.

Least squares is easy to compute through a linear solve. Full column rank gives a unique solution, and well-conditioned data make it numerically stable. Squaring errors makes the fit sensitive to outliers. Singular data invalidate the inverse formula; nearly dependent columns can also make coefficients numerically sensitive. Historically, Gauss used least squares in 1801 to predict the trajectory of Ceres.

10.5 Logistic regression fits probabilities directly

LOGISTIC REGRESSION combines a logistic predictor with cross-entropy loss and a sum or mean cost. It is usually used for classification, with $y_i \in \{0, 1\}$, but targets can also be probabilities in $[0, 1]$. Retain the intercept feature and define

$$s_i = s(X_i \cdot w), \quad J(w) = - \sum_{i=1}^n [y_i \ln s_i + (1 - y_i) \ln(1 - s_i)].$$

The sigmoid guarantees $0 < s_i < 1$ for finite weights. With the usual $1/2$ threshold, the class boundary is $X_i \cdot w = 0$ because $s(0) = 1/2$.

QDA and LDA are generative: they fit class-conditional densities and then obtain posterior probabilities. Logistic regression is discriminative: it fits the posterior-shaped function directly. LDA's logistic posterior motivates this shape, but logistic regression does not fit Gaussians as an intermediate step. The method dates to David Cox in 1958.

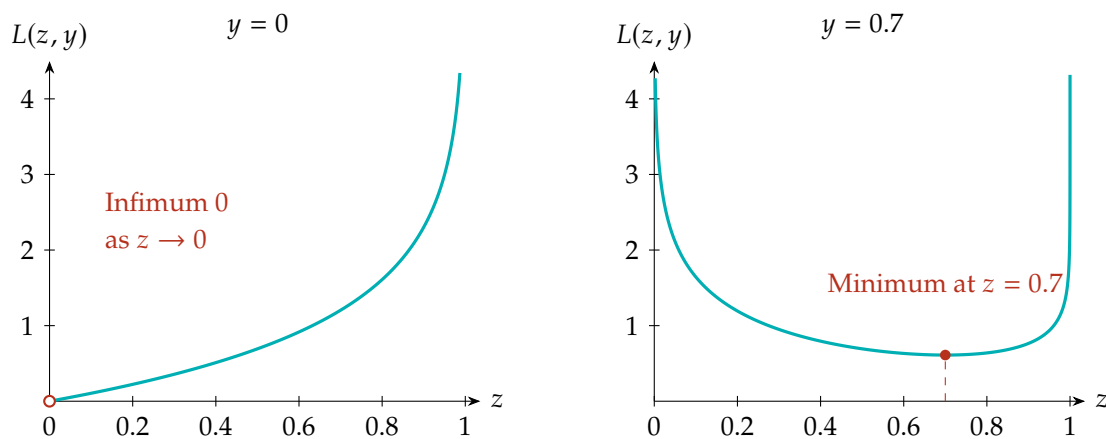


Figure 59: Cross-entropy loss as a function of the predicted probability. For $y = 0$, its infimum is approached as $z \rightarrow 0$. For the soft target $y = 0.7$, the minimum is attained at $z = 0.7$.

For $z \in (0, 1)$, differentiation explains these shapes:

$$\frac{\partial L}{\partial z} = -\frac{y}{z} + \frac{1-y}{1-z} = \frac{z-y}{z(1-z)}, \quad \frac{\partial^2 L}{\partial z^2} = \frac{y}{z^2} + \frac{1-y}{(1-z)^2} > 0.$$

For an interior target $0 < y < 1$, the derivative vanishes at $z = y$. For $y = 0$ or 1 , the best probability is a limiting endpoint, outside the sigmoid's finite-weight range.

10.6 The sigmoid derivative and the cost gradient

The derivative of the sigmoid has a useful factored form:

$$\begin{aligned} s'(t) &= \frac{e^{-t}}{(1 + e^{-t})^2} \\ &= \frac{1}{1 + e^{-t}} \left(1 - \frac{1}{1 + e^{-t}} \right) = s(t)(1 - s(t)). \end{aligned}$$

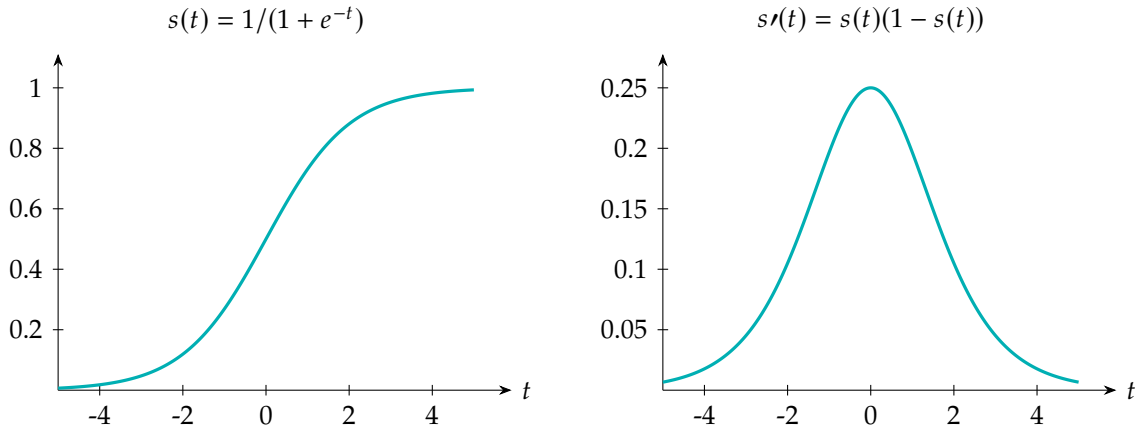


Figure 60: The sigmoid and its derivative. The slope is largest at $t = 0$, where $s = 1/2$ and $s' = 1/4$; it tends to zero at both extremes.

Because X_i is a column vector, $\nabla_w s_i = s_i(1 - s_i)X_i$. The chain rule gives

$$\begin{aligned} \nabla_w J &= - \sum_{i=1}^n \left(\frac{y_i}{s_i} - \frac{1 - y_i}{1 - s_i} \right) \nabla_w s_i \\ &= - \sum_{i=1}^n \left(\frac{y_i}{s_i} - \frac{1 - y_i}{1 - s_i} \right) s_i(1 - s_i)X_i \\ &= - \sum_{i=1}^n (y_i - s_i)X_i = X^\top (s(Xw) - y), \end{aligned}$$

where $s(Xw) = [s_1, \dots, s_n]^\top$ applies s to each coordinate. The sigmoid factors cancel with the denominators in the loss derivative, leaving a simple prediction-error expression.

Proposition 10.3 [THE LOGISTIC COST IS CONVEX IN THE WEIGHTS]. *For targets in $[0, 1]$, $J(w)$ is convex.*

Proof. Along any line $w + tv$, differentiating the gradient gives

$$\frac{d^2}{dt^2}J(w + tv) = \sum_{i=1}^n s_i(t)(1 - s_i(t))(X_i \cdot v)^2 \geq 0,$$

where $s_i(t) = s(X_i \cdot (w + tv))$. Every restriction to a line is convex, so the full cost is convex. $\square$

Convexity of L in its probability argument alone would not justify convexity after composing it with a nonlinear sigmoid. The calculation above establishes convexity in w directly. Convexity also does not guarantee that the minimum is attained at finite weights.

10.7 Gradient descent and stochastic updates

Subtracting the gradient gives the full-data update

$$w \leftarrow w + \epsilon X^T(y - s(Xw)), \quad \epsilon > 0.$$

For the mean loss, divide the gradient by n , or equivalently adjust the step size. A single-observation stochastic update is

$$w \leftarrow w + \epsilon(y_i - s(X_i \cdot w))X_i.$$

Shuffle observations and process them one at a time. For a very large training set, stochastic updates may reach a useful fit before one complete pass. Initializing $w = 0$ works well in practice and begins with every predicted probability equal to $1/2$.

Algorithm 10.1: Logistic regression with shuffled stochastic gradient descent

Input: Augmented observations $X_1, \dots, X_n$; targets $y_i \in [0, 1]$; step size $\epsilon > 0$; a stopping criterion

```

1  $w \leftarrow 0$ 
2 while the stopping criterion is not met do
3   Choose a random permutation  $\pi$  of  $1, \dots, n$ 
4   for  $k = 1, \dots, n$  do
5      $i \leftarrow \pi(k)$ 
6      $s_i \leftarrow 1/(1 + e^{-X_i \cdot w})$ 
7      $w \leftarrow w + \epsilon(y_i - s_i)X_i$ 
```

Output: The fitted weight vector w

The update resembles the perceptron rule: it changes w in the direction of the observation. Here the multiplier is the continuous probability error $y_i - s_i$. Even a correctly classified point can still contribute an update because its predicted probability is not yet its target.

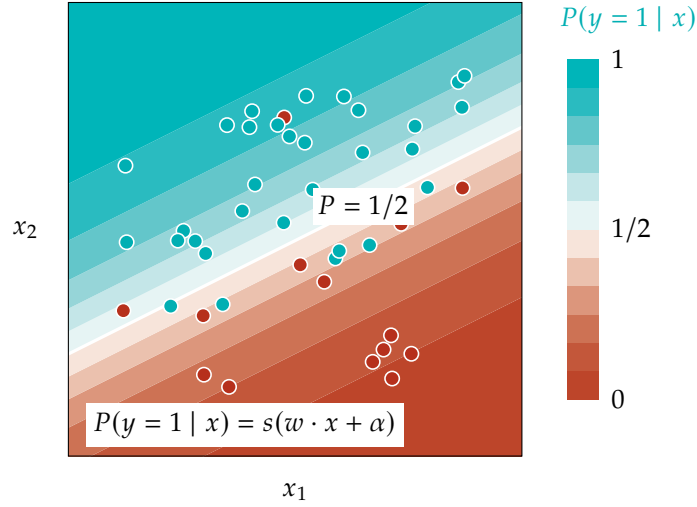


Figure 61: A schematic redraw of a logistic probability map for two features. Straight, parallel contours have different probabilities; the white contour is the $1/2$ classification boundary. The points illustrate a noisy two-class sample.

10.8 Separable data: decreasing cost without a finite minimum

Suppose $y_i \in \{0, 1\}$ and a weight vector w_0 strictly separates the observations, including the intercept coordinate. Then

$$X_i \cdot w_0 > 0 \text{ if } y_i = 1, \quad X_i \cdot w_0 < 0 \text{ if } y_i = 0.$$

Scale the vector by $c > 0$. As $c \rightarrow \infty$,

$$s(cX_i \cdot w_0) \longrightarrow \begin{cases} 1, & y_i = 1, \\ 0, & y_i = 0, \end{cases} \quad J(cw_0) \longrightarrow 0.$$

The separating boundary does not move under positive scaling; the probabilities become more extreme.

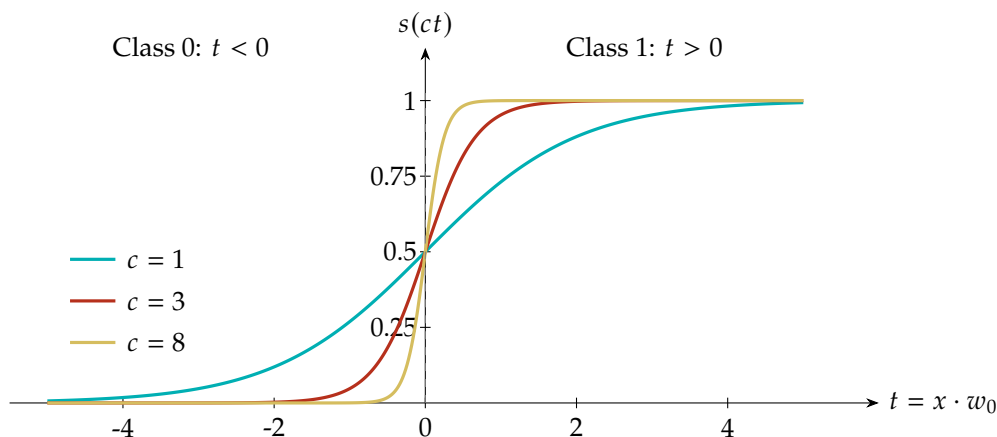


Figure 62: Scaling a separating weight vector sharpens the probability transition while preserving the zero-score boundary. Every finite scale still gives strictly positive loss on binary targets.

Proposition 10.4 [THE INFIMUM NEED NOT BE ATTAINED]. *For a nonempty, strictly linearly separable binary training set, unregularized logistic regression has infimum cost 0 and no finite minimizer.*

Proof. At finite w , each s_i is strictly between 0 and 1. The loss is $-\ln s_i > 0$ for $y_i = 1$ and $-\ln(1-s_i) > 0$ for $y_i = 0$, so $J(w) > 0$. Scaling a strict separator drives every loss to zero, proving that the infimum is 0. If a sequence of weights stayed bounded, a convergent subsequence and continuity of J would give a finite weight vector with cost zero, a contradiction. Thus approaching zero cost requires unbounded weight norms. $\square$

Since the cost is convex, there is no finite local minimum either. A run of gradient descent can keep reducing the cost and find a separator while the weight norm continues to grow. Once every individual binary loss is below $\ln 2$, every observation is on the correct side of the $1/2$ threshold; in particular, $J < \ln 2$ is a sufficient condition because all terms are nonnegative.

For standard gradient descent on separable data, the weight direction can approach the maximum-margin separator asymptotically under the conditions of that convergence result. The approach is extremely slow. Finding a separator reasonably quickly does not make logistic regression a practical way to maximize the margin.

10.9 Revision checklist

Common mistakes.

- Confusing one-point loss with the aggregate cost, or forgetting that summed and averaged gradients have different scales.
- Measuring least-squares residuals perpendicular to the fitted plane instead of in the target coordinate.
- Using $(X^T X)^{-1}$ without checking column independence, or explicitly computing an inverse to solve a linear system.
- Treating $X^+ X = I_p$ as $XX^+ = I_n$. The latter matrix projects onto the column space.
- Adding an extra sigmoid derivative to the simplified logistic gradient, or reversing the sign of the descent update.
- Assuming convexity guarantees a finite minimizer on separable data.

Check yourself.

1. Explain how model, loss, and aggregation distinguish least squares, logistic regression, and least absolute deviations.
2. Starting from $\|Xw - y\|^2$, derive the normal equations and explain why they make the residual orthogonal to every column.
3. State the dimensions of X , w , X^+ , H , and $\hat{y}$. When can a singular H still give a perfect fit?
4. Derive $\nabla_w J = X^T(s(Xw) - y)$ and write both descent updates with the correct sign.
5. Explain how a fixed separating boundary can have decreasing logistic cost while its weight vector has no finite limit.

Source. Jonathan Richard Shewchuk, [CS 189/289A, Spring 2022, Lecture 10: Regression, including Least-Squares Linear and Logistic Regression](#), February 23, 2022, printed pages 53–57. The diagrams are original vector redraws of the regression geometry, loss curves, sigmoid curves, and probability contours, with an additional schematic for the implied projection interpretation. The final asymptotic margin observation is the result summarized in the source’s closing paragraph.

LECTURE 11: NEWTON'S METHOD, ROC CURVES, AND POLYNOMIAL REGRESSION

11.1 Polynomial regression is linear in its coefficients

A straight line or plane cannot fit every relationship. To obtain a polynomial predictor, change the features rather than the fitting procedure. Replace each observation X_i by a feature vector $\Phi(X_i)$ containing all monomials of total degree 0 through r .

For two original coordinates and degree $r = 2$,

$$\Phi(x) = [x_1^2 \quad x_1x_2 \quad x_2^2 \quad x_1 \quad x_2 \quad 1]^\top.$$

Then $w \cdot \Phi(x)$ represents every polynomial of total degree at most two in x_1, x_2 . The constant feature already supplies an intercept; do not append another column of ones.

Definition: LEAST-SQUARES POLYNOMIAL REGRESSION performs linear least squares on polynomial features:

$$\min_w \sum_{i=1}^n (w \cdot \Phi(X_i) - y_i)^2.$$

The predictor is nonlinear in the original coordinates but linear in w . The design matrix now has rows $\Phi(X_i)^\top$, and the normal equations have the same form as before.

The same feature replacement works with logistic regression: $h(x) = s(w \cdot \Phi(x))$. With quadratic features, the score is a quadratic polynomial and its zero set is a quadric decision boundary. For two classes, this has the same functional form as QDA's logistic posterior. Having the same form does not mean that the two training procedures choose the same coefficients.

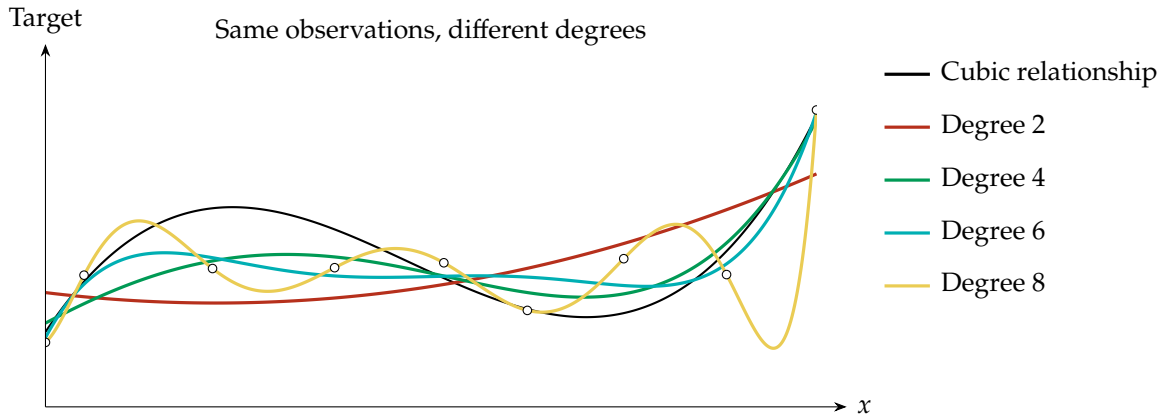


Figure 63: Schematic redraw of fitting noisy samples from a cubic relationship with degrees 2, 4, 6, 8. The degree-2 curve misses the shape; degree 4 follows it reasonably well; higher degrees can follow noise and oscillate between observations. The drawing illustrates the supplied comparison rather than measured values.

11.2 Overfitting, oscillation, and extrapolation

Choosing a polynomial degree is part of choosing model complexity. A low degree can underfit, while a high degree can fit noise. In the cubic-data comparison, degree 4 fits the underlying shape well, whereas degrees 6 and 8 oscillate. A very flexible polynomial needs enough densely spaced observations to constrain its behavior between sample points.

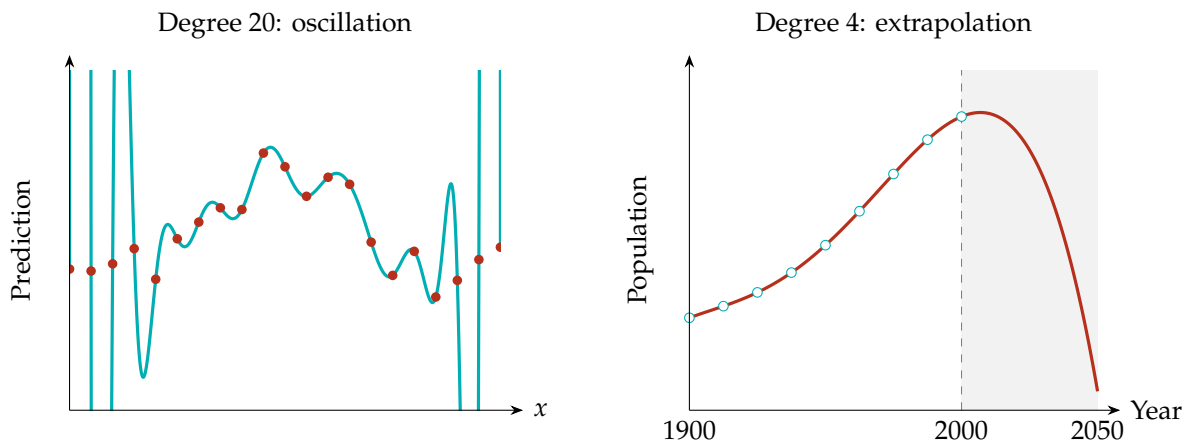


Figure 64: Two schematic redraws of polynomial failure. Left: a degree-20 fit can oscillate wildly between observations. Right: a smooth degree-4 population fit can look plausible over the observed years and predict an implausible decline afterward. The population drawing is qualitative, not census measurements.

INTERPOLATION predicts within the range of the observed inputs. EXTRAPOLATION predicts beyond that range. An extrapolated curve can fail even when it has no visible oscillations in the training interval. The population example makes this distinction clear: a smooth fit can still bend toward an unreasonable future prediction. Extrapolation is generally much harder than interpolation.

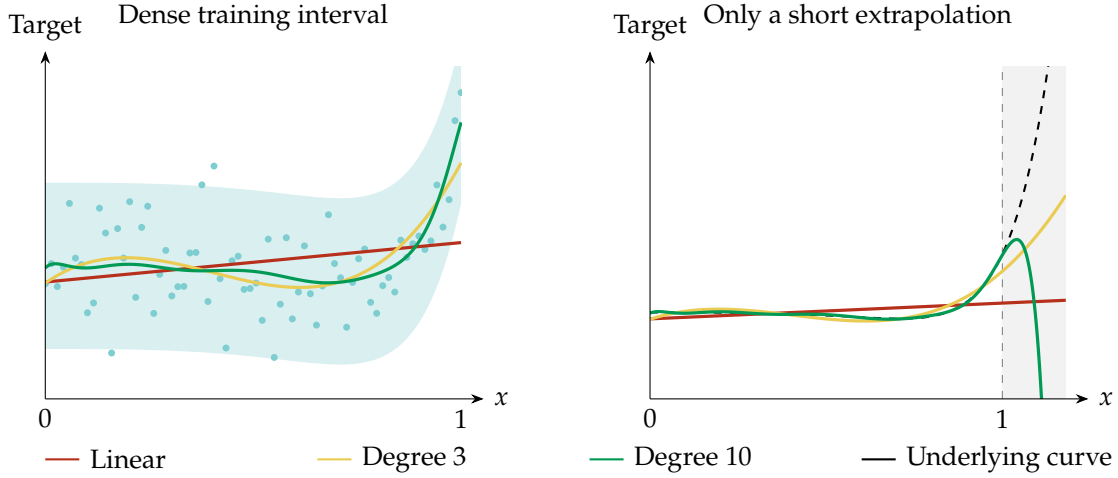


Figure 65: Schematic redraw of a degree-10 example with 10,000 noisy training observations. Dense data can constrain a high-degree fit within the training interval. A short extrapolation can succeed when the underlying relationship also has degree 10, while farther extrapolation can fail. The shaded cloud summarizes training density.

The dense-data example uses a degree-10 generating polynomial and noise with standard deviation 1. The fitted degree-10 polynomial can recover the shape despite the noise, and it extrapolates reasonably for a short distance. That success depends on the matching polynomial model; it does not justify trusting arbitrary high-degree extrapolation.

11.3 Weighted least squares

Some observations are more trustworthy, or more important to fit, than others. Assign larger weights to those observations and smaller weights to suspected outliers. Retain the intercept feature in the design matrix $X \in \mathbb{R}^{n \times p}$, with $w \in \mathbb{R}^p$.

Definition: WEIGHTED LEAST-SQUARES REGRESSION assigns fixed nonnegative weights ω_i and minimizes

$$J(w) = \sum_{i=1}^n \omega_i (X_i \cdot w - y_i)^2 = (Xw - y)^T \Omega (Xw - y), \quad \Omega = \text{diag}(\omega_1, \dots, \omega_n).$$

Here Ω is $n \times n$. A larger ω_i makes the same squared residual more costly. A zero weight removes an observation's contribution to this objective.

Expand and differentiate just as in ordinary least squares:

$$\begin{aligned} J(w) &= w^T X^T \Omega X w - 2y^T \Omega X w + y^T \Omega y, \\ \nabla_w J &= 2X^T \Omega X w - 2X^T \Omega y. \end{aligned}$$

The weighted normal equations are

$$X^T \Omega X w = X^T \Omega y.$$

Solve this linear system rather than explicitly forming an inverse. For any v ,

$$v^T X^T \Omega X v = \sum_{i=1}^n \omega_i (X_i \cdot v)^2 \geq 0.$$

The solution is unique exactly when the positively weighted rows span the coefficient space. In particular, strictly positive weights and full column rank of X give a positive definite system matrix.

11.4 Newton's method uses a local quadratic model

Gradient descent uses the slope and a chosen step size. For a smooth objective, NEWTON'S METHOD also uses curvature. At the current point v , it approximates the objective by a quadratic, moves to that quadratic's critical point, and repeats.

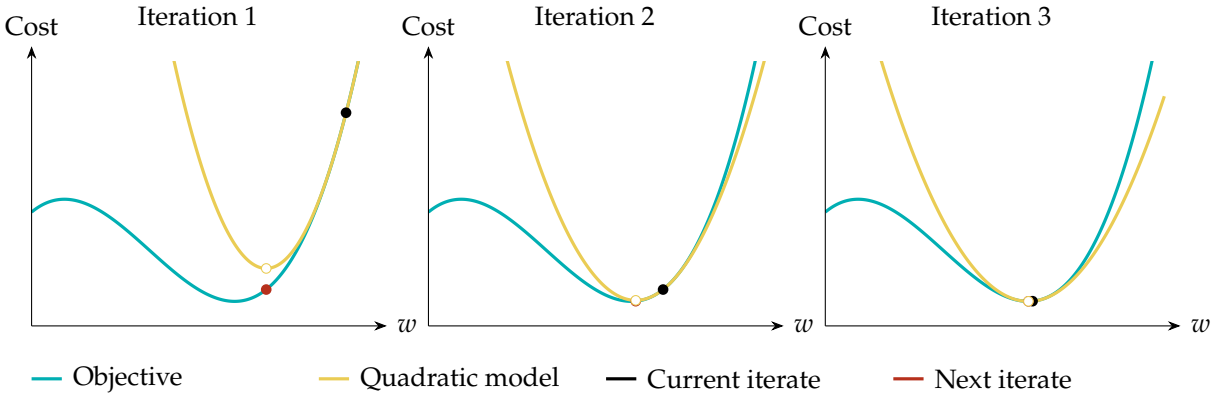


Figure 66: Schematic redraw of three Newton iterations. Each gold parabola matches the blue objective's value, slope, and curvature at the current point. The next iterate is the parabola's critical point, not necessarily the true objective's minimum.

The HESSIAN $B(v) = \nabla^2 J(v)$ is the matrix of second partial derivatives: $B_{jk}(v) = \partial^2 J / \partial w_j \partial w_k$. For a twice continuously differentiable objective it is symmetric. Writing a candidate step as $e = w - v$, the quadratic model is

$$q_v(v + e) = J(v) + \nabla J(v)^T e + \frac{1}{2} e^T B(v) e.$$

Its gradient with respect to e is $\nabla J(v) + B(v)e$. Equivalently, the gradient's Taylor expansion is

$$\nabla J(v + e) = \nabla J(v) + B(v)e + O(\|e\|^2)$$

With the local regularity needed for this remainder, discard the higher-order terms and set the model gradient to zero. This gives

$$B(v)e = -\nabla J(v), \quad v + e = v - B(v)^{-1}\nabla J(v)$$

if the Hessian is invertible. The inverse expression describes the update; computation uses a linear solve.

Algorithm 11.1: Newton's method

Input: A smooth objective J ; a starting point w ; a stopping criterion

1 **while** the stopping criterion is not met **do**

2 $g \leftarrow \nabla J(w)$

3 $B \leftarrow \nabla^2 J(w)$

4 Solve $Be = -g$ for e

5 $w \leftarrow w + e$

Output: The resulting iterate w

For a positive definite Hessian, the quadratic model has a unique minimum. An invertible indefinite Hessian gives a unique critical point that need not be a minimum. A singular Hessian can make the step equation underdetermined or inconsistent. Cholesky factorization or conjugate gradients can solve a positive definite step system without explicitly forming its inverse.

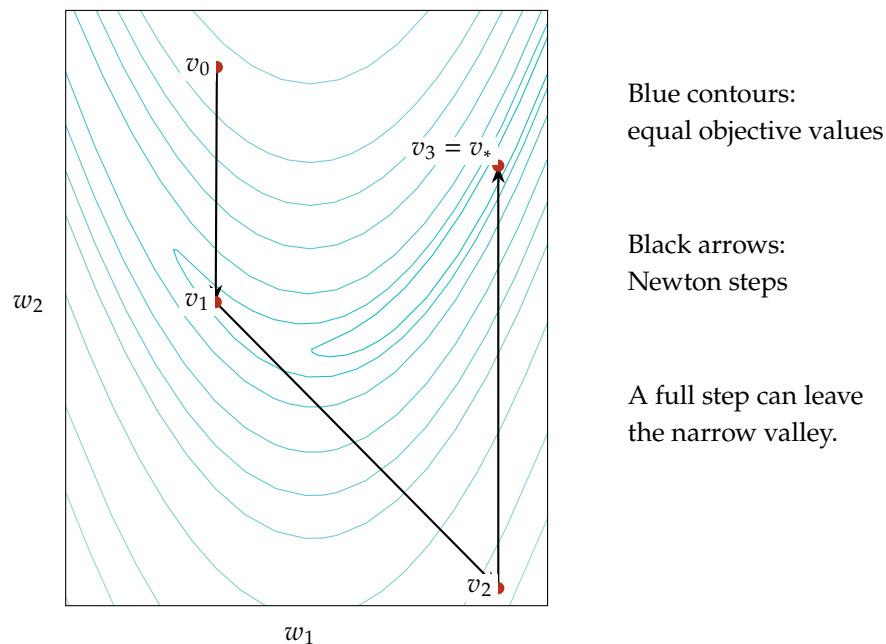


Figure 67: Schematic redraw of Newton steps in a curved two-dimensional valley. Curvature changes both the direction and length of a step. The path can leave the valley before reaching the critical point; Newton's method does not guarantee a decrease at every full step.

11.5 What Newton's method can and cannot guarantee

Proposition 11.1 [ONE STEP FOR AN EXACT QUADRATIC]. *If $J(w) = \frac{1}{2}w^\top Aw + b^\top w + c$, with symmetric invertible A , one Newton step reaches its unique critical point. If A is positive definite, that point is the global minimum.*

Proof. The gradient is $Aw + b$ and the Hessian is A . Solving $Ae = -(Aw + b)$ gives $A(w + e) = -b$, exactly the critical-point equation. If A is positive definite, expanding about that point leaves the nonnegative term $\frac{1}{2}v^\top Av$, which is positive for every nonzero displacement v . $\square$

An objective that is nearly quadratic near the desired solution can converge rapidly. Newton's method chooses a step length from the local model and can choose a better direction than steepest descent. Its starting point must nevertheless be close enough for that model to be useful. The method seeks critical points and does not inherently distinguish minima, maxima, and saddles.

Its main cost is computing and solving with a Hessian at every iteration. This is attractive in smaller coefficient spaces and expensive when there are many weights, as in large neural networks. It also requires suitable smoothness. The perceptron risk is piecewise linear: its Hessian is zero within a smooth piece and undefined at a kink, so this Newton step does not supply a useful update for it.

11.6 Newton's method for logistic regression

Use augmented observations $X_i \in \mathbb{R}^p$, sigmoid $s(t) = 1/(1 + e^{-t})$, and $s_i = s(X_i \cdot w)$. The summed cross-entropy cost has gradient

$$\nabla_w J = X^\top(s - y), \quad s = [s_1 \ \cdots \ s_n]^\top.$$

Since $\nabla_w s_i = s_i(1 - s_i)X_i$, differentiating again gives

$$\nabla_w^2 J(w) = \sum_{i=1}^n s_i(1 - s_i)X_i X_i^\top = X^\top \Omega(w) X, \quad \Omega(w) = \text{diag}(s_i(1 - s_i))_{i=1}^n.$$

This Ω contains changing curvature weights. It is distinct from the fixed observation weights in weighted least squares.

Proposition 11.2 [CURVATURE AND CONVEXITY]. *At every finite w , $\Omega(w)$ is positive definite and the logistic Hessian is positive semidefinite. It is positive definite when X has full column rank.*

Proof. Finite scores give $0 < s_i < 1$, so every diagonal entry $s_i(1 - s_i)$ is positive. For any v ,

$$v^T \nabla^2 J(w) v = \sum_{i=1}^n s_i(1 - s_i)(X_i \cdot v)^2 \geq 0.$$

Equality holds exactly when $Xv = 0$. This proves the rank condition, and the nonnegative curvature in every direction proves convexity. $\square$

The Newton step equation becomes

$$(X^T \Omega X)e = X^T(y - s), \quad w \leftarrow w + e.$$

Both s and Ω must be recomputed from the current w .

Algorithm 11.2: Newton iterations for logistic regression

Input: Augmented design matrix X with full column rank; targets $y \in [0, 1]^n$; a stopping criterion

```

1  $w \leftarrow 0$ 
2 while the stopping criterion is not met do
3    $s \leftarrow s(Xw)$ , applying the sigmoid coordinatewise
4    $\Omega \leftarrow \text{diag}(s_i(1 - s_i))$ 
5   Solve  $(X^T \Omega X)e = X^T(y - s)$  for  $e$ 
6    $w \leftarrow w + e$ 
```

Output: The fitted iterate w

If the iterates converge to a finite stationary point, convexity makes that point globally optimal. Convergence itself is not guaranteed by convexity. In particular, strictly separable binary data have no finite minimizer of the unregularized cost. Even with full column rank, probabilities near 0 or 1 make some curvature weights very small and can make the numerical step difficult.

11.7 Why the Newton solve is called iteratively reweighted least squares

Each logistic Newton iteration solves a system resembling weighted normal equations. Because its weights change with the current fit, the procedure is an example of ITERATIVELY REWEIGHTED LEAST SQUARES (IRLS). The resemblance needs care:

$$\begin{aligned} \text{ordinary weighted fit: } & X^T \Omega X w = X^T \Omega y, \\ \text{logistic Newton step: } & X^T \Omega X e = X^T(y - s). \end{aligned}$$

The Newton right-hand side has no factor Ω multiplying the raw probability residual $y - s$. Small curvature weights therefore do not mean that a point is being declared untrustworthy.

To make the least-squares interpretation explicit, define the correction targets

$$\delta_i = \frac{y_i - s_i}{s_i(1 - s_i)}, \quad \delta = \Omega^{-1}(y - s).$$

The weighted normal equations for fitting e to these targets are

$$X^T \Omega X e = X^T \Omega \delta = X^T (y - s),$$

exactly the Newton step system. Thus the targets change along with the weights. This identity explains the name without changing the algorithm: solve the displayed step system directly.

For binary targets, a confidently wrong prediction has a large $|y_i - s_i|$ and a tiny $s_i(1 - s_i)$, so its correction target has large magnitude. A confidently correct prediction has a small probability residual and contributes little to the gradient. Near the decision boundary, $s_i \approx 1/2$ and the curvature weight is largest. Misclassified points far from the boundary can dominate the correction; if those are absent, points near the boundary tend to be most influential. The exact step also depends on the geometry of all the feature vectors in the linear system.

For a very large n , a speedup is to use a random subsample during each iteration and increase the sample size as the fit progresses. Early iterations need not use every observation to obtain a useful direction; later iterations need more accurate information.

11.8 LDA versus logistic regression

Both methods can produce linear boundaries, but they fit different models. LDA estimates class means, a shared covariance, and priors. Logistic regression fits a sigmoid to the labels directly.

Where LDA has an advantage	Where logistic regression has an advantage
Well-separated classes can give a stable LDA fit, while unregularized logistic coefficients can grow without a finite optimum.	Its probability errors focus fitting on the decision boundary and on badly misclassified observations.
Multiple classes fit naturally into the Gaussian model; binary logistic regression needs a modification such as softmax regression.	It can classify every strictly linearly separable binary training observation correctly by driving the cost sufficiently low.
Nearly Gaussian classes can favor LDA's accuracy, particularly when the training set is small.	Some non-Gaussian distributions, such as strongly skewed ones, can favor direct logistic fitting.
Every observation contributes equally to its class's Gaussian fit.	Targets can naturally be probabilities anywhere in $[0, 1]$.

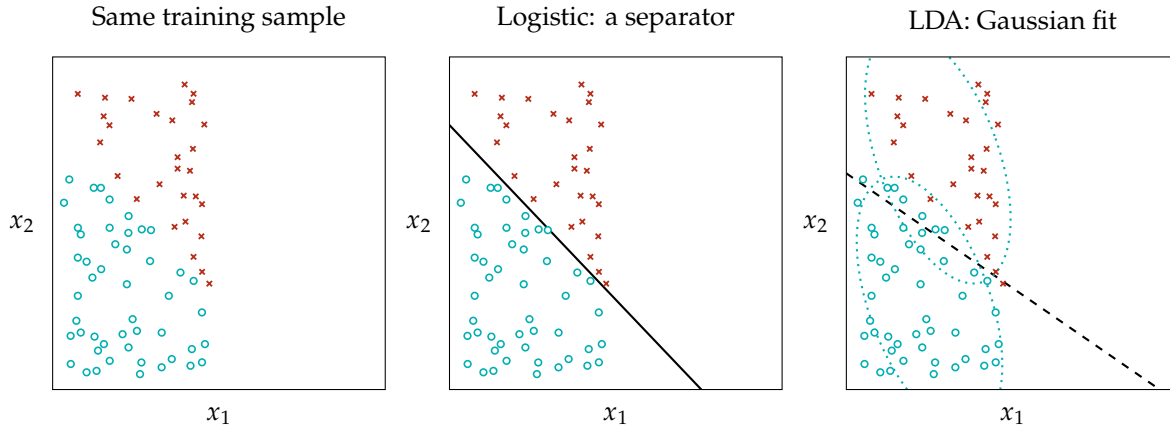


Figure 68: Vector redraw of the supplied narrow-margin, linearly separable sample, with observation locations approximated from the figure. Center: a separating logistic boundary. Right: LDA's Gaussian fit can misclassify training observations even though a linear separator exists. Dotted ellipses show the fitted class geometry.

Driving the logistic cost toward zero on a strictly separable binary sample eventually classifies every observation correctly. LDA has no such training-separation guarantee: it optimizes the Gaussian model fit instead. Logistic regression's emphasis on badly misclassified observations can reduce training error but can also increase sensitivity to bad data. Correctly classified points far from the boundary have little effect on logistic fitting; their effect is still larger than in an SVM once they lie outside its margin.

Adding quadratic features to logistic regression gives a quadric boundary and a sigmoid of a quadratic score, just as in two-class QDA. It generally does not give the same classifier: direct cross-entropy fitting and Gaussian parameter fitting optimize different objectives.

11.9 ROC curves measure a threshold tradeoff

A trained probability predictor gives a score, but a classification decision also needs a threshold. Predict positive when the score is at least τ . Changing τ trades false positives against false negatives without refitting the model.

Definition: A RECEIVER OPERATING CHARACTERISTIC (ROC) curve plots the true positive rate against the false positive rate as the threshold varies, measured on a test or validation set containing both classes:

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \quad \text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

The horizontal axis is the fraction of actual negatives classified positive; the vertical axis is the fraction of actual positives classified positive. Neither axis is the threshold itself.

The true positive rate is also SENSITIVITY. The FALSE NEGATIVE RATE is $1 - \text{TPR}$, while SPECIFICITY, or the true negative rate, is $1 - \text{FPR}$. Each denominator conditions on the actual class, rather than on the predicted class.

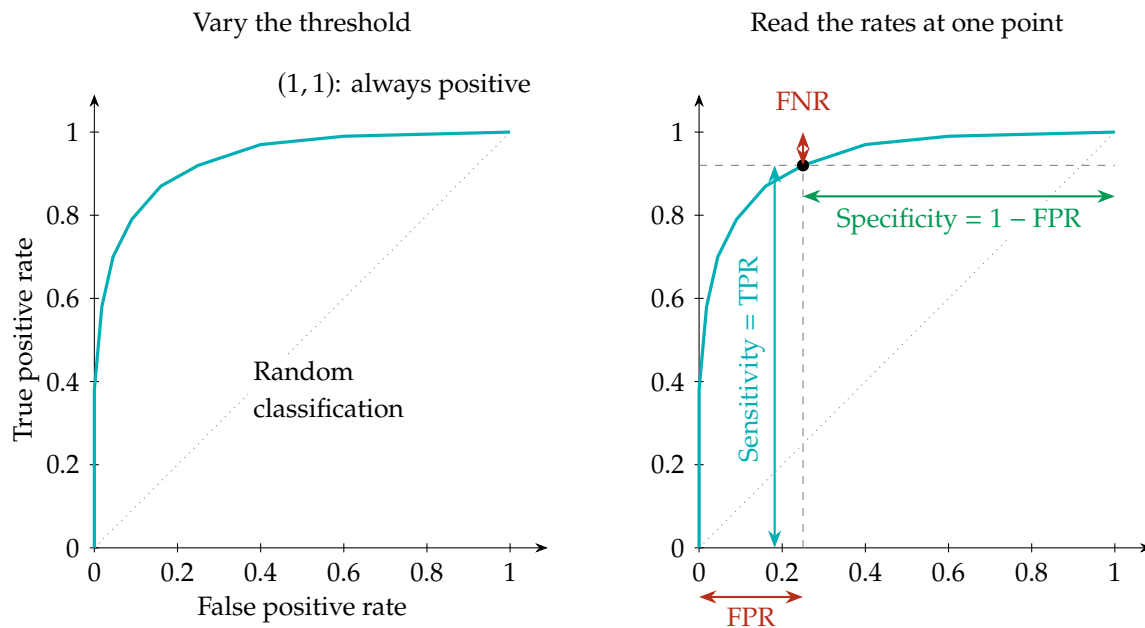


Figure 69: A schematic ROC curve and the rates at one operating point. The vertical gap to the top is the false negative rate; the horizontal gap to the right is specificity. Lowering the probability threshold moves toward predicting more positives.

Algorithm 11.3: Construct an empirical ROC curve

Input: Fixed probability scores and true binary labels on held-out data, with at least one observation of each class

- 1 **for** each distinct score threshold τ , including all-negative and all-positive settings **do**
 - 2 Predict positive exactly when the score is at least τ
 - 3 Count TP, FP, TN, FN
 - 4 Plot $(\text{FP}/(\text{FP} + \text{TN}), \text{TP}/(\text{TP} + \text{FN}))$
-

All observations tied at a threshold change class together under this rule. A threshold greater than 1 predicts every probability score negative, giving (0,0). A threshold of 0 predicts every score positive, giving (1,1). Lowering the threshold cannot decrease either rate, although either one can stay constant.

The diagonal $\text{TPR} = \text{FPR}$ describes the expected performance of random classification independent of the true class. The AREA UNDER THE ROC CURVE (AUC) is a rough summary of discrimination: perfect score ordering gives area 1, and the random diagonal has area $1/2$.

Choose an operating point according to the acceptable tradeoff between false positives and false negatives, using their observed rates on held-out data. A theoretically motivated threshold alone does not establish that the required practical tradeoff has been met.

11.10 Revision checklist

Common mistakes.

- Calling polynomial regression nonlinear in its coefficients, or adding the constant feature twice.
- Trusting a smooth extrapolation because the training fit looks good.
- Forming an inverse instead of solving the weighted or Newton system.
- Assuming a Newton critical point must be a minimum, or that convexity guarantees convergence to finite logistic coefficients.
- Treating logistic curvature weights as fixed reliability weights, or multiplying the Newton right-hand side by Ω again.
- Putting the threshold on a ROC axis, swapping the axes, or using predicted-class totals in the rate denominators.

Check yourself.

1. Write the complete quadratic feature vector for two inputs. Why is its fitting problem still linear least squares?
2. Derive $X^T \Omega X w = X^T \Omega y$ and explain when its solution is unique.
3. Starting from the quadratic model, derive the Newton step. Why does an exact quadratic require only one step?
4. Derive the logistic Hessian and explain why both s and Ω change at each Newton iteration.
5. Explain why LDA and logistic regression can have boundaries of the same form but classify the same training sample differently.
6. Define both ROC axes, sensitivity, specificity, and the two endpoint settings. What practical choice does an operating point make?

Source. Jonathan Richard Shewchuk, [CS 189/289A, Spring 2022, Lecture 11: More Regression; Newton's Method; ROC Curves](#), February 28, 2022, printed pages 58–63. The vector drawings recreate the supplied polynomial comparisons, Newton geometry, classifier comparison, and ROC interpretation. Plots marked schematic preserve the illustrated behavior rather than reproducing empirical values. The weighted-rank argument, exact-quadratic proof, logistic curvature proof, and correction-target identity complete the algebra behind the supplied formulas.